VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 6: Transaction Classes and Verification Planning

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 6 Overview 4

Part 1: Transactions and Transaction Classes 6
- 1.1 What Is a Transaction? 6
- 1.2 Why Do We Need Transaction Classes? 8
- 1.3 Building a Transaction Class: Properties 9
- 1.4 Transaction Methods and the Constructor 13

Part 2: Randomizing and Constraining Transactions 17
- 2.1 Randomizing Transactions 17
- 2.2 Constraint-Based Transactions 20

Part 3: Printing, Copying, Comparing and Reusing Transactions 26
- 3.1 Transaction Printing 26
- 3.2 Transaction Copying 29
- 3.3 Transaction Comparison 33
- 3.4 Transaction Reuse 36
- 3.5 The Complete alu_txn Class 41

Part 4: Verification Planning 45
- 4.1 What Is Verification? 45
- 4.2 What Is a Verification Plan? 47
- 4.3 What Is a Test Plan? 50
- 4.4 How to Derive Test Scenarios from a Specification 52
- 4.5 Directed Testing 54
- 4.6 Constrained-Random Testing 55
- 4.7 Functional Coverage 58
- 4.8 Code Coverage 59
- 4.9 The Self-Checking Testbench 62
- 4.10 Expected Versus Actual Output Comparison 68

Part 5: The Complete Verification Flow, Worked Through for the ALU 73
- 5.1 The Flow at a Glance 73
- 5.2 Step 1: Understand the Specification 74
- 5.3 Step 2: Extract the Features 75
- 5.4 Step 3: The Verification Plan 75
- 5.5 Step 4: Testbench Architecture 76
- 5.6 Step 5: The Test Plan 77
- 5.7 Step 6: The Coverage Plan and the Assertion List 78
- 5.8 Step 7: Build, Bring Up, Run and Debug 79
- 5.9 Step 8: Regression 81
- 5.10 Step 9: Coverage Closure 82
- 5.11 Step 10: Sign-Off, or What "Done" Means 83

Part 6: Module 6 Revision and Practice 86
- 6.1 Revision Summary 86
- 6.2 Interview Questions and Answers 88
- 6.3 Output-Prediction Questions 91
- 6.4 Find-the-Error and Debugging Questions 95
- 6.5 Coding Exercises with Solutions 98

6.6 Concepts You Must Understand Before Moving Forward 105
6.7 What Comes Next 106

Module 6 Overview

This module has two halves that belong together.

The first half is about the **transaction class**: the object that carries one unit of stimulus or response through a class-based testbench. You already know every language feature it uses. Module 4 taught classes, handles and copying; Module 5 taught `rand`, constraints and `randomize()`. Here those features are assembled, step by step, into `alu_txn`, the transaction used by every component of the layered testbench in Module 9.

The second half is about **verification planning**: what verification is, how a specification is turned into a list of things to check, how tests and coverage are planned, how a testbench checks itself, and how a project decides that verification is finished. Interviewers for Design Verification roles ask about this at least as much as they ask about syntax, because it shows whether a candidate understands the job.

What you will be able to do after this module

- Explain what a transaction is and why testbenches work with transactions instead of pin values.
- Write a complete transaction class with properties, a constructor, constraints, and `print`, `copy`, `compare` and `convert2string` methods.
- Reuse a transaction through inheritance to create test-specific stimulus without editing the testbench.
- Explain verification, the verification plan, the test plan, directed and constrained-random testing, code and functional coverage, and the self-checking testbench.
- Take a specification (the 8-bit ALU) and produce a feature list, a verification plan, a test plan, a coverage plan and sign-off criteria.

What you need from earlier modules

From	You must already know
Module 1	Two-state and four-state types, `enum`, `==` versus `===`, the ALU design (`alu_pkg`, `alu`)
Module 2	Queues and `foreach` (used to hold lists of transactions)
Module 3	Functions, tasks, default arguments, `automatic` lifetime
Module 4	Classes, `new()`, handles, `null`, shallow and deep copy, inheritance, `virtual` methods
Module 5	`rand`, constraint blocks, `dist`, `randomize()`, `randomize() with`, `constraint_mode()`

What comes later

Topic touched here	Covered fully in
Passing transactions between components with a mailbox	Module 7
Driving and sampling DUT pins through `alu_if` and `vif`	Module 8
Generator, driver, monitor, scoreboard, environment, test	Module 9
Covergroups and assertions	Module 10

About the examples. The examples use the ALU package from Module 1. A copy of `alu_pkg.sv` (and of `alu.sv`, needed by one example) is in `examples/module-06/` so that the folder is complete by itself. Every `tb_*.sv` file in this module needs `alu_pkg.sv` compiled before it.

> **Note:** As in the other modules, the "Expected output" blocks were derived by hand from the IEEE 1800 rules and were not captured from a simulator. Where an example uses randomization, the exact values cannot be predicted; such output is labelled as a sample, and the example prints at least one line that is the same on every run.

alu_pkg.sv

```
package alu_pkg;

  typedef enum logic [2:0] {
    ALU_ADD = 3'b000,
    ALU_SUB = 3'b001,
    ALU_AND = 3'b010,
    ALU_OR  = 3'b011,
    ALU_XOR = 3'b100,
    ALU_NOT = 3'b101,
    ALU_SHL = 3'b110,
    ALU_SHR = 3'b111
  } alu_op_e;

endpackage
```

Part 1: Transactions and Transaction Classes

1.1 What Is a Transaction?

In plain English

Look at the ALU from the outside. To make it do one addition, the testbench must put a value on `a`, a value on `b`, the code for "add" on `op`, raise `valid_in`, wait for a clock edge, lower `valid_in`, and one cycle later read `result` while `valid_out` is high. That is seven signal-level actions spread over two clock cycles.

But in your head the whole thing is one idea: **"add 200 and 100, expect 300"**.

A **transaction** is that one idea, written down as data. It records *what* is to be done (operation, operands) and *what came back* (result). It does not record *how* the pins must wiggle or *when*. The "how" and "when" belong to the driver and the monitor.

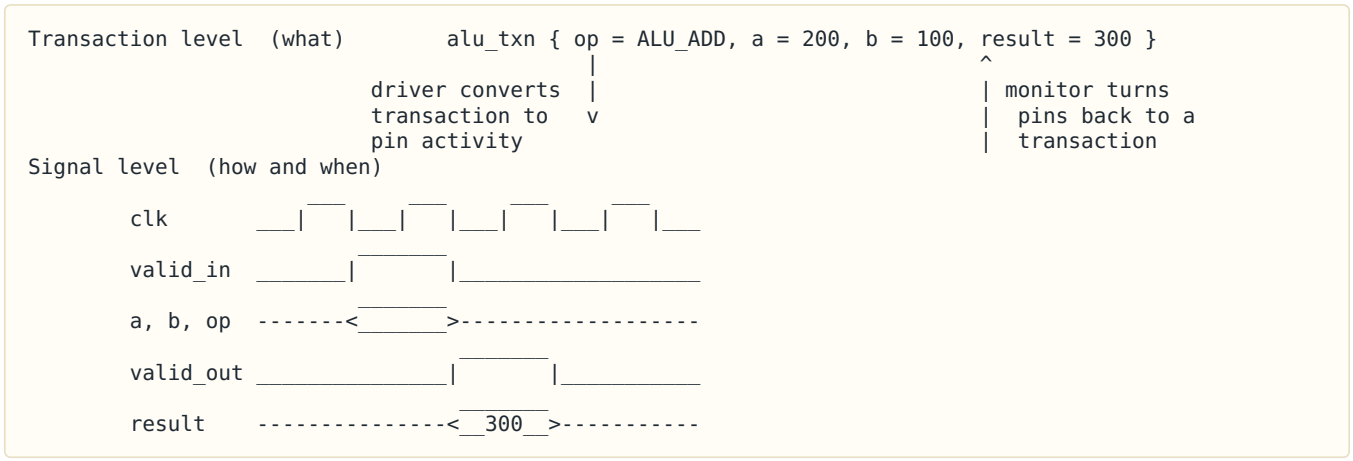
Technical definition

A transaction is an abstract, pin-independent and time-independent description of one complete, meaningful unit of activity on an interface. In a SystemVerilog testbench it is an object of a **transaction class**, whose properties are the fields of that activity and whose methods operate on those fields.

Other names for the same thing: *sequence item* (UVM), *packet*, *data item*, *transfer*, *bus item*.

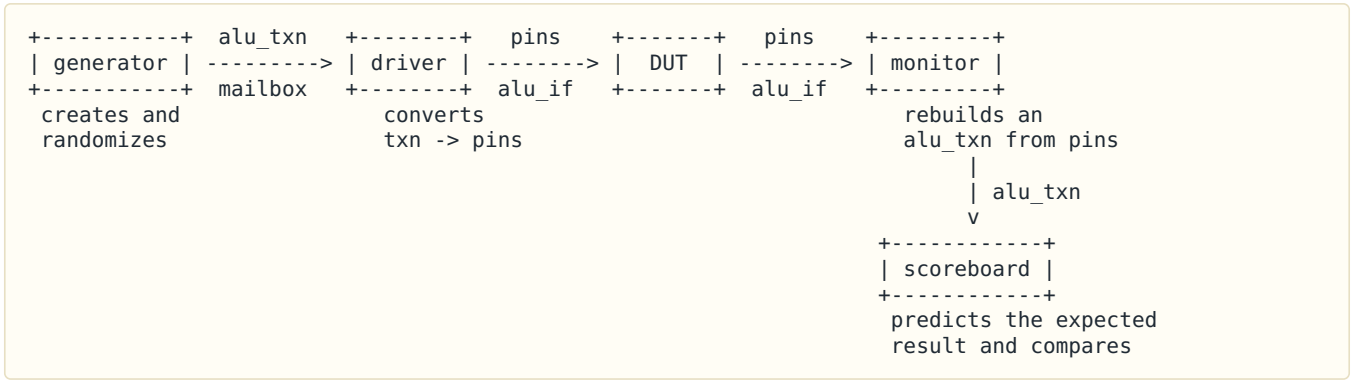
Design	One transaction is	Typical fields
8-bit ALU (this course)	One operation	`op`, `a`, `b`, `result`
Synchronous FIFO	One write or one read	`kind` (write/read), `data`
Memory or register bus (APB, AXI-Lite)	One read or write transfer	`addr`, `data`, `write`, `strobe`, `response`
AXI	One burst	`addr`, `len`, `size`, `burst`, a queue of data beats, `id`
Ethernet	One frame	`dst`, `src`, `type`, payload bytes, `crc`
UART	One character	`data`, `parity_error`, `framing_error`
Processor	One instruction	`opcode`, register numbers, immediate

Levels of abstraction



Everything above the line is easy to generate, print, copy, compare and reason about. Everything below the line is detail that only two components (driver and monitor) need to know.

Where transactions travel in a testbench



Notice that the transaction class is the only type that **every** component uses. That is why it is designed first and designed carefully.

Interview questions

Q1. What is a transaction? A high-level, pin-independent description of one complete operation on an interface: for example one ALU operation, one bus read or write, or one packet. In a class-based testbench it is an object whose properties are the fields of that operation.

Q2. What is the difference between transaction-level and signal-level? Signal level describes individual pin values at individual clock edges. Transaction level describes the operation as a whole (operands, command, result) without timing. The driver converts from transaction level to signal level, and the monitor converts back.

Q3. Give an example of a transaction for a memory. One write or read: `rand bit write; rand bit [15:0] addr; rand bit [31:0] wdata; logic [31:0] rdata;`.

Q4. Does a transaction contain clock and reset? No. Clock, reset and handshake timing are signal-level concepts that live in the interface and are handled by the driver and monitor. A transaction may contain *timing intent* as data, for example `rand int unsigned delay;` meaning "wait this many idle cycles before driving me", but not the clock itself.

1.2 Why Do We Need Transaction Classes?

The testbench without transactions

In a Verilog-style testbench, stimulus is written directly on the pins:

```
@(negedge clk); a = 8'd200; b = 8'd100; op = ALU_ADD; valid_in = 1;
@(negedge clk); valid_in = 0;
if (result !== 9'd300) $error("add failed");

@(negedge clk); a = 8'd5;   b = 8'd10;  op = ALU_SUB; valid_in = 1;
@(negedge clk); valid_in = 0;
if (result !== 9'h1FB) $error("sub failed");
// ... and so on, three lines per operation, for every operation you can think of
```

Problems with this style:

Problem	Consequence
What to send and how to send it are mixed in the same lines	A change in the protocol (say, `valid_in` must be held for two cycles) means editing every test
Every value is typed by hand	Only the cases the engineer thought of are tested
Nothing can be randomized as a unit	There is no object on which to call `randomize()` or write constraints
The expected value is typed by hand	Slow, error-prone, impossible for thousands of operations
There is no object to pass around	Generation, driving, monitoring and checking cannot be split into separate, reusable components
Nothing can be printed or counted as an operation	Debugging means reading waveforms

What the transaction class gives you

Need	How a transaction class provides it
Separate "what" from "how"	The class holds only the fields. Pin-level protocol lives in one place, the driver
Random stimulus	Fields are declared `rand`; one call to `randomize()` produces a complete legal operation
Legal stimulus	Constraint blocks inside the class define what is legal and what is interesting
Communication	A handle to the object is passed through a mailbox from one component to the next
Automatic checking	The scoreboard receives an object containing inputs and actual result, computes the expected result from the inputs, and compares
Debug	A `print()` method shows the whole operation in one line of the log
Reuse	A test extends the class and adds constraints; the generator, driver, monitor and scoreboard are untouched
Coverage	A covergroup samples the fields of the object

Why a class and not a struct?

Module 1 showed that a `struct` can also group fields. The comparison explains why testbenches use a class:

Requirement of a transaction	`struct`	`class`
Group the fields under one name	Yes	Yes
`rand` fields with a `randomize()` method	No method of its own	Yes
Constraint blocks	No	Yes
Methods (`print` , `copy` , `compare`) kept with the data	No	Yes
Extend with new fields or constraints without editing the original	No	Yes, by inheritance
Replace by a specialised version at run time	No	Yes, by polymorphism
Created on demand, any number, during simulation	Fixed at declaration	Yes, with `new()`
Passed between components cheaply	The whole struct is copied	Only the handle is copied
Synthesizable	Yes	No (not needed in a testbench)

Structs remain the right choice for the DUT side (packed bit fields on ports). The testbench side uses classes.

> **How to say it in an interview:** "A transaction class raises the testbench from signal level to transaction level. It bundles the fields of one operation, lets me randomize them under constraints, gives me one object to pass from generator to driver and from monitor to scoreboard, and keeps the print, copy and compare routines with the data. Because it is a class, a test can extend it to change the stimulus without touching the rest of the environment."

Interview questions

Q1. Why do we need a transaction class? To separate what is being tested from how it is driven on the pins; to allow constrained-random generation of whole operations; to give the testbench components a common object to exchange; and to make stimulus and results easy to print, copy, compare, cover and reuse.

Q2. Why is a transaction a class and not a struct? A class supports `randomize()` with constraint blocks, methods, inheritance and polymorphism, and dynamic creation. A struct supports none of these, so stimulus could not be constrained, extended per test, or overridden.

Q3. Which testbench components use the transaction? All of them. The generator creates and randomizes it, the driver converts it to pin activity, the monitor rebuilds it from pin activity, the scoreboard compares it, and the coverage collector samples it.

Q4. If the pin-level protocol of the DUT changes but its function does not, which parts of the testbench change? Only the driver and the monitor (and the interface). The transaction class, generator, scoreboard and tests are unaffected. This is the practical benefit of the separation.

1.3 Building a Transaction Class: Properties

How to decide which fields a transaction needs

Start from the DUT's port list and its specification, and sort every signal into one of four groups:

Group	Question to ask	In the transaction?	ALU
Stimulus data	Does the testbench choose this value for each operation?	Yes, as a `rand` field	`a`, `b`, `op`
Response data	Does the DUT produce this value for each operation?	Yes, as a non-`rand` field	`result`
Protocol and timing	Is this a handshake, clock or reset signal?	No. The driver and monitor handle it	`clk`, `rst_n`, `valid_in`, `valid_out`
Bookkeeping	Does the testbench need it for tracking or debug?	Yes, as a non-`rand` field	`id`

Choosing the type of each field

This is a direct application of Module 1.

Field	Declaration	Why this type
Operands	`rand bit [7:0] a, b;`	The testbench chooses them. The solver only produces 0 and 1, so two-state is correct and faster. Unsigned, exactly the port width
Operation	`rand alu_op_e op;`	Using the enum means the solver can only pick one of the eight legal names, logs print `ALU_ADD` instead of `0`, and coverage gets one bin per name
Result	`logic [8:0] result;`	It is sampled from the DUT. If the DUT drives X, the transaction must hold X so that the scoreboard can report it. Therefore four-state. Not `rand`: the testbench does not choose it
Identifier	`int unsigned id;`	A serial number so that a line in the log can be matched to a transaction. Not `rand`

> **Interview trap:** "Why is `result` declared `logic` when `a` and `b` are `bit`?" Because of where the value comes from. `a` and `b` are generated by the testbench and can never be X. `result` is copied from a DUT pin by the monitor; a two-state field would silently turn an X output into 0 and the comparison could pass. Fields that hold DUT outputs are four-state.

The first version of the class

tb_txn_basic.sv

```
module tb_txn_basic;
  import alu_pkg::*;

  class alu_txn;
    // ---- stimulus: chosen by the testbench, therefore rand ----
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand alu_op_e  op;

    // ---- response: sampled from the DUT, therefore four-state, not rand ----
    logic [8:0]    result;

    // ---- bookkeeping ----
    int unsigned   id;
  endclass

  alu_txn t1, t2;

  initial begin
    t1 = new();
    t2 = new();

    t1.id = 1;
    t1.a  = 8'd200;
    t1.b  = 8'd100;
    t1.op = ALU_ADD;

    $display("t1: id=%0d a=%0d b=%0d op='%s' result=%h",
             t1.id, t1.a, t1.b, t1.op.name(), t1.result);
    $display("t2: id=%0d a=%0d b=%0d op='%s' result=%h",
             t2.id, t2.a, t2.b, t2.op.name(), t2.result);
  end

endmodule
```

EXPECTED OUTPUT

```
t1: id=1 a=200 b=100 op='ALU_ADD' result=xxx
t2: id=0 a=0 b=0 op='' result=xxx
```

Line by line

Line	Explanation
`import alu_pkg::*;`	Makes `alu_op_e` and the opcode names visible. The transaction uses the same type as the DUT port, so the two can never disagree about encodings.
`class alu_txn;`	For teaching, the class is declared inside the demo module. In the real testbench it lives in the package `alu_tb_pkg` (Section 3.5).
`rand bit [7:0] a;`	`rand` marks the field as one that `randomize()` may change. Until `randomize()` is called, a `rand` field is an ordinary variable with its default value.
`logic [8:0] result;`	Nine bits, like the DUT output. Four-state.
`t1 = new();`	Creates an object. No constructor was written, so the built-in default constructor is used: it allocates the object and gives every property its default value.
`t1.a = 8'd200;`	A `rand` field can also be assigned by hand. This is how directed tests build transactions.
Output line 1	`result` was never assigned, so all nine bits are X; `%h` prints three hex digits `xxx`.

Line	Explanation
Output line 2	An untouched object. `a`, `b` and `id` are two-state and start at 0. `op` is an enum whose base type is `logic [2:0]`, so it starts at X, which is not one of the eight members; `name()` returns an empty string.

> **Common mistake:** Assuming a freshly constructed transaction contains a valid opcode. Because `alu_op_e` has a four-state base type, `op` is X until it is assigned or randomized. A driver that puts such a transaction on the pins drives X into the DUT. Always randomize or assign every stimulus field before use, or initialise `op` in the constructor.

What should not be in a transaction

Do not put this in the transaction	Reason
The virtual interface handle	The transaction would then depend on the pins, which is exactly what it exists to avoid
Clock or reset values	They are not per-operation data
Tasks that consume time (`@`, `#`)	A transaction is data. Timing belongs to the driver and monitor
Mailboxes, events, handles to components	A transaction is passed between components; it should not know about them
Large data that is never used	Thousands of transaction objects may be alive at once

Common additional fields in real projects

Field	Purpose
`rand int unsigned delay;`	Number of idle cycles the driver inserts before this transaction. Randomizing it finds timing-dependent bugs
`time start_time, end_time;`	Filled in by the monitor; used for latency checks and debug
`rand kind_e kind;`	Type of operation, when one class models several kinds of transfer
`bit error;` or a response enum	Status returned by the DUT
`rand bit inject_error;`	Tells the driver to corrupt this transaction on purpose (bad parity, bad CRC) to test error handling

Interview questions

Q1. How do you decide which signals become fields of the transaction? Data that the testbench chooses per operation becomes a `rand` field. Data that the DUT returns per operation becomes a non-`rand` field. Clock, reset and handshake signals are protocol and stay in the interface, driver and monitor. Tracking information such as an ID is added as non-`rand` bookkeeping.

Q2. Which fields are `rand` and which are not? Stimulus fields are `rand`. Response fields, IDs, timestamps and anything computed are not.

Q3. Why use an enum for the opcode instead of `bit [2:0]`? Readable logs and waveforms; the solver generates only declared members, which matters when not all encodings are legal; coverage bins are created per name automatically; and the compiler rejects an assignment of an arbitrary number.

Q4. What does a newly constructed `alu_txn` contain? The default value of each property: 0 for the two-state fields `a`, `b`, `id`; X for the four-state `result`; and X for `op`, because the enum's base type is four-state.

Q5. Should a transaction contain a virtual interface? No. The transaction must stay independent of pins and timing. Only the driver and monitor hold the virtual interface.

1.4 Transaction Methods and the Constructor

The standard set of methods

A transaction is more than a record. Each testbench component needs a particular service from it, and that service is provided as a method of the class so that it is written once and kept next to the fields it handles.

Method	Purpose	Called by	Section
`new()`	Create the object and set initial values	Generator, monitor	1.4
`randomize()` (built in)	Give the `rand` fields legal random values	Generator	2.1
`pre_randomize()`, `post_randomize()` (built-in hooks)	Prepare before, or compute derived fields after, randomization	Called automatically by `randomize()`	2.2
`convert2string()`	Return the contents as one formatted string	`print()`, error messages	3.1
`print(tag)`	Display the contents in the log	Every component	3.1
`copy()`	Return a new, independent object with the same field values	Generator, monitor, scoreboard	3.2
`compare(rhs)`	Return 1 if two transactions have the same contents	Scoreboard	3.3
`pack()` / `unpack()` (optional)	Convert to and from a flat bit vector	Driver and monitor of serial or bus protocols	Revision exercises

The names `copy`, `compare`, `print` and `convert2string` are a deliberate choice. UVM's base class for transactions provides methods with exactly these names, so a hand-written class that uses them is immediately familiar to anyone who knows UVM, and learning UVM later is easier.

All of these are **functions**, not tasks: none of them consumes simulation time. Recall from Module 3 that a function must return in zero time; a method that only manipulates fields has no reason to wait.

The constructor

Definition. The constructor is the special function `new()`. It runs automatically when an object is created, has no return type, and is used to put the object into a defined initial state (Module 4).

For a transaction the constructor is usually very small, because the stimulus fields will be set by `randomize()` anyway. Three styles are found in practice:

Style 1: no constructor. The built-in default constructor is used. Acceptable when every field's default value is acceptable.

Style 2: a constructor with no arguments that sets safe initial values.

```
function new();
  op     = ALU_ADD;     // a legal opcode instead of the default X
  result = 'x;          // "no result captured yet"
endfunction
```

Style 3: a constructor with default arguments, convenient for directed tests.

```
function new(alu_op_e op = ALU_ADD, bit [7:0] a = 0, bit [7:0] b = 0);
  this.op = op;         // this.op is the property; op is the argument
  this.a  = a;
  this.b  = b;
endfunction

// usage
t = new();                          // ALU_ADD, 0, 0
t = new(ALU_SUB, 8'd5, 8'd10);      // a directed transaction in one line
```

`this` is needed because the arguments have the same names as the properties (Module 4).

> **Key point:** Give every constructor argument a default value. A class that can be constructed as `new()` with no arguments can be created by generic code, such as a parameterized generator or a factory, that knows nothing about its fields. UVM requires this.

Assigning the identifier

There are two common ways to give each transaction its `id`.

Method	Code	Remarks
The creator assigns it	`t.id = count++;` in the generator	Simple and explicit. Used in this course
A static counter in the class	`static int unsigned next_id; function new(); id = next_id++; endfunction`	Automatic, but every `new()` consumes a number, including the `new()` inside `copy()` and the objects built by the monitor, so the numbers no longer mean "the n-th generated transaction" unless the code compensates

Complete example

tb_txn_constructor.sv

```
module tb_txn_constructor;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    function new(alu_op_e op = ALU_ADD, bit [7:0] a = 0, bit [7:0] b = 0);
      this.op = op;
      this.a  = a;
      this.b  = b;
    endfunction

    function bit is_unary();
      return op inside {ALU_NOT, ALU_SHL, ALU_SHR};
    endfunction
  endclass

  alu_txn t;

  initial begin
    t = new();
    $display("1. op=%s a=%0d b=%0d unary=%0d", t.op.name(), t.a, t.b, t.is_unary());

    t = new(ALU_SUB, 8'd5, 8'd10);
    $display("2. op=%s a=%0d b=%0d unary=%0d", t.op.name(), t.a, t.b, t.is_unary());

    t = new(ALU_NOT, 8'h0F);
    $display("3. op=%s a=%0d b=%0d unary=%0d", t.op.name(), t.a, t.b, t.is_unary());
  end

endmodule
```

EXPECTED OUTPUT

```
1. op=ALU_ADD a=0 b=0 unary=0
2. op=ALU_SUB a=5 b=10 unary=0
3. op=ALU_NOT a=15 b=0 unary=1
```

Line by line

Line	Explanation
`function new(alu_op_e op = ALU_ADD, ...)`	A custom constructor. Every argument has a default, so `new()` is still legal.
`this.op = op;`	Left side: the property of the object. Right side: the argument. Without `this`, the statement would assign the argument to itself and the property would stay X.
`function bit is_unary();`	A small helper method. It answers a question about the transaction ("does this operation ignore `b`?") so that the driver, scoreboard and coverage do not each repeat the list of unary opcodes.
`t = new();`	All defaults: a legal opcode and zero operands.
`t = new(ALU_SUB, 8'd5, 8'd10);`	A directed transaction. The handle `t` now refers to a new object; the previous object has no handle left and its memory is reclaimed automatically.
`t = new(ALU_NOT, 8'h0F);`	Trailing arguments may be omitted; `b` takes its default of 0.

> **Common mistake:** Writing `function void new()` or `function alu_txn new()`. A constructor has no return type at all; either form is a compile error.

> **Common mistake:** Doing real work in the constructor, such as calling `randomize()` inside `new()`. It hides the randomization from the code that creates the object, makes it impossible to check the return value of `randomize()` properly, and prevents a test from adding inline constraints. Construct first, then randomize.

Interview questions

Q1. Which methods does a typical transaction class contain? A constructor, `print` (often built on `convert2string`), `copy`, `compare`, and sometimes `pack` / `unpack`, plus the built-in `randomize()`, `pre_randomize()` and `post_randomize()`.

Q2. Why are transaction methods functions and not tasks? They only read and write fields and return immediately. Declaring them as functions guarantees that they cannot block, and lets them be called from other functions, from constraints' helper code and from `$display` argument lists.

Q3. What is the role of the constructor in a transaction class? To leave the new object in a known, safe state: for example a legal opcode instead of X. It should not randomize or perform time-consuming actions.

Q4. Why should constructor arguments have default values? So that the object can be created by code that does not know the argument list, such as a generic generator, and so that a derived class does not have to pass arguments through `super.new()`.

Q5. Why does the course call the methods `copy`, `compare` and `print`? These are the conventional names, and the same names are used by UVM's transaction base class, so the habit transfers directly.

Part 2: Randomizing and Constraining Transactions

2.1 Randomizing Transactions

In plain English

A directed test fills in the transaction by hand: "add 200 and 100". That tests one case. To test thousands of cases without writing thousands of lines, the testbench asks the simulator to fill in the fields with random values. Because the stimulus fields were declared `rand`, one method call does it:

```
t = new();
if (!t.randomize()) $fatal(1, "randomize() failed");
```

After the call, `a`, `b` and `op` hold a new random combination. Call it again and they hold another.

Technical definition

Every class has a built-in virtual method `randomize()`. When it is called on an object, the constraint solver finds values for all active `rand` and `randc` properties of that object such that every active constraint is satisfied, and writes those values into the object. It returns 1 on success and 0 if no solution exists. Module 5 covers the mechanism in depth; this section applies it to the transaction.

What happens inside one call

```
t.randomize()
     |
     v
1. pre_randomize() is called (does nothing unless you defined it)
     |
     v
2. The solver collects: the rand fields that are switched on  (a, b, op)
                        the constraint blocks that are switched on
                        any inline constraints given with "with { ... }"
     |
     v
3. Is there at least one combination of values that satisfies everything?
     |                                    |
    yes                                   no
     |                                    |
     v                                    v
4. One solution is chosen at         The object is left unchanged.
   random and written into           randomize() returns 0.
   a, b, op.                         post_randomize() is NOT called.
     |
     v
5. post_randomize() is called
     |
     v
6. randomize() returns 1

Fields that are not rand (result, id) are never modified.
```

Rules that matter for transactions

Rule	Consequence in a testbench
Only `rand` / `randc` fields are changed	`result` and `id` keep their values across `randomize()`
The return value reports failure	It must always be checked. An unchecked failure leaves the previous values in place, and the test silently repeats the same transaction or drives default values
An enum field receives only declared members	`op` can never be an illegal encoding
Four-state `rand` fields receive only 0 and 1	Random stimulus never contains X or Z
With no constraints, every value is equally likely	`a` and `b` are uniform over 0 to 255, `op` uniform over the eight operations
The same seed gives the same sequence of values	A failing test can be re-run with its seed to reproduce the failure exactly

Ways to check the return value

```
// 1. if + fatal: simple, always active, the form used in this course
if (!t.randomize()) $fatal(1, "randomize() failed");

// 2. immediate assertion: shorter, but see the trap below
assert (t.randomize()) else $fatal(1, "randomize() failed");

// 3. cast to void: compiles without warning but HIDES failures. Do not use.
void'(t.randomize());
```

Interview trap: "What is wrong with `assert(t.randomize());`?" Nothing while assertions are enabled. But assertions can be switched off globally (with `$assertoff` or a simulator option), and some tools then do not evaluate the expression at all, so `randomize()` is never called and the test runs with unrandomized transactions. The `if (!t.randomize())` form has no such risk.

New object or the same object each time?

A generator can call `randomize()` repeatedly on one object, or create a new object for each transaction. The difference matters as soon as the handle is passed to another component.

Approach	Code	Safe?
New object per transaction	`t = new(); t.randomize(); send(t);`	Yes. Every receiver gets its own object
One object, send a copy	`bp.randomize(); send(bp.copy());`	Yes. The "blueprint" pattern of Section 3.4
One object, send the handle	`t.randomize(); send(t);` in a loop	**No.** Every receiver holds a handle to the same object, whose fields keep changing. Section 3.2 demonstrates the bug

Complete example

tb_txn_randomize.sv

```
module tb_txn_randomize;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    function void print(string tag = "");
      $display("%s: id=%0d op=%s a=0x%h b=0x%h", tag, id, op.name(), a, b);
    endfunction
  endclass

  alu_txn      t;
  int unsigned n_legal_op, n_id_kept;

  initial begin
    // 1. Four random transactions, a new object for each
    for (int i = 0; i < 4; i++) begin
      t    = new();
      t.id = i;
      if (!t.randomize()) $fatal(1, "randomize() failed");
      t.print("RAND");
    end

    // 2. Properties that hold on every run, checked over 500 randomizations
    t    = new();
    t.id = 77;
    repeat (500) begin
      if (!t.randomize()) $fatal(1, "randomize() failed");
      if (t.op.name() != "") n_legal_op++;     // op is always a declared member
      if (t.id == 77)        n_id_kept++;      // a non-rand field is never touched
    end
    $display("legal opcode in %0d of 500, id unchanged in %0d of 500",
             n_legal_op, n_id_kept);
  end

endmodule
```

Sample output; actual values in the first four lines differ per seed and simulator. The last line is the same on every run.

EXPECTED OUTPUT

```
RAND: id=0 op=ALU_XOR a=0x3d b=0xb1
RAND: id=1 op=ALU_ADD a=0xe6 b=0x07
RAND: id=2 op=ALU_SHR a=0x58 b=0xfa
RAND: id=3 op=ALU_SUB a=0x91 b=0x2c
legal opcode in 500 of 500, id unchanged in 500 of 500
```

Line by line

Line	Explanation
`t = new(); t.id = i;`	A new object for each transaction; the creator assigns the serial number.
`if (!t.randomize()) $fatal(...)`	Randomize and check. `$fatal` stops the simulation: continuing after a randomization failure produces meaningless results.
`t.print("RAND");`	One log line per transaction. `op.name()` prints the opcode by name.

Line	Explanation
`if (t.op.name() != "")`	`name()` returns an empty string for a value that is not a member. It never does here, because the solver only chooses declared members of an enum.
`if (t.id == 77)`	`id` is not `rand`, so 500 calls to `randomize()` leave it alone.

Interview questions

Q1. How do you randomize a transaction? Declare the stimulus fields `rand`, create the object, and call its built-in `randomize()` method, checking that it returns 1.

Q2. What does `randomize()` return, and what happens to the object if it fails? 1 on success, 0 on failure. On failure the `rand` fields keep their previous values and `post_randomize()` is not called.

Q3. Does `randomize()` change non-`rand` fields? No. Only active `rand` and `randc` properties are assigned.

Q4. Can `randomize()` give an enum field an illegal value? No. The solver restricts an enum variable to its declared members.

Q5. A test fails in the nightly regression. How do you reproduce it? Re-run the same test with the same random seed, which the simulator prints in the log. SystemVerilog guarantees the same sequence of random values for the same seed and the same code (random stability), so the failure repeats exactly.

Q6. Why should the return value of `randomize()` not be discarded with `void'()`? Because a failure then goes unnoticed: the transaction keeps its old or default values and the test continues with stimulus that nobody intended, often passing while testing nothing.

2.2 Constraint-Based Transactions

In plain English

Unconstrained random values are legal for the ALU, because it accepts any operands and any opcode. But "legal" is not the same as "useful". With `a` uniform over 256 values, the value `8'hFF` appears about once in 256 transactions, and the pair `a = FF, b = FF` about once in 65,536. Yet the corners (zero, all ones, carry, borrow) are exactly where arithmetic bugs live.

Constraints put the designer's knowledge into the transaction. They do two jobs:

1. **Legality:** rule out stimulus the DUT is specified not to receive. (For many designs this is essential: an address must be inside the memory, a length must match the payload, a reserved opcode must not be sent.)
2. **Interest:** steer the random values toward the cases most likely to expose bugs.

Technical definition

A constraint block is a named, declarative member of a class that states relationships which the `rand` properties must satisfy after `randomize()`. All active constraints are solved simultaneously. A transaction whose legal and interesting values are defined by constraint blocks inside the class is called a constraint-based (or constrained-random) transaction.

Kinds of constraints found in a transaction class

Kind	Purpose	Example	Where it lives
Validity (legal range)	Never generate what the specification forbids	`addr inside {[0:1023]};`	Base transaction, always on
Relationship between fields	Keep fields consistent	`len == payload.size();`	Base transaction, always on
Distribution	Make some values more frequent	`op dist {ALU_ADD := 2, ...};`	Base transaction, may be switched off
Corner bias	Hit boundary values often	`a dist {8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8};`	Base transaction, may be switched off
Default that tests may override	A sensible value unless a test says otherwise	`soft delay inside {[0:3]};`	Base transaction
Test-specific	Focus one test on one feature	`op == ALU_ADD; a + b > 255;`	Derived class, or inline `with { }`
Error injection	Deliberately generate illegal stimulus	Validity constraint switched off with `constraint_mode(0)`	Error test

> **Key point:** Keep validity constraints and distribution constraints in **separate, named blocks**. A test that wants a different distribution can then switch off or replace one block without accidentally permitting illegal stimulus.

Constraints for the ALU transaction

```
// Arithmetic is the most complex logic, so ADD and SUB get double weight.
constraint op_dist_c {
  op dist { ALU_ADD := 2, ALU_SUB := 2, ALU_AND := 1, ALU_OR  := 1,
            ALU_XOR := 1, ALU_NOT := 1, ALU_SHL := 1, ALU_SHR := 1 };
}

// Operands: 10 percent zero, 10 percent all ones, 80 percent anything in between.
constraint corner_c {
  a dist { 8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8 };
  b dist { 8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8 };
}
```

Reading the `dist` weights (Module 5):

- `:=` gives the weight to **each** value. `ALU_ADD := 2` gives ADD a weight of 2. The weights of `op` sum to 10, so ADD is chosen about 2 times in 10.
- `:/` shares the weight across the **whole range**. `[8'h01:8'hFE] :/ 8` gives the 254 middle values a total weight of 8 between them. So `a` is `00` about 1 time in 10, `FF` about 1 time in 10, and somewhere in between 8 times in 10.

The effect on corner coverage is large:

Event	Unconstrained probability	With `corner_c`
`a == 8'hFF`	1 in 256	1 in 10
`a == 8'hFF` and `b == 8'hFF`	1 in 65,536	1 in 100
`a == 0` and `b == 0`	1 in 65,536	1 in 100

Controlling constraints from a test

Mechanism	Syntax	Effect	Typical use
Inline constraint	`t.randomize() with { op == ALU_SUB; a < b; }`	Adds constraints for this one call	A quick, local restriction
Switch a block off or on	`t.corner_c.constraint_mode(0);`	The block is ignored until switched back on	Remove a bias; error injection
Fix a field	`t.op.rand_mode(0);`	The field keeps its current value and is treated as a constant by the solver	Hold one field while the others vary
Derived class	`class alu_carry_txn extends alu_txn; constraint carry_c {...} endclass`	Adds constraints, or replaces a base block of the same name	A reusable, named test scenario (Section 3.4)
`soft` constraint	`constraint d_c { soft delay == 0; }`	Applies unless a harder constraint contradicts it	Defaults that a test may override without switching anything off

pre_randomize() and post_randomize() in a transaction

These two built-in methods are called automatically by `randomize()`. They are empty unless the class defines them.

Hook	Runs	Typical use in a transaction
`pre_randomize()`	Before the solver	Switch constraints or `rand_mode` according to a configuration field; set non-random fields that constraints depend on
`post_randomize()`	After a successful solve	Compute fields that are a function of the random ones (CRC, parity, checksum, length); clear a stale response field; print the transaction for debug

```
function void post_randomize();
  result = 'x;      // a newly randomized stimulus has no response yet
endfunction
```

A packet class would compute its checksum here: the payload is random, and the checksum is not random but derived from the payload, so it cannot be a `rand` field with a simple constraint.

Complete example

tb_txn_constraints.sv

```
module tb_txn_constraints;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    constraint op_dist_c {
      op dist { ALU_ADD := 2, ALU_SUB := 2, ALU_AND := 1, ALU_OR  := 1,
                ALU_XOR := 1, ALU_NOT := 1, ALU_SHL := 1, ALU_SHR := 1 };
    }

    constraint corner_c {
```

```
      a dist { 8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8 };
      b dist { 8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8 };
    }

    function void post_randomize();
      result = 'x;
    endfunction
  endclass

  alu_txn      t;
  int unsigned n_add, n_zero, n_max, n_ok;

  initial begin
    t = new();

    // 1. Default constraints: count how often the biased values appear
    repeat (1000) begin
      if (!t.randomize()) $fatal(1, "randomize() failed");
      if (t.op == ALU_ADD) n_add++;
      if (t.a  == 8'h00)   n_zero++;
      if (t.a  == 8'hFF)   n_max++;
    end
    $display("1. bias on  : ADD=%0d  a==00: %0d  a==FF: %0d  (of 1000)",
             n_add, n_zero, n_max);

    // 2. Switch the corner bias off: a becomes uniform again
    t.corner_c.constraint_mode(0);
    n_zero = 0;
    n_max  = 0;
    repeat (1000) begin
      if (!t.randomize()) $fatal(1, "randomize() failed");
      if (t.a == 8'h00) n_zero++;
      if (t.a == 8'hFF) n_max++;
    end
    $display("2. bias off : a==00: %0d  a==FF: %0d  (of 1000)", n_zero, n_max);
    t.corner_c.constraint_mode(1);

    // 3. Inline constraint: only subtractions that borrow
    repeat (100) begin
      if (!t.randomize() with { op == ALU_SUB; a < b; }) $fatal(1, "randomize() failed");
      if (t.op == ALU_SUB && t.a < t.b) n_ok++;
    end
    $display("3. inline constraint satisfied in %0d of 100", n_ok);

    // 4. Contradictory constraints: no solution exists
    if (!t.randomize() with { a == 8'h00; a > b; })
      $display("4. randomize() returned 0: no value of b is less than 0");
  end

endmodule
```

Sample output; the counts in lines 1 and 2 differ per seed and simulator but stay close to the values shown. Lines 3 and 4 are the same on every run. Before line 4 the simulator also prints its own message about the constraint conflict.

EXPECTED OUTPUT

```
1. bias on  : ADD=204  a==00: 97  a==FF: 105  (of 1000)
2. bias off : a==00: 5  a==FF: 3  (of 1000)
3. inline constraint satisfied in 100 of 100
4. randomize() returned 0: no value of b is less than 0
```

Line by line

Line	Explanation
`constraint op_dist_c {...}`	The weights sum to 10; ADD has 2 of them, so about 200 of 1000 transactions are ADD (unweighted it would be about 125).

Line	Explanation
`constraint corner_c {...}`	Two independent `dist` expressions in one block, one per operand. About 100 of 1000 values of `a` are `00` and about 100 are `FF`.
`function void post_randomize();`	Runs after every successful randomization and marks the response as "not captured yet".
`t.corner_c.constraint_mode(0);`	Switches off that one block in that one object. `a` is now uniform, so each specific value appears about 4 times in 1000.
`t.corner_c.constraint_mode(1);`	Switches it back on. A constraint that is switched off stays off until this call.
`randomize() with { op == ALU_SUB; a < b; }`	The inline constraints are solved **together with** the class constraints. `op == ALU_SUB` is compatible with `op_dist_c` because SUB has a non-zero weight.
`if (t.op == ALU_SUB && t.a < t.b) n_ok++;`	The testbench checks its own stimulus. Every one of the 100 satisfies the constraint.
`with { a == 8'h00; a > b; }`	`a` must be 0 and also greater than the unsigned `b`. Impossible, so `randomize()` returns 0 and leaves the object unchanged.

Common mistake: Giving a value a `dist` weight of zero, or leaving it out of the `dist` list, and then asking for it inline. `op dist {ALU_ADD := 1, ALU_SUB := 1}` makes the other six opcodes impossible; `randomize() with { op == ALU_XOR; }` then fails. A `dist` is also a constraint on the set of allowed values.

Common mistake: Over-constraining the base transaction. If the base class contains `a < b`, no test can ever generate `a >= b` without switching the constraint off, and the engineer who writes the next test may not know it exists. The base class should contain what is **always** true; narrowing belongs in tests.

Verification application: In the Module 9 testbench the generator never mentions `ALU_ADD` or `8'hFF`. It only calls `randomize()`. What the stimulus looks like is decided entirely by the constraints in `alu_txn` and its test-specific extensions. That division is what lets one environment run many different tests.

Constraints compared with procedural code

A beginner often asks why constraints are needed when the same effect seems achievable with `$urandom` and `if` statements.

Aspect	Procedural (`$urandom_range`, `if`)	Constraints
Style	Step by step: "how to compute a value"	Declarative: "what must be true"
Related fields	The order of assignments decides which field is chosen first; hard to keep relations correct	All fields are solved together
Changing behaviour in a test	Edit the code	Add, replace or switch off a constraint; the original code is untouched
Combining requirements from several places	Manual	Automatic: base class + derived class + inline are solved as one set
Impossible requirements	Silent wrong values	`randomize()` returns 0

Interview questions

Q1. What is a constraint-based transaction? A transaction class whose `rand` fields are restricted and shaped by constraint blocks, so that each `randomize()` call produces a legal and useful operation.

Q2. What are the two purposes of constraints in a transaction? Legality (never generate stimulus the specification forbids) and interest (bias the stimulus toward corner cases and important scenarios).

Q3. Why split constraints into several named blocks? So that each can be switched off with `constraint_mode(0)` or replaced in a derived class independently. A test can change the distribution without disturbing the validity rules.

Q4. How do you make corner values such as 0 and all ones appear frequently? With a `dist` constraint that gives those values explicit weights, for example `a dist {8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8};`.

Q5. What is the difference between `:=` and `:/` in `dist`? `:=` assigns the weight to every value in the item; `:/` divides the weight equally among the values of the item.

Q6. What is `post_randomize()` used for in a transaction? For fields that depend on the randomized ones but cannot be expressed as simple constraints, such as a CRC or checksum, and for clean-up such as clearing the response field.

Q7. A test needs only ADD operations that produce a carry. Give two ways to get them. Inline: `t.randomize() with { op == ALU_ADD; a + b > 255; }`. Or a derived class `alu_carry_txn` with a constraint block containing the same expressions, which the test supplies to the generator.

Q8. How would you generate an illegal transaction on purpose? Switch off the validity constraint with `constraint_mode(0)` (and usually add a constraint that forces the illegal value), in a test dedicated to error handling. The scoreboard must know that an error is expected.

Part 3: Printing, Copying, Comparing and Reusing Transactions

3.1 Transaction Printing

In plain English

When a test fails, the first thing you read is the log file, not the waveform. A good log tells the story of the simulation in transactions: "generator created #17: SUB 5, 10", "monitor observed #17: result 1FB", "scoreboard: #17 PASS". For that, every transaction must be able to describe itself in one readable line.

Technical definition

A print method is a member function of the transaction class that displays the values of its properties in a fixed, readable format. It is normally split in two:

- `convert2string()` returns the formatted text as a `string` and prints nothing.
- `print(tag)` displays that string, with the simulation time and a caller-supplied tag.

Why two methods?

Reason	Explanation
Reuse of the formatting	The same text is needed in `print`, in `$error` messages from the scoreboard, and sometimes in files. Building it once in `convert2string()` avoids three copies of the format string
Control of the output	The caller decides whether to display it, pass it to `$error`, write it to a file or ignore it
Extension	A derived class with one more field overrides `convert2string()`, calls `super.convert2string()` and appends its field; `print()` needs no change

Why not just use $display in each component?

Without a print method, the driver, monitor and scoreboard each contain their own `$display` with their own format. When a field is added to the transaction, each one has to be found and edited, and the log shows the same data in three different layouts. With a print method there is one format and one place to change it.

Syntax

```
virtual function string convert2string();
  return $sformatf("id=%0d op=%s a=0x%h b=0x%h result=0x%h", id, op.name(), a, b, result);
endfunction

virtual function void print(string tag = "");
  $display("[%0t] %s: %s", $time, tag, convert2string());
endfunction
```

`$sformatf` works like `$display` but returns the formatted text as a string instead of printing it (Module 3). The methods are `virtual` so that a derived transaction's version is used even when the object is reached through a base-class handle (Module 4).

Complete example

tb_txn_print.sv

```
module tb_txn_print;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    virtual function string convert2string();
      return $sformatf("id=%0d op=%s a=0x%h b=0x%h result=0x%h",
                       id, op.name(), a, b, result);
    endfunction

    virtual function void print(string tag = "");
      $display("[%0t] %s: %s", $time, tag, convert2string());
    endfunction
  endclass

  alu_txn t;

  initial begin
    t    = new();
    t.id = 1;
    t.op = ALU_ADD;
    t.a  = 8'd200;
    t.b  = 8'd100;
    t.print("GEN");

    #10;
    t.result = 9'd300;
    t.print("MON");

    $display("SCB: unexpected result in {%s}", t.convert2string());
  end

endmodule
```

EXPECTED OUTPUT

```
[0] GEN: id=1 op=ALU_ADD a=0xc8 b=0x64 result=0xxxx
[10] MON: id=1 op=ALU_ADD a=0xc8 b=0x64 result=0x12c
SCB: unexpected result in {id=1 op=ALU_ADD a=0xc8 b=0x64 result=0x12c}
```

Line by line

Line	Explanation
`$sformatf(...)`	Builds the string. `%s` with `op.name()` prints the opcode name; `%h` prints each field in hex at its full width (two digits for 8 bits, three for 9 bits).
`t.print("GEN");`	The tag identifies the component that printed. At time 0 the result has not been captured: the three hex digits are `xxx`, shown after the literal `0x`.
`#10; t.result = 9'd300;`	Stands in for the monitor capturing the DUT output. 300 is `12c` in hex.
`t.print("MON");`	Same object, same format, different tag and time.

Line	Explanation
Last `$display`	The scoreboard embeds the same text in its own message by calling `convert2string()` directly.

What a good print method includes

Include	Reason
The transaction ID	To follow one transaction through the generator, driver, monitor and scoreboard lines
Enum fields by name	`ALU_SUB` is understood at a glance; `1` is not
Data in hex, counts in decimal	Convention; hex shows bit patterns and X clearly
Everything on one line	So that `grep` on the log returns complete transactions
A fixed field order and fixed labels	So that logs can be compared and searched
Simulation time and a component tag	Supplied by `print`, not by `convert2string`

`$display("%p", t)` also prints the properties of an object in most simulators and is handy for a quick debug session, but its layout differs between tools and it cannot be customised, so it does not replace a print method.

> **Common mistake:** Printing every transaction unconditionally. A regression with a million transactions produces gigabytes of log and runs much slower. Real testbenches print through a verbosity control (for example `if (verbosity >= HIGH) t.print("DRV");`), which is one of the services UVM's reporting macros provide.

Interview questions

Q1. Why does a transaction class need a print method? So that every component reports transactions in one consistent, readable format, the format is maintained in one place, and logs can be used for debugging without opening waveforms.

Q2. What is the difference between `print()` and `convert2string()`? `convert2string()` returns the formatted contents as a string without displaying anything. `print()` displays it. `print()` is normally implemented by calling `convert2string()`.

Q3. What is the difference between `$display`, `$sformatf` and `$sformat`? `$display` prints to the log. `$sformatf` is a function that returns the formatted string. `$sformat` is a task that writes the formatted string into a string variable given as its first argument.

Q4. How do you print an enum field by name? With its `name()` method and `%s`.

Q5. Why are the print methods declared `virtual`? So that when a derived transaction with additional fields is handled through a base-class handle, the derived class's version runs and the additional fields are printed.

3.2 Transaction Copying

In plain English

A class variable does not contain an object. It contains a **handle**, a reference to an object (Module 4). The assignment `t2 = t1;` therefore does not make a second transaction; it makes a second name for the same transaction. Change a field through `t2` and `t1` sees the change, because there is only one object.

In a testbench, transactions are handed from component to component and stored in queues and mailboxes. If two places hold a handle to the same object and one of them modifies it, the other is corrupted. A **copy** method creates a genuinely separate object with the same field values, so that each holder has its own.

Technical definition

A copy method is a member function that constructs a new object of the class, assigns the value of every property of the current object to it, and returns the handle of the new object.

```
virtual function alu_txn copy();
  alu_txn c = new();
  c.a      = a;
  c.b      = b;
  c.op     = op;
  c.result = result;
  c.id     = id;
  return c;
endfunction
```

How it works internally

```
   Handle assignment           t2 = t1;

      t1 ----+
             +---->  [ object:  id=1  op=ADD  a=200  b=100 ]      ONE object, two handles
      t2 ----+


   Copy                        t2 = t1.copy();

      t1 --------->  [ object:  id=1  op=ADD  a=200  b=100 ]      TWO objects
      t2 --------->  [ object:  id=1  op=ADD  a=200  b=100 ]      same contents, independent
```

The three ways to "copy", compared

Operation	Syntax	What is created	Fields that are handles to other objects	Use
Handle assignment	`t2 = t1;`	Nothing; a second reference	Not applicable	Passing an object on when the sender will not touch it again
Shallow copy	`t2 = new t1;`	A new object; every property copied as it is	The **handle** is copied, so the inner object is shared	Classes with no handle properties
Deep copy (user-written `copy()`)	`t2 = t1.copy();`	A new object	The method also copies each inner object	Any class; the standard choice for transactions

For `alu_txn`, which contains only plain values, the shallow copy `new t1` and the hand-written `copy()` produce the same result. A `copy()` method is still written, for three reasons: it is the place where inner objects get copied when the class grows; it can be made `virtual` so that it works through a base handle; and it gives every transaction class the same interface.

A transaction that contains another object needs the deep copy:

```
class frame_txn;
  rand bit [7:0] payload [];      // dynamic array: copied by value on assignment
  hdr_txn        hdr;             // handle to another object

  virtual function frame_txn copy();
    frame_txn c = new();
    c.payload = payload;          // arrays are copied element by element
    c.hdr     = hdr.copy();       // the inner object is copied, not shared
    return c;
  endfunction
endclass
```

Dynamic arrays, queues and associative arrays are **not** handles: assigning one to another copies the elements. Only class handles need the explicit inner `copy()`.

Where copies are needed in a testbench

Place	Why a copy is needed
Generator that re-randomizes one blueprint object	Each transaction sent to the driver must be a separate object, otherwise the next `randomize()` changes a transaction the driver has not finished with
Monitor that reuses one object to collect pin values	Each observed transaction sent to the scoreboard must be a separate object
Scoreboard storing expected transactions in a queue	The stored object must not change when the sender reuses or modifies its own
Any component that modifies a received transaction	It should modify its own copy, not the object other components still hold
One transaction sent to two receivers (scoreboard and coverage)	If either may modify it, each needs its own

Complete example: the shared-handle bug

tb_txn_copy.sv

```
module tb_txn_copy;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    virtual function alu_txn copy();
      alu_txn c = new();
      c.a      = a;
      c.b      = b;
      c.op     = op;
      c.result = result;
      c.id     = id;
      return c;
    endfunction
  endclass

  alu_txn t;
  alu_txn q_bad  [$];
  alu_txn q_good [$];

  initial begin
    t = new();
    for (int i = 0; i < 3; i++) begin
      t.id = i;
      t.a  = 10 * i;
      q_bad.push_back(t);             // BUG : the same handle is stored three times
      q_good.push_back(t.copy());     // GOOD: three separate objects are stored
    end

    foreach (q_bad[i])
      $display("q_bad [%0d]: id=%0d a=%0d", i, q_bad[i].id, q_bad[i].a);
    foreach (q_good[i])
      $display("q_good[%0d]: id=%0d a=%0d", i, q_good[i].id, q_good[i].a);

    $display("q_bad[0]  and q_bad[2]  are the same object: %0d", q_bad[0]  == q_bad[2]);
    $display("q_good[0] and q_good[2] are the same object: %0d", q_good[0] == q_good[2]);
  end

endmodule
```

EXPECTED OUTPUT

```
q_bad [0]: id=2 a=20
q_bad [1]: id=2 a=20
q_bad [2]: id=2 a=20
q_good[0]: id=0 a=0
q_good[1]: id=1 a=10
q_good[2]: id=2 a=20
q_bad[0]  and q_bad[2]  are the same object: 1
q_good[0] and q_good[2] are the same object: 0
```

Line by line

Line	Explanation
`t = new();`	One object is created, **outside** the loop.
`q_bad.push_back(t);`	Stores the handle. After three iterations the queue holds three handles, all referring to the one object.

Line	Explanation
`q_good.push_back(t.copy());`	`copy()` creates a new object holding the current values, and that new object's handle is stored.
`q_bad` output	Every entry shows the values from the last iteration (`id=2`, `a=20`), because every entry is the same object and that is what it now contains.
`q_good` output	Each entry kept the values it had when it was copied.
`q_bad[0] == q_bad[2]`	Comparing two handles with `==` asks "is this the same object?". For the bad queue the answer is 1.

This is the single most common bug in hand-written generators: every transaction received by the driver is identical to the last one generated. It is asked about in interviews in many forms.

> **Interview trap:** "The generator randomizes ten transactions and puts them in a mailbox, but the driver receives the same transaction ten times. Why?" The generator created one object before the loop and put the same handle into the mailbox ten times. By the time the driver reads them, the object holds the values from the last `randomize()`. Fix: construct a new object inside the loop, or put `t.copy()` into the mailbox.

> **Common mistake:** Adding a field to the transaction and forgetting to add it to `copy()` (and to `compare()` and `convert2string()`). The copy then silently has the default value for that field. Whenever a field is added, update all three methods.

> **Common mistake:** Declaring `copy()` in a derived class without `virtual` in the base class. Calling `copy()` through a base-class handle then runs the base version and the derived fields are lost.

Interview questions

Q1. What is the difference between `t2 = t1;` and `t2 = t1.copy();`? `t2 = t1;` copies the handle: both variables refer to the same object. `t2 = t1.copy();` creates a second object with the same field values; the two are independent afterwards.

Q2. What is the difference between a shallow copy and a deep copy? A shallow copy (`new t1`) copies every property as it is, so a property that is a handle still refers to the same inner object in both copies. A deep copy also creates copies of the inner objects, so nothing is shared.

Q3. Why does a testbench need to copy transactions? Because objects are passed by handle. If a sender keeps using or re-randomizing an object after passing its handle on, the receiver's data changes underneath it. Copying gives each holder a separate object.

Q4. Does `copy()` need to handle dynamic arrays specially? No. Assigning a dynamic array, queue or associative array copies its elements. Only properties that are class handles need their own `copy()` call.

Q5. Write a copy method for a transaction with fields `addr`, `data` and `write`. `virtual function bus_txn copy(); bus_txn c = new(); c.addr = addr; c.data = data; c.write = write; return c; endfunction`

3.3 Transaction Comparison

In plain English

The scoreboard holds two transactions for each operation: what the DUT **should** have produced (expected) and what the monitor **saw** it produce (actual). It needs to ask "are these the same?". Comparing the two handles is useless, because they are two different objects and the answer is always "no". What must be compared is the **contents**, field by field.

Technical definition

A compare method is a member function that takes a handle to another object of the same class and returns 1 if every field that is relevant for checking has the same value in both objects, and 0 otherwise.

```
virtual function bit compare(alu_txn rhs);
  if (rhs == null) return 0;
  return (a      ==  rhs.a)  &&
         (b      ==  rhs.b)  &&
         (op     === rhs.op) &&
         (result === rhs.result);
endfunction
```

`rhs` stands for "right-hand side": the call `lhs.compare(rhs)` reads like `lhs == rhs`.

Design decisions inside a compare method

Decision	Choice for `alu_txn`	Reason
Which fields are compared?	`a`, `b`, `op`, `result`	These define the operation and its outcome
Which fields are ignored?	`id`	Bookkeeping. The expected and actual objects may be numbered by different components
Which operator for four-state fields?	`===`	With `==`, an X in `result` makes the comparison X; the function would return 0 here, but only by luck of the conversion to `bit`, and an `if (exp.result != act.result)` written elsewhere would miss the X entirely. `===` gives a definite answer (Module 1, Section 4.5)
Which operator for two-state fields?	`==`	No X is possible, so `==` and `===` are equivalent
What if `rhs` is `null`?	Return 0	Accessing a field through a null handle is a fatal run-time error. The check turns a crash into a clean mismatch
Should it print?	No, it only returns the verdict	The scoreboard decides what to report. A variant that reports the differing fields is shown below

Handle comparison versus content comparison

Expression	Question it answers	Result for two separate objects with equal fields
`t1 == t2`	Do both handles refer to the same object?	0

Expression	Question it answers	Result for two separate objects with equal fields
`t1.compare(t2)`	Do the two objects hold the same values?	1

Complete example

tb_txn_compare.sv

```
module tb_txn_compare;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    virtual function alu_txn copy();
      alu_txn c = new();
      c.a = a;  c.b = b;  c.op = op;  c.result = result;  c.id = id;
      return c;
    endfunction

    virtual function bit compare(alu_txn rhs);
      if (rhs == null) return 0;
      return (a      ==  rhs.a)  &&
             (b      ==  rhs.b)  &&
             (op     === rhs.op) &&
             (result === rhs.result);
    endfunction
  endclass

  alu_txn exp, act, same, none;

  initial begin
    exp        = new();
    exp.id     = 1;
    exp.op     = ALU_ADD;
    exp.a      = 8'd200;
    exp.b      = 8'd100;
    exp.result = 9'd300;

    act  = exp.copy();
    same = exp;

    $display("1. exp == act  (handles)       : %0d", exp == act);
    $display("2. exp == same (handles)       : %0d", exp == same);
    $display("3. exp.compare(act)            : %0d", exp.compare(act));

    act.id = 99;
    $display("4. different id                : %0d", exp.compare(act));

    act.result = 9'd301;
    $display("5. different result            : %0d", exp.compare(act));

    act.result = 'x;
    $display("6. result is X                 : %0d", exp.compare(act));
    $display("7. X with == gives %b, with === gives %b",
             exp.result == act.result, exp.result === act.result);

    $display("8. compare with a null handle  : %0d", exp.compare(none));
  end

endmodule
```

EXPECTED OUTPUT

```
1. exp == act  (handles)       : 0
2. exp == same (handles)       : 1
3. exp.compare(act)            : 1
4. different id                : 1
5. different result            : 0
6. result is X                 : 0
7. X with == gives x, with === gives 0
8. compare with a null handle  : 0
```

Line by line

Line	Explanation
1	`act` is a copy: a different object. Handle equality is 0 even though the contents are identical.
2	`same = exp;` copied the handle, so both refer to one object: 1.
3	Content comparison of the copy: all four compared fields match.
4	`id` is deliberately not compared, so changing it does not cause a mismatch.
5	300 versus 301: mismatch.
6	The DUT output is X. `===` compares X literally against the expected bits and reports a mismatch.
7	The same comparison with `==` evaluates to X. Inside an `if`, that X would count as false, which is how a scoreboard written with `!=` lets an X output pass.
8	`none` was never constructed, so it is `null`. The guard at the top of `compare()` returns 0 instead of crashing.

A compare method that reports what differs

For debugging it helps to know *which* field mismatched. A common variant returns the verdict and also fills in a message:

```
virtual function bit compare_report(alu_txn rhs, output string msg);
  msg = "";
  if (rhs == null) begin
    msg = "rhs is null";
    return 0;
  end
  if (op !== rhs.op)
    msg = {msg, $sformatf(" op: %s vs %s;", op.name(), rhs.op.name())};
  if (a != rhs.a)
    msg = {msg, $sformatf(" a: 0x%h vs 0x%h;", a, rhs.a)};
  if (b != rhs.b)
    msg = {msg, $sformatf(" b: 0x%h vs 0x%h;", b, rhs.b)};
  if (result !== rhs.result)
    msg = {msg, $sformatf(" result: 0x%h vs 0x%h;", result, rhs.result)};
  return (msg == "");
endfunction
```

`{msg, ...}` concatenates strings. The function returns 1 only if no difference was recorded.

> **Interview trap:** "The scoreboard compares `if (exp == act)` and always reports a mismatch, although the waveform is correct. Why?" `exp` and `act` are handles to two different objects, so `==` is always 0. The fields must be compared, with a `compare()` method.

> **Common mistake:** Comparing fields that legitimately differ, such as timestamps or IDs assigned by different components, which produces false failures; or leaving out a field that matters, which produces false passes. Decide field by field, and write the decision down as a comment in the method.

> **Verification application:** In the Module 9 scoreboard the check is three lines: compute `exp_result = alu_predict(act);`, then `if (act.result !== exp_result) $error(...)`, else count a pass. When the expected value is a whole transaction produced by a reference model, the check is `if (!exp.compare(act)) $error(...)`.

Interview questions

Q1. Why can two transaction objects not be compared with `==`? `==` on class handles tests whether they refer to the same object. Two separate objects with identical contents are not equal by that test. A user-written `compare()` method compares the fields.

Q2. Which fields would you leave out of `compare()`? Bookkeeping that is not part of the functional behaviour: IDs, timestamps, delay fields, debug strings.

Q3. Why use `===` for the result field? The result is sampled from the DUT and may contain X or Z. `===` treats X and Z as values and returns a definite 0 or 1, so an unknown output is reported as a mismatch. `==` would return X.

Q4. What should `compare()` do if the argument is `null`? Return 0 (not equal) before touching any field, to avoid a null-handle run-time error.

Q5. What is the difference between `copy()` and `compare()` in terms of where they are used? `copy()` is used by producers (generator, monitor) to hand out independent objects. `compare()` is used by the checker (scoreboard) to decide pass or fail.

3.4 Transaction Reuse

In plain English

One ALU test wants completely random operations. Another wants only additions that overflow. A third wants only shifts. A fourth, written next year for a wider ALU, needs one extra field. If each of these required editing `alu_txn`, or editing the generator, the testbench would be rewritten for every test and every project.

Reuse means: write the transaction (and the components that use it) **once**, and obtain each new behaviour by **adding** something, never by editing what already works.

The three levels of reuse

Level	What is reused	Mechanism
Across components	One class is used by the generator, driver, monitor, scoreboard and coverage	All components import the same package
Across tests	The same environment runs different stimulus	A derived transaction class adds or replaces constraints; the test hands an object of that class to the generator

Level	What is reused	Mechanism
Across projects	The transaction, and the agent built around it, is used wherever the same interface appears	The class is kept free of anything project-specific (no pins, no hierarchy paths, no hard-coded test knowledge)

Reuse across tests: inheritance and polymorphism

Three things from Modules 4 and 5 combine here.

1. **A derived class inherits every property, method and constraint** of its base class and may add more.
2. **A constraint block in a derived class with the same name as one in the base class replaces it.** A block with a new name is added to the inherited ones.
3. **A base-class handle can refer to a derived-class object**, and `randomize()` is virtual: called through the base handle, it applies the constraints of the **actual object**.

```
// Adds a constraint: all inherited constraints still apply
class alu_carry_txn extends alu_txn;
  constraint carry_c { op == ALU_ADD; a + b > 255; }
endclass

// Replaces a constraint: same block name as in the base class
class alu_shift_txn extends alu_txn;
  constraint op_dist_c { op inside {ALU_SHL, ALU_SHR}; }
endclass
```

In `a + b > 255` the literal `255` is 32 bits wide, so the sum is evaluated in 32 bits and the carry is not lost (Module 1, Section 4.4).

The blueprint pattern

The generator is written against the base class only. It owns a handle called the **blueprint**. For every transaction it randomizes the blueprint and sends a copy.

```
   test                                  generator (never edited)
   ----                                  ------------------------------------------------
   alu_carry_txn c = new();              alu_txn blueprint;        // base-class handle
   gen.blueprint = c;      ---------->   repeat (n) begin
                                           blueprint.randomize();  // constraints of the
                                                                   // actual object apply
                                           send(blueprint.copy()); // independent object
                                         end
```

- With a plain `alu_txn` as blueprint, the generator produces the default stimulus.
- With an `alu_carry_txn` as blueprint, the **same generator code** produces only carry-generating additions.

The test changed the behaviour of the whole environment by supplying a different object. This idea, in a more automated form, is the UVM factory override.

Complete example

tb_txn_reuse.sv

```
module tb_txn_reuse;
  import alu_pkg::*;

  // ---------------- base transaction: written once ----------------
  class alu_txn;
```

```
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    constraint op_dist_c {
      op dist { ALU_ADD := 2, ALU_SUB := 2, ALU_AND := 1, ALU_OR  := 1,
                ALU_XOR := 1, ALU_NOT := 1, ALU_SHL := 1, ALU_SHR := 1 };
    }

    virtual function string convert2string();
      return $sformatf("id=%0d op=%s a=0x%h b=0x%h", id, op.name(), a, b);
    endfunction

    virtual function alu_txn copy();
      alu_txn c = new();
      c.a = a;  c.b = b;  c.op = op;  c.result = result;  c.id = id;
      return c;
    endfunction
  endclass

  // ---------------- test-specific transactions: added later ----------------
  class alu_carry_txn extends alu_txn;
    constraint carry_c { op == ALU_ADD; a + b > 255; }           // added
  endclass

  class alu_shift_txn extends alu_txn;
    constraint op_dist_c { op inside {ALU_SHL, ALU_SHR}; }       // replaces the base block
  endclass

  // ---------------- "generator": knows only the base class ----------------
  function automatic void make_stimulus(alu_txn blueprint, int unsigned n,
                                        ref alu_txn out_q[$]);
    for (int unsigned i = 0; i < n; i++) begin
      if (!blueprint.randomize()) $fatal(1, "randomize() failed");
      blueprint.id = i;
      out_q.push_back(blueprint.copy());
    end
  endfunction

  alu_txn       q[$];
  alu_carry_txn carry_bp;
  alu_shift_txn shift_bp;
  int unsigned  n_ok;

  initial begin
    // Test 1: additions with carry
    carry_bp = new();
    make_stimulus(carry_bp, 50, q);
    n_ok = 0;
    foreach (q[i]) if (q[i].op == ALU_ADD && (q[i].a + q[i].b) > 255) n_ok++;
    $display("carry test: %0d of %0d transactions are ADD with carry", n_ok, q.size());
    $display("  first   : %s", q[0].convert2string());

    // Test 2: shifts only
    q.delete();
    shift_bp = new();
    make_stimulus(shift_bp, 50, q);
    n_ok = 0;
    foreach (q[i]) if (q[i].op inside {ALU_SHL, ALU_SHR}) n_ok++;
    $display("shift test: %0d of %0d transactions are shifts", n_ok, q.size());
    $display("  first   : %s", q[0].convert2string());
  end

endmodule
```

Sample output; the two lines beginning with "first" differ per seed and simulator. The two count lines are the same on every run.

EXPECTED OUTPUT

```
carry test: 50 of 50 transactions are ADD with carry
  first   : id=0 op=ALU_ADD a=0xd3 b=0x9a
shift test: 50 of 50 transactions are shifts
  first   : id=0 op=ALU_SHR a=0x47 b=0xe0
```

Line by line

Line	Explanation
`class alu_carry_txn extends alu_txn;`	Inherits `a`, `b`, `op`, `result`, `id`, `op_dist_c`, `convert2string()` and `copy()`. Adds one constraint block.
`constraint carry_c { op == ALU_ADD; a + b > 255; }`	A new name, so it is solved **together with** the inherited `op_dist_c`. They are compatible because ADD has a non-zero weight there.
`constraint op_dist_c { op inside {ALU_SHL, ALU_SHR}; }`	The same name as the base block, so the base block is replaced. If it were only added, it would still work here, but the 2:1 weighting of ADD would be pointless.
`function automatic void make_stimulus(alu_txn blueprint, ...)`	The argument type is the base class. The function contains no reference to any derived class. A function with a `ref` argument must be `automatic`.
`blueprint.randomize()`	`randomize()` is virtual: the constraints of the object that `blueprint` actually refers to are used.
`out_q.push_back(blueprint.copy());`	A separate object for each transaction (Section 3.2). `copy()` here returns a base-class object carrying the randomized values, which is all that the driver needs.
`make_stimulus(carry_bp, 50, q);`	A derived handle is passed where a base handle is expected. This upcast is always legal and needs no `$cast`.
Count lines	The testbench checks its own stimulus: all 50 satisfy the test's constraint.

Extending a transaction with a new field

When a derived transaction adds a field, it overrides the methods and calls the base version with `super` so that nothing is duplicated:

```
class alu_delay_txn extends alu_txn;
  rand int unsigned delay;                          // idle cycles before driving
  constraint delay_c { delay inside {[0:5]}; }

  virtual function string convert2string();
    return {super.convert2string(), $sformatf(" delay=%0d", delay)};
  endfunction

  virtual function alu_txn copy();
    alu_delay_txn c = new();
    c.a = a;  c.b = b;  c.op = op;  c.result = result;  c.id = id;
    c.delay = delay;
    return c;                                       // returned through a base-class handle
  endfunction
endclass
```

Because `copy()` and `convert2string()` are virtual, a generator that holds this object through an `alu_txn` handle copies and prints the extra field correctly. A driver that wants to use `delay` obtains a derived handle with `$cast` (Module 4).

Rules for a reusable transaction

Rule	Why
Declare `copy`, `compare`, `print`, `convert2string` as `virtual`	Derived versions must run through base handles
Give each constraint block one purpose and a clear name	Tests override or disable by name
Keep only always-true constraints hard in the base class	Anything narrower blocks future tests
Use `soft` for defaults that tests may want to change	A test's hard constraint then wins without `constraint_mode` calls
No pin names, no hierarchical paths, no interface handles	The class must compile in any environment
Put the class in a package	Every component and every test imports the same definition
Make the constructor callable with no arguments	Generic code can create it

> **Common mistake:** Editing the base transaction for each new test ("just add an `if (test_mode == 3)` to the constraint"). After a few tests the base class is a tangle of special cases and a change for one test breaks another. Extend; do not edit.

> **Common mistake:** A derived constraint that contradicts an inherited one. If the base class has `a < 128` and the derived class adds `a > 200` under a **different** block name, both apply and `randomize()` fails. Either replace the base block by using the same name, or switch it off.

Interview questions

Q1. How do you create a new test without changing the generator or driver? Extend the transaction class, add or replace constraints in the derived class, and have the test give an object of the derived class to the generator as its blueprint. The generator works through a base-class handle, so its code does not change.

Q2. What happens when a derived class declares a constraint block with the same name as one in the base class? The derived block replaces the base block. A block with a different name is added to the inherited constraints.

Q3. What is the blueprint pattern? The generator holds one base-class handle, randomizes that object repeatedly, and sends a copy each time. A test replaces the blueprint object with a derived-class object to change what is generated.

Q4. Why must `copy()` be virtual? The generator calls `copy()` through a base-class handle. If `copy()` were not virtual, the base version would run even for a derived object, and any fields added by the derived class would not be copied.

Q5. What makes a transaction class reusable in another project? It describes only the operation on its interface: no pins, no hierarchy, no project-specific assumptions. Its methods are virtual, its constraints are separated by purpose, and it lives in a package.

3.5 The Complete alu_txn Class

This is the final version of the transaction, combining everything from Parts 1 to 3. It is the class used by the layered testbench in Module 9.

The file

The class is written in a file of its own. The file contains only the class (and the reference-model function described below), and is pulled into a package with `` `include ``. This is the normal industry layout: one class per file, one package that includes them all.

alu_txn.sv

```
// alu_txn.sv
// Not compiled by itself: it is `included inside package alu_tb_pkg,
// which has already imported alu_pkg.

class alu_txn;

  // ---------------- stimulus ----------------
  rand bit [7:0] a;
  rand bit [7:0] b;
  rand alu_op_e  op;

  // ---------------- response ----------------
  logic [8:0]    result;

  // ---------------- bookkeeping ----------------
  int unsigned   id;

  // ---------------- constraints ----------------
  constraint op_dist_c {
    op dist { ALU_ADD := 2, ALU_SUB := 2, ALU_AND := 1, ALU_OR  := 1,
              ALU_XOR := 1, ALU_NOT := 1, ALU_SHL := 1, ALU_SHR := 1 };
  }

  constraint corner_c {
    a dist { 8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8 };
    b dist { 8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8 };
  }

  // ---------------- methods ----------------
  function new();
    op     = ALU_ADD;      // a legal opcode instead of the default X
    result = 'x;           // no response captured yet
  endfunction

  function void post_randomize();
    result = 'x;
  endfunction

  function bit is_unary();
    return op inside {ALU_NOT, ALU_SHL, ALU_SHR};
  endfunction

  virtual function string convert2string();
    return $sformatf("id=%0d op=%s a=0x%h b=0x%h result=0x%h",
                     id, op.name(), a, b, result);
  endfunction

  virtual function void print(string tag = "");
    $display("[%0t] %s: %s", $time, tag, convert2string());
  endfunction

  virtual function alu_txn copy();
    alu_txn c = new();
    c.a      = a;
    c.b      = b;
```

```
    c.op     = op;
    c.result = result;
    c.id     = id;
    return c;
  endfunction

  // id is bookkeeping and is deliberately not compared
  virtual function bit compare(alu_txn rhs);
    if (rhs == null) return 0;
    return (a      ==  rhs.a)  &&
           (b      ==  rhs.b)  &&
           (op     === rhs.op) &&
           (result === rhs.result);
  endfunction

endclass


// Reference model: the expected result for the inputs held in a transaction.
// Written from the specification, not copied from the RTL.
function automatic logic [8:0] alu_predict(alu_txn t);
  case (t.op)
    ALU_ADD: return {1'b0, t.a} + {1'b0, t.b};
    ALU_SUB: return {1'b0, t.a} - {1'b0, t.b};
    ALU_AND: return {1'b0, t.a & t.b};
    ALU_OR : return {1'b0, t.a | t.b};
    ALU_XOR: return {1'b0, t.a ^ t.b};
    ALU_NOT: return {1'b0, ~t.a};
    ALU_SHL: return {t.a, 1'b0};
    ALU_SHR: return {2'b00, t.a[7:1]};
    default: return 'x;
  endcase
endfunction
```

alu_tb_pkg.sv

```
// alu_tb_pkg.sv : compile after alu_pkg.sv
package alu_tb_pkg;
  import alu_pkg::*;

  `include "alu_txn.sv"

endpackage
```

In Module 9 the generator, driver, monitor, scoreboard, environment and test classes are added to this package in the same way.

The reference model function

`alu_predict` answers the question "given these inputs, what must the result be?". It is a first, small example of a **reference model**: an independent description of the DUT's function, written at a higher level and used by the scoreboard to produce the expected value. Section 4.10 discusses it further. Points to notice:

Code	Explanation
`{1'b0, t.a} + {1'b0, t.b}`	Both operands are widened to nine bits, so the carry appears in bit 8, as the specification requires.
`{1'b0, t.a} - {1'b0, t.b}`	Nine-bit subtraction. When `a < b` the result wraps and bit 8 is 1, which is the borrow.
`{1'b0, ~t.a}`	The inversion is done on eight bits and then a zero is placed in bit 8. Writing `~{1'b0, t.a}` would set bit 8 to 1, which is wrong.
`default: return 'x;`	An unknown opcode gives an unknown prediction, so the comparison with `!==` reports it.

Using the class

tb_alu_txn_final.sv

```
// Compile order: alu_pkg.sv, alu_tb_pkg.sv (which includes alu_txn.sv), this file
module tb_alu_txn_final;
  import alu_pkg::*;
  import alu_tb_pkg::*;

  int unsigned n;
  alu_txn      t, c;

  function automatic void show(alu_op_e op, bit [7:0] a, bit [7:0] b);
    alu_txn x = new();
    x.id     = n++;
    x.op     = op;
    x.a      = a;
    x.b      = b;
    x.result = alu_predict(x);
    x.print("EXP");
  endfunction

  initial begin
    show(ALU_ADD, 8'd200, 8'd100);
    show(ALU_SUB, 8'd5,   8'd10);
    show(ALU_AND, 8'hF0,  8'h3C);
    show(ALU_NOT, 8'h0F,  8'h00);
    show(ALU_SHL, 8'h81,  8'h00);
    show(ALU_SHR, 8'h81,  8'h00);

    t = new();
    $display("new object     : %s  unary=%0d", t.convert2string(), t.is_unary());

    if (!t.randomize() with { op == ALU_XOR; a == 8'hAA; b == 8'h0F; })
      $fatal(1, "randomize() failed");
    t.result = alu_predict(t);
    c = t.copy();
    $display("copy equal     : %0d", t.compare(c));
    c.result = 'x;
    $display("X result equal : %0d", t.compare(c));
    t.print("XOR");
  end

endmodule
```

EXPECTED OUTPUT

```
[0] EXP: id=0 op=ALU_ADD a=0xc8 b=0x64 result=0x12c
[0] EXP: id=1 op=ALU_SUB a=0x05 b=0x0a result=0x1fb
[0] EXP: id=2 op=ALU_AND a=0xf0 b=0x3c result=0x030
[0] EXP: id=3 op=ALU_NOT a=0x0f b=0x00 result=0x0f0
[0] EXP: id=4 op=ALU_SHL a=0x81 b=0x00 result=0x102
[0] EXP: id=5 op=ALU_SHR a=0x81 b=0x00 result=0x040
new object     : id=0 op=ALU_ADD a=0x00 b=0x00 result=0xxxx  unary=0
copy equal     : 1
X result equal : 0
[0] XOR: id=0 op=ALU_XOR a=0xaa b=0x0f result=0x0a5
```

Checking the predictions by hand

Operation	Calculation	Result
ADD 200, 100	300 = `1_0010_1100`	`12c` (bit 8 = carry)
SUB 5, 10	5 - 10 = -5; in nine bits 512 - 5 = 507 = `1_1111_1011`	`1fb` (bit 8 = borrow)
AND F0, 3C	`1111_0000 & 0011_1100` = `0011_0000`	`030`
NOT 0F	`~0000_1111` = `1111_0000`, bit 8 = 0	`0f0`

Operation	Calculation	Result
SHL 81	`{1000_0001, 0}` = `1_0000_0010`	`102`
SHR 81	`{00, 1000_000}` = `0_0100_0000`	`040`
XOR AA, 0F	`1010_1010 ^ 0000_1111` = `1010_0101`	`0a5`

Points to notice

Code	Explanation
`alu_txn x = new();` inside `show`	A declaration with an initialiser is safe here because the function is `automatic`: the initialiser runs on every call.
"new object" line	The constructor has set `op` to a legal value; `result` is X.
`randomize() with { op == ALU_XOR; a == 8'hAA; b == 8'h0F; }`	Inline constraints that pin every field. It shows that the class constraints (`op_dist_c`, `corner_c`) allow these values, and makes the output predictable.
`t.result = alu_predict(t);`	Must come **after** `randomize()`, because `post_randomize()` sets `result` to X.
"X result equal"	The copy's result is corrupted to X; `compare()` uses `===` and reports the difference.

> **Common mistake:** Writing `alu_txn x = new();` as a declaration inside a `begin` ... `end` block of an `initial` block in a module. Variables there are static by default, so the initialiser runs **once**, before time 0, and not each time the block is entered; a loop would then reuse one object (the shared-handle bug again), and some simulators reject the declaration unless `static` or `automatic` is written explicitly. In module-level code, declare the handle and call `new()` in a separate statement, or declare it `automatic`.

Interview questions

Q1. Describe the transaction class you would write for an ALU. `rand bit [7:0] a, b;` and `rand alu_op_e op;` for stimulus; `logic [8:0] result;` for the response; an `id` for tracking. Constraint blocks for the opcode distribution and corner-value bias. Methods: constructor, `convert2string`, `print`, `copy`, `compare`, all virtual, and `post_randomize` to clear the result.

Q2. Why is the class placed in a package? So that there is exactly one definition, imported by the testbench top, the tests and every component. Classes declared separately in two scopes are different types even if their text is identical.

Q3. What is a reference model? An independent, usually untimed, model of the DUT's function that computes the expected output from the same inputs the DUT received. The scoreboard compares its output with the actual DUT output.

Q4. Why must the reference model not be copied from the RTL? If the same code, or the same misunderstanding, is in both, they agree with each other even when both are wrong, and the bug is not found. The model is written from the specification, preferably by a different person and in a different style.

Part 4: Verification Planning

Parts 1 to 3 built the object that carries stimulus. This part steps back and asks the larger questions: what are we trying to achieve with that stimulus, how do we decide what to test, and how do we know when to stop?

4.1 What Is Verification?

In plain English

A designer reads the specification and writes RTL. Verification asks one question about that RTL: **does it do what the specification says, in every situation the specification covers?** And it must answer before the design is manufactured, because a chip cannot be patched after it leaves the factory.

Verification is therefore not "running a few tests to see that it works". It is a systematic search for every way in which the design might *not* work.

Technical definition

Functional verification is the process of demonstrating that a design's implementation (RTL) conforms to its specification, before tape-out. The design being checked is the DUT. The work is done mainly by simulation, supported by formal methods, emulation and prototyping.

Related terms that interviewers ask you to distinguish

Term	Question it answers	When	Done on
Verification (pre-silicon)	Does the RTL match the specification?	Before tape-out	RTL and netlist, in simulators, formal tools, emulators
Validation (post-silicon)	Does the manufactured chip work in the real system, and is it what the customer needs?	After silicon returns	Real chips on boards
Testing (manufacturing test, DFT)	Is *this particular chip* free of manufacturing defects?	On every chip produced	Each die, on automatic test equipment

A short way to remember it: verification asks "did we build it right?" (against the specification); validation asks "did we build the right thing?" (against the real need); manufacturing test asks "was this copy fabricated correctly?".

Why verification is needed

Reason	Explanation
Cost of a late bug	A bug found in block-level simulation costs minutes to fix. Found in system simulation, days. Found in silicon, it costs a new mask set, months of schedule and possibly the product's market window. Found by a customer, it can mean a recall
Complexity	The number of possible input sequences and internal states of even a modest design is far beyond anything that can be tried by hand

Reason	Explanation
Human error	Designers misread specifications, forget corner cases and make typing mistakes. A second, independent reading of the specification catches these
Ambiguous specifications	Writing checks forces every vague statement to be resolved. Many "bugs" found by verification are in the specification
Integration	Blocks that work alone may fail together because of mismatched assumptions about an interface

For these reasons verification usually takes more effort than design, and verification engineers commonly outnumber designers on a project.

The main verification techniques

Technique	How it works	Strength	Limitation
Simulation (dynamic verification)	The RTL is executed with stimulus from a testbench; outputs are checked	Flexible, full visibility, works for any design. The subject of this course	Slow for long scenarios; only checks what is simulated
Formal verification	A tool proves mathematically that a property holds for all possible inputs, or finds a counter-example	Exhaustive for the properties written	Limited to smaller blocks; needs skill to write properties and constraints
Emulation / FPGA prototyping	The design is mapped onto special hardware and run thousands of times faster	Can boot software and run real workloads	Poor visibility; long compile times; expensive
Gate-level simulation	The synthesised netlist is simulated, often with timing	Catches synthesis mismatches, reset and X problems, timing-related issues	Very slow; only a small set of tests is run
Static checks (lint, CDC, RDC)	Tools analyse the code without running it	Fast; finds coding errors and clock-domain-crossing problems early	Finds only structural problems, not functional ones

Levels of verification

```
Block / unit level      one module (the ALU)           full control and visibility,
     |                                                 exhaustive corner cases
     v
Subsystem / cluster     several blocks together        interfaces between blocks
     |
     v
Chip / SoC level        the whole design               integration, connectivity,
     |                                                 software-driven scenarios
     v
System level            chip + software + board model  real use cases
```

Most functional bugs should be found at block level, where simulation is fast and every signal is easy to control. Higher levels concentrate on what cannot be seen lower down: how the blocks interact.

What a verification engineer actually does

1. Reads the specification and questions it.
2. Writes the verification plan: the list of things to verify and how.

3. Designs and builds the testbench.
4. Writes tests, runs them, and debugs failures (both testbench bugs and design bugs).
5. Reports design bugs and confirms the fixes.
6. Measures coverage, analyses the holes and closes them.
7. Runs regressions and reports status until the sign-off criteria are met.

> **Key point:** The mindset is the opposite of the designer's. The designer tries to make the design work. The verification engineer tries to make it fail, and success is measured by the bugs found before tape-out.

Interview questions

Q1. What is verification? The process of confirming, before manufacture, that the design implementation behaves as its specification requires, by simulation, formal analysis and related techniques.

Q2. What is the difference between verification and validation? Verification is pre-silicon and checks the design against the specification. Validation is post-silicon and checks that the real chip works in the real system and meets the user's needs.

Q3. What is the difference between verification and testing? In semiconductor usage, "testing" normally means manufacturing test: checking every fabricated chip for physical defects. Verification checks the design itself for functional errors, once, before fabrication.

Q4. Why is verification done at block level if the whole chip will be verified anyway? At block level the design is small, simulations are fast, and every input can be controlled directly, so corner cases can be reached and bugs are cheap to find and debug. At chip level the same corner may be impossible to reach or take hours of simulation.

Q5. Can verification prove that a design has no bugs? No. Simulation can only show the presence of bugs, never their absence. Verification reduces the risk to an acceptable level, and coverage metrics measure how thoroughly the planned behaviour has been exercised.

4.2 What Is a Verification Plan?

In plain English

Before building anything, write down what "verified" will mean for this design: which behaviours must be checked, how each will be stimulated, how each will be checked, and how you will know it has happened. That document is the verification plan. It is the contract between the verification team, the design team and management.

Technical definition

The **verification plan** (also called *vplan*, and in some companies loosely called the test plan) is a document, derived from the design specification, that lists every feature to be verified and, for each, the verification method, the stimulus strategy, the checking mechanism and the coverage metric, together with the testbench architecture, resources, schedule and completion criteria.

Why it is needed

Without a plan	With a plan
Tests are written as ideas occur; features are forgotten	Every feature of the specification is listed and tracked
"Are we done?" is answered by opinion	"Done" is defined by measurable criteria agreed in advance
The testbench is built for the first test and patched afterwards	The testbench architecture is designed for all the planned tests
Design and verification interpret the specification separately	The plan review exposes different interpretations early
Progress cannot be reported	Progress is the fraction of plan items passing and covered

What a verification plan contains

Section	Contents
1. Introduction and scope	The DUT, the specification version it is based on, what is and is not verified at this level
2. Feature list	Every feature extracted from the specification, each with an identifier and a priority
3. Verification strategy	For each feature: simulation, formal, or both; directed or constrained-random; block or chip level
4. Testbench architecture	Block diagram, components, interfaces, reference model, reuse from other projects
5. Stimulus plan	Transactions, constraints, sequences and scenarios needed
6. Checking plan	Scoreboard, reference model, assertions, protocol checkers, end-of-test checks
7. Coverage plan	Functional coverage points and crosses mapped to the features; code coverage goals
8. Test list (the test plan)	The individual tests, see Section 4.3
9. Completion (exit) criteria	The measurable conditions for sign-off
10. Resources and schedule	People, tools, licences, compute, milestones
11. Risks and assumptions	Late specification changes, unavailable models, known limitations

The core of the plan is a table with one row per feature. A shortened example for the ALU (the complete one is in Part 5):

ID	Feature	How stimulated	How checked	How measured
F3	ADD produces `a + b` with carry in bit 8	Random `alu_txn` with `op == ALU_ADD`; corner bias on operands	Scoreboard compares with `alu_predict`	Coverpoint on `op`; cross of operand corners with ADD; carry bin
F11	Outputs are 0 after reset	Reset applied at start and in mid-operation	Assertion on `valid_out` and `result` during reset	Cover property: reset while `valid_in` is high

How it is created and used

```
Specification
     |
     v
Feature extraction  ---->  review with designers and architects
     |
     v
Verification plan (features, strategy, checks, coverage, tests, exit criteria)
     |
     +---->  Testbench is built to implement the plan
     |
     +---->  Tests and coverage are written from the plan
     |
     +---->  Regression results and coverage are mapped back onto the plan
     |
     v
Sign-off: every plan item is covered and passing
```

The plan is a **living document**. It is updated when the specification changes, when a bug reveals a missing scenario, and when coverage analysis shows a hole that the plan did not anticipate. Verification-management tools can link each plan item to a coverage item and show the percentage achieved against each feature; this is called plan-driven or metric-driven verification.

> **How to say it in an interview:** "The verification plan is the document that defines what will be verified and how. I derive it from the specification by listing every feature, and for each one I state how it will be stimulated, how it will be checked and which coverage item proves it was exercised. It also describes the testbench architecture and the exit criteria. It is reviewed with the designers and it is updated throughout the project."

Interview questions

Q1. What is a verification plan? A document derived from the specification that lists the features to verify and, for each one, the stimulus, checking and coverage, plus the testbench architecture and the completion criteria.

Q2. Who writes and who reviews the verification plan? The verification engineer or lead writes it. It is reviewed by the designers, the architects or specification owners, and often by software or system engineers who will use the block.

Q3. When is the verification plan written? At the start, in parallel with the RTL design, as soon as the specification is stable enough. It is then maintained for the life of the project.

Q4. Why is the plan derived from the specification and not from the RTL? The RTL may be wrong; that is what verification is trying to find out. A plan derived from the RTL would describe what the design does, not what it should do.

Q5. What are exit criteria? The measurable conditions under which verification is declared complete: for example all planned tests passing, 100 percent functional coverage, code coverage above an agreed threshold with reviewed exclusions, and no open high-priority bugs.

4.3 What Is a Test Plan?

In plain English

The verification plan says *what* must be verified. The test plan says *which tests* will do it: a concrete list, each test with a name, a purpose, the stimulus it uses, what it expects, and which features it covers.

Technical definition

A **test plan** is the list of test cases (testcases) to be written and run, each described by its objective, the configuration and stimulus it applies, its pass/fail criteria, and the features and coverage items it targets, along with its priority and current status.

Verification plan versus test plan

Usage varies between companies, and many use the two names for one combined document. When they are distinguished, the difference is:

Aspect	Verification plan	Test plan
Answers	What will be verified and how do we know it is done?	Which tests will be run?
Level	Strategy	Execution
Organised by	Feature	Test case
Contains	Features, methods, testbench architecture, checkers, coverage model, exit criteria, schedule	Test names, objectives, stimulus, expected results, priority, status
Relationship	The parent document	A section of, or a companion to, the verification plan
Changes	When the specification or strategy changes	Frequently, as tests are added and results come in

Interview trap: If asked "what is the difference between a verification plan and a test plan?", do not answer that they are completely different documents, and do not answer that they are the same. Say that the verification plan is the overall strategy (features, methodology, checking, coverage, exit criteria) and the test plan is the list of test cases that implements it, and add that many teams keep both in one document.

What one test-plan entry contains

Field	Example
Test name	`alu_add_carry_test`
Objective	Verify ADD when the sum exceeds 255
Features covered	F3, F12
Type	Constrained-random
Stimulus	500 transactions of `alu_carry_txn`
Expected result / checks	Scoreboard reports 0 mismatches; bit 8 of every result is 1

Field	Example
Coverage targeted	Carry bin of the ADD coverpoint
Priority	High
Status	Written / passing / failing / blocked

Kinds of tests found in a test plan

Kind	Purpose
Sanity (smoke) test	A short test that checks the design is alive: reset, one or two basic operations. Run first, and after every code change
Feature (directed) test	One specific feature or scenario with chosen values
Constrained-random test	Many random transactions within constraints; the main source of coverage
Corner-case test	Boundary values and rare conditions: maximum, minimum, overflow, full, empty
Error (negative) test	Illegal or erroneous stimulus, to check that the design detects it or recovers
Reset test	Reset at start, in the middle of an operation, repeated resets
Stress test	Maximum rate, back-to-back transactions, long duration
Register test	Read and write every register; reset values; access types
Regression	Not a single test: the complete collection, re-run regularly with many seeds

Interview questions

Q1. What is a test plan? The list of test cases needed to carry out the verification plan, each with its objective, stimulus, expected outcome, the features it covers and its status.

Q2. What is the difference between a verification plan and a test plan? The verification plan is the strategy document organised by feature: what to verify, how to stimulate, check and measure it, the testbench architecture and the exit criteria. The test plan is the list of concrete tests, organised by test case. In many teams the test plan is a section of the verification plan.

Q3. What is a sanity test? A short, basic test (reset plus a few simple operations) that shows the DUT and testbench are functional. It is run before anything else and after every change, so that a broken build is detected in minutes.

Q4. What is a negative test? A test that deliberately applies illegal or erroneous stimulus to confirm that the design handles it as specified, for example by flagging an error and continuing to operate.

Q5. What is a regression? The repeated running of the whole test suite, with different random seeds, to confirm that changes to the design or testbench have not broken anything that previously passed.

4.4 How to Derive Test Scenarios from a Specification

In plain English

Interviewers like to hand a candidate a small specification (a FIFO, a counter, an arbiter, a traffic-light controller) and ask "how would you verify this?". They are not looking for code. They want to see a method: a repeatable way of turning a page of specification into a list of scenarios, without missing the unusual ones.

The method

Step 1. Read the specification and mark every requirement. Each sentence that says what the design *shall*, *must* or *will* do is one or more features. So is each row of a table and each waveform.

Step 2. List the interfaces. For every port: direction, width, protocol, timing, legal values, reset value.

Step 3. List the features by category. Going through a fixed set of categories prevents omissions:

Category	Questions to ask
Reset and initialisation	What are the outputs during and after reset? What happens if reset arrives in the middle of an operation? Synchronous or asynchronous?
Basic function	What does each operation or mode do in the normal case?
Configuration and modes	What parameters, registers or mode pins change the behaviour? All combinations?
Data values	Minimum, maximum, zero, all ones, alternating patterns, a single bit set
Boundaries	Full, empty, overflow, underflow, wrap-around, the last address
Protocol and timing	Back-to-back operations, idle gaps, the shortest and longest allowed timing, signals asserted simultaneously
Sequences	Does the result depend on what came before? Which orders of operations are interesting?
Concurrency	What if two things happen in the same cycle (read and write, request and reset)?
Errors and illegal input	What is illegal? What must the design do when it happens?
Performance	Latency, throughput, anything with a number in the specification

Step 4. Apply systematic test-design techniques to each feature.

Technique	Idea	Example
Equivalence partitioning	Divide the input space into classes that the design should treat alike; test at least one member of each	Operands: zero, small, large, maximum
Boundary value analysis	Bugs cluster at the edges of each class; test just below, at, and just above each boundary	254, 255, and the sum that first produces a carry (256)
Cross combinations	Features that work alone may fail together	Every operation with every operand class
State and transition coverage	Visit every state and every transition, including those that should be impossible	Empty → not empty → full → not full → empty
Negative testing	Do what the specification forbids	Write when full; read when empty

Technique	Idea	Example
"What if?" questions	Ask about every pair of events: what if they coincide, or come in the opposite order?	What if reset is released in the same cycle as a request?

Step 5. For each scenario, decide how it is stimulated, how it is checked and how its occurrence is measured. This gives one row of the verification plan.

Step 6. Review the list with the designer. Ask "what are you worried about in this block?". The answers become high-priority scenarios.

Worked example: a synchronous FIFO

The specification (one paragraph). A synchronous FIFO, 8 bits wide and 16 entries deep. Ports: `clk`, `rst_n`, `wr_en`, `din[7:0]`, `rd_en`, `dout[7:0]`, `full`, `empty`. Data written while `wr_en` is high and the FIFO is not full is stored. Data is read in the order written, while `rd_en` is high and the FIFO is not empty. `full` is high when 16 entries are stored; `empty` is high when none are. A write when full and a read when empty are ignored. Reset empties the FIFO.

Scenarios derived by the method

#	Category	Scenario	Check
1	Reset	After reset, `empty` = 1, `full` = 0	Assertion
2	Reset	Reset while the FIFO holds data: it becomes empty	Assertion; scoreboard is flushed
3	Basic	Write one item, read it back	Scoreboard: data read equals data written
4	Basic	Write N items, read N items: order is preserved	Scoreboard models a queue
5	Boundary	Fill completely: `full` rises exactly on the 16th write	Assertion on `full` versus a model count
6	Boundary	Empty completely: `empty` rises exactly on the last read	Assertion
7	Boundary	Pointer wrap-around: write and read more than 16 items in total	Scoreboard
8	Error	Write when full: the item is dropped, stored data is not corrupted	Scoreboard does not expect the dropped item
9	Error	Read when empty: `dout` and the state do not change	Scoreboard; assertion that `empty` stays high
10	Concurrency	Read and write in the same cycle when partly full: count unchanged	Scoreboard; cover the event
11	Concurrency	Read and write in the same cycle when full	Specification must say what happens; check accordingly
12	Concurrency	Read and write in the same cycle when empty	As above
13	Data	All zeros, all ones, alternating `55` / `AA`, walking one	Scoreboard
14	Timing	Back-to-back writes; back-to-back reads; alternating write/read	Scoreboard
15	Stress	Long random mix of reads and writes with varying rates, so that the FIFO repeatedly reaches full and empty	Scoreboard; coverage on fill level
16	Flags	`full` and `empty` are never high together	Assertion

Scenarios 11 and 12 are valuable even before a single test runs: the specification paragraph does not say what happens, so the verification engineer goes back to the specification owner. Finding such gaps is part of the job.

> **How to say it in an interview:** "I start from the specification and list the interfaces and features. I go through fixed categories so I do not miss anything: reset, normal operation, modes, data values, boundaries, timing, sequences, simultaneous events, and illegal inputs. For each feature I apply boundary-value analysis and consider combinations. Then for each scenario I say how I would stimulate it, how I would check it, and which coverage point shows that it happened."

Interview questions

Q1. Given a specification, how do you come up with test scenarios? Use the method above: extract features category by category, apply equivalence partitioning, boundary analysis, combinations, sequences and negative cases, and for each scenario define stimulus, check and coverage.

Q2. What is boundary value analysis? Selecting test values at and around the edges of each input or state range, such as 0, 1, maximum minus 1, maximum, and the transition into overflow, because design errors concentrate at boundaries.

Q3. What is equivalence partitioning? Dividing the possible inputs into groups whose members the design is expected to treat in the same way, and testing representative values from each group instead of every value.

Q4. List the corner cases for a FIFO. Full, empty, write when full, read when empty, simultaneous read and write at full, at empty and in between, pointer wrap-around, reset while not empty, and the exact cycle at which `full` and `empty` change.

Q5. What do you do when the specification does not say what should happen in a scenario? Raise it with the specification owner or designer, get the intended behaviour agreed and written into the specification, and then add the scenario to the plan. It must not be decided by looking at what the RTL does.

4.5 Directed Testing

Definition

In **directed testing** the engineer writes each test for one specific scenario, choosing every stimulus value and stating the expected result explicitly.

```
// A directed test for the ALU: every value is chosen by hand
directed(ALU_ADD, 8'hFF, 8'h01);     // carry out of bit 7
directed(ALU_ADD, 8'hFF, 8'hFF);     // largest possible sum
directed(ALU_SUB, 8'h00, 8'h01);     // borrow
directed(ALU_SHL, 8'h80, 8'h00);     // the bit shifted out must appear in result[8]
```

How it works

```
   for each scenario in the test plan:
        write a test  ->  run it  ->  it passes  ->  that scenario is verified
                                                     (and nothing else is)
```

Advantages and disadvantages

Advantages	Disadvantages
Simple to write for a small design; no complex testbench needed	One test verifies one scenario; effort grows in step with the number of scenarios
The scenario tested is exactly known and reproducible	Only scenarios the engineer thought of are tested; unexpected bugs are missed
Quick to reach a specific corner	Tests are tied to particular values and timing, and break when the design changes
Easy to debug: short, and the intent is obvious	Many tests to maintain
Useful from the first day, before the random environment is ready	Coverage rises slowly and levels off below 100 percent

Where directed tests are still used

- Sanity tests at the start of a project.
- Reset behaviour and register reset values.
- Specific corner cases that random stimulus is unlikely to reach.
- Re-creating a reported bug, and keeping that test so the bug cannot return.
- Closing the last coverage holes at the end of a project.

4.6 Constrained-Random Testing

Definition

In **constrained-random testing** (CRT, also called constrained-random verification, CRV) the testbench generates stimulus automatically by randomizing transactions within constraints that keep it legal and steer it toward interesting cases. Results are checked automatically, and functional coverage records which scenarios have occurred.

```
// A constrained-random test for the ALU: the values are chosen by the solver
repeat (1000) begin
  t = new();
  if (!t.randomize()) $fatal(1, "randomize() failed");
  drive_and_check(t);
end
```

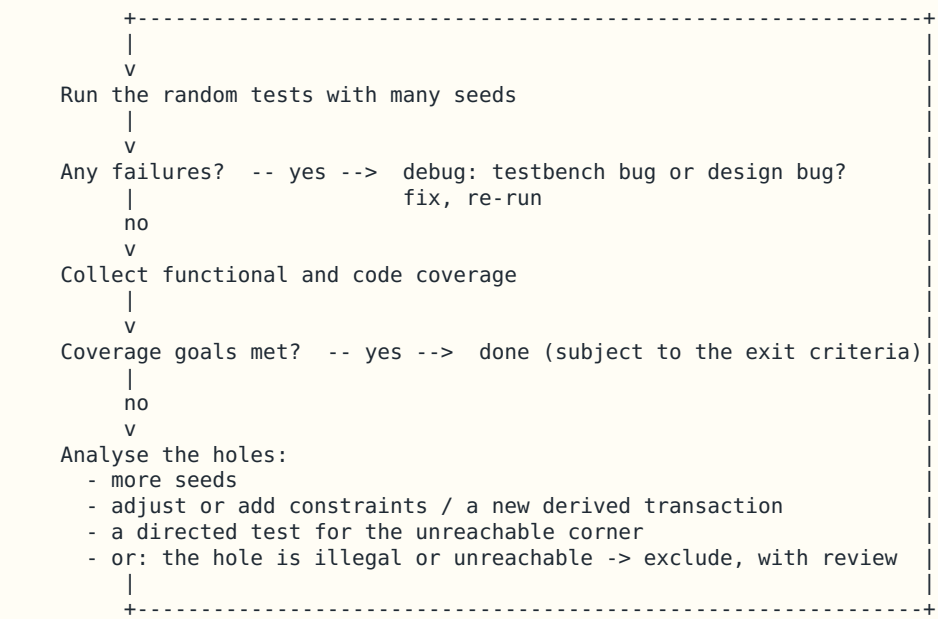
What it requires

Random stimulus alone is not enough. Three things must be present together:

Requirement	Why	Module
Constraints	Unconstrained values are mostly illegal or uninteresting	5, and Part 2 here
Automatic checking (a self-checking testbench)	Nobody can work out by hand the expected results of ten thousand random operations	Sections 4.9, 4.10

Requirement	Why	Module
Functional coverage	With random stimulus you do not know what was tested unless you measure it	10, and Section 4.7

How it works: the coverage-driven loop

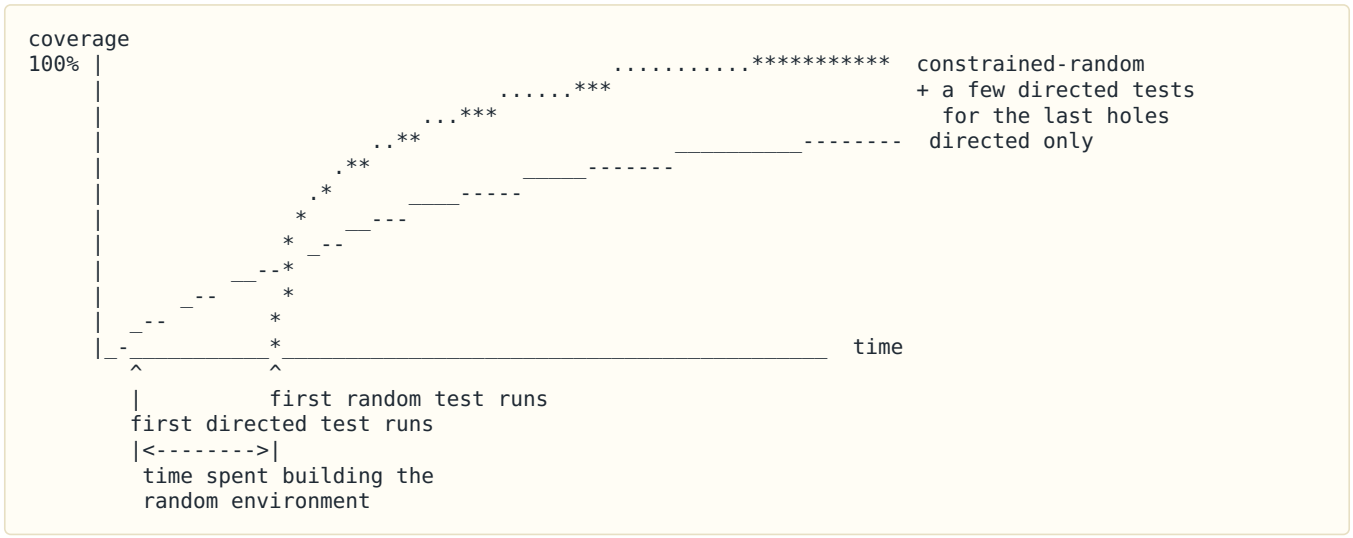


This loop is called **coverage-driven verification**.

Directed versus constrained-random

Aspect	Directed testing	Constrained-random testing
Who chooses the stimulus	The engineer, value by value	The constraint solver, within rules set by the engineer
One test covers	One scenario	Thousands of scenarios; a different set for each seed
Bugs found	Those the engineer anticipated	Also those nobody anticipated
Checking	Often a hand-written expected value	Must be automatic: scoreboard, reference model, assertions
Knowing what was tested	By reading the test	Only through functional coverage
Initial effort	Low: the first test runs on day one	High: an environment must be built first
Effort per additional scenario	High	Low: a new constraint or a new seed
Reproducing a failure	Trivial	Requires the seed
Maintenance when the design changes	Many tests to edit	Mostly the transaction, driver and monitor
Best suited to	Small designs, sanity checks, specific corners	Medium and large designs

Progress over time



Directed testing shows progress immediately but slows down, because each new scenario costs a new test. Constrained-random testing shows nothing while the environment is built, then covers the bulk of the plan quickly, and levels off near the end, where the last few holes are closed with tighter constraints or directed tests. Real projects use both.

> **Interview trap:** "Is constrained-random testing always better than directed testing?" No. For a very small block the environment costs more than it returns. Random stimulus may take a very long time to hit a narrow corner that one directed test reaches at once. And without functional coverage, random testing gives a false sense of security. The correct answer is that the two are complementary.

> **Common mistake:** Saying "random testing" when you mean constrained-random. Purely random stimulus is rarely useful; the constraints, the automatic checking and the coverage are what make it a methodology.

Interview questions

Q1. What is directed testing? A method in which each test is written by hand for one specific scenario, with chosen stimulus values and a known expected result.

Q2. What is constrained-random testing? A method in which the testbench generates stimulus by randomizing transactions under constraints, checks the results automatically, and measures with functional coverage which scenarios were exercised.

Q3. What are the advantages of constrained-random over directed testing? Far more scenarios per unit of effort, discovery of bugs nobody anticipated, and low cost for each additional scenario once the environment exists.

Q4. What are its disadvantages? A larger initial investment in the environment, the need for automatic checking and coverage, failures that are harder to debug because the stimulus was not written by a person, and difficulty reaching some narrow corner cases.

Q5. Why is functional coverage essential with constrained-random testing? Because the stimulus is not written explicitly, there is no other way to know which scenarios actually occurred. Coverage turns "we ran a lot of random tests" into "these planned scenarios were exercised and these were not".

Q6. Do projects still write directed tests? Yes: for sanity checks, reset and register tests, bug reproductions, and for the last coverage holes that random stimulus does not reach.

Q7. What is coverage-driven verification? The iterative flow of running constrained-random tests, measuring coverage, analysing the holes and adjusting constraints or adding tests until the coverage goals in the verification plan are met.

4.7 Functional Coverage

This section treats coverage at the planning level: what it is, why it is in the plan, and how its items are derived. The `covergroup` syntax is the subject of Module 10.

In plain English

After running a thousand random ALU operations you are asked: "Did you test a subtraction that borrows? An addition of `FF` and `FF`? Every opcode with a zero operand?" With random stimulus you cannot answer from memory or by reading the test. Functional coverage is a set of counters, written by the verification engineer, that record whether each scenario of interest actually happened.

Technical definition

Functional coverage is a user-defined metric that measures how much of the design's specified functionality has been exercised by the tests. The engineer describes the values, combinations and sequences that matter (the **coverage model**), and the simulator records which of them occurred. It is derived from the specification and the verification plan, not from the code.

Why it is needed

Question	Answered by
Is the DUT output correct?	Checkers: scoreboard and assertions
Did the test exercise the scenario I care about?	Functional coverage

The two are independent and both are necessary. A test in which the scoreboard reports no errors and coverage is 20 percent has verified 20 percent of the plan. A test with 100 percent coverage and no checking has verified nothing.

How a coverage plan is derived

Each feature and scenario in the verification plan is turned into something countable:

Plan item	Coverage item
Every operation is exercised	A coverage point on `op` with one bin per opcode
Operand corner values	Coverage points on `a` and `b` with bins: zero, maximum, everything else
Every operation with every operand class	A **cross** of the `op` point with the `a` and `b` points
ADD with and without carry	A bin for `result[8]` equal to 0 and to 1, sampled only when `op` is ADD
Back-to-back operations	A transition or sequence item: `valid_in` high in two consecutive cycles

Plan item	Coverage item
Reset during an operation	A cover property

A preview of how the first few rows look in code (explained in Module 10):

```
covergroup alu_cg with function sample(alu_txn t);
  cp_op : coverpoint t.op;                                  // 8 automatic bins
  cp_a  : coverpoint t.a {
            bins zero = {8'h00};
            bins max  = {8'hFF};
            bins mid  = {[8'h01:8'hFE]};
          }
  cx_op_a : cross cp_op, cp_a;                              // 8 x 3 = 24 bins
endgroup
```

Properties of functional coverage

Property	Explanation
Written by hand	It is only as good as the coverage model. A scenario nobody thought of has no bin, and 100 percent says nothing about it
Derived from the specification	It measures intended behaviour, independent of how the RTL is written
Reported as a percentage of bins hit	Merged across all tests and seeds of a regression
Meaningful only with checking	A bin may be hit only in a test that failed; results from failing tests are not counted

4.8 Code Coverage

In plain English

Code coverage answers a simpler, mechanical question: **which parts of the RTL source code were executed by the tests?** If a line of the design never ran, no test can have checked it. The simulator collects this automatically; the engineer writes nothing.

Technical definition

Code coverage is a tool-generated metric that measures how much of the design's implementation (statements, branches, conditions, signal toggles, state-machine states and transitions) has been activated during simulation. It is enabled with a simulator option and is collected on the DUT, not on the testbench.

The types of code coverage

Using this fragment:

```
always_comb begin
  y = '0;                          // statement 1
  if (en && (a > b || c))          // one branch with two outcomes; three conditions
    y = a;                         // statement 2
  else
    y = b;                         // statement 3
end
```

Type	Measures	In the fragment
Statement (line)	Was each executable statement executed?	All three assignments must run. Two tests suffice: one taking the `if`, one taking the `else`
Branch (decision)	Was each branch of every `if`, `case` and `?:` taken, both true and false?	The `if` must evaluate true at least once and false at least once. An `if` with no `else` still has an implicit false branch
Condition	Did each sub-condition of a decision take both values?	`en`, `a > b` and `c` must each be seen true and false
Expression	Did each operand of an expression on the right-hand side control the result?	Same idea, applied to assignments such as `y = p & (q | r)`
Toggle	Did each bit of each signal change 0→1 and 1→0?	Every bit of `a`, `b`, `y`, and `en`, `c`
FSM state	Was each state of each state machine entered?	Not applicable here
FSM transition (arc)	Was each possible state-to-state transition taken?	Not applicable here

Why the stricter types matter: with `en=1, a>b=1, c=0` and `en=0`, statement and branch coverage are both 100 percent, yet `c` has never been 1. If the designer wrote `||` where `&&` was intended, these two tests would not show it. Condition coverage reports that `c` was never exercised.

Code coverage versus functional coverage

Aspect	Code coverage	Functional coverage
Measures	How much of the implementation was executed	How much of the specified functionality was exercised
Derived from	The RTL source	The specification and verification plan
Written by	Nobody; generated by the tool	The verification engineer
Effort	A simulator switch	Design, coding and review of the coverage model
Can detect a missing feature	No. Code that was never written cannot be uncovered	Yes, if the plan lists the feature
Can detect unexercised code	Yes	No
Can detect unreachable (dead) code	Yes	No
Knows about combinations and sequences	No	Yes, through crosses and transitions
100 percent means	Every line, branch and so on was activated	Every scenario in the model occurred

Aspect	Code coverage	Functional coverage
100 percent does **not** mean	That the outputs were checked, or that every feature exists	That the model was complete

Reading the two together

Functional coverage	Code coverage	Interpretation	Action
High	High	The plan has been exercised and so has the code	Check the remaining criteria and sign off
High	Low	The planned scenarios occurred but part of the RTL never ran	The coverage model is missing features, or there is dead or unreachable code, or an undocumented feature exists. Review the uncovered code with the designer
Low	High	The code was executed but the planned scenarios did not occur	Stimulus is too shallow: lines were touched without the interesting values, combinations or sequences. Improve constraints, add tests. (Or the model contains unreachable bins)
Low	Low	Early in the project, or the tests are ineffective	More tests, more seeds, better constraints

Interview trap: "We have 100 percent code coverage. Are we done?" No. Code coverage shows that the code was executed, not that its results were checked, and it cannot reveal a feature the designer forgot to implement. It is a necessary condition for sign-off, not a sufficient one.

Interview trap: "We have 100 percent functional coverage. Are we done?" No. The coverage model was written by people and may be incomplete; code coverage is used to find what it missed. Sign-off needs both, plus all tests passing and no open critical bugs.

Coverage holes, exclusions and waivers

A **coverage hole** is an item that has not been hit. Each one is analysed:

Cause	Response
The stimulus never produced the condition	More seeds, new constraints, or a directed test
A constraint prevents the condition	Relax the constraint if the condition is legal
The condition is illegal or cannot happen by design	Mark the bin as illegal or ignored; exclude the code with a written justification, reviewed by the designer
The code is dead or left over	Remove it from the RTL
The feature is not implemented	A design bug: report it

An exclusion without a reviewed justification is a way of hiding a problem. Exclusion files are part of what is audited at sign-off.

Interview questions

Q1. What is code coverage? A tool-generated measure of how much of the RTL (statements, branches, conditions, toggles, FSM states and transitions) was exercised during simulation.

Q2. What is functional coverage? A user-defined measure of how many of the scenarios, values and combinations listed in the verification plan actually occurred during simulation.

Q3. What are the types of code coverage? Statement (line), branch, condition, expression, toggle, and FSM state and transition coverage.

Q4. What is the difference between branch and condition coverage? Branch coverage checks that each decision as a whole evaluated both true and false. Condition coverage checks that each individual sub-expression inside the decision took both values.

Q5. Code coverage is 100 percent but functional coverage is 60 percent. What does that mean? All the code was executed, but 40 percent of the planned scenarios did not occur: the tests touched the code without producing the interesting combinations or sequences. Stimulus needs to be improved, or some bins are unreachable and need review.

Q6. Functional coverage is 100 percent but code coverage is 80 percent. What does that mean? A fifth of the RTL was never executed even though every planned scenario occurred. Either the coverage model omits some features, or the RTL contains dead, unreachable or undocumented logic. The uncovered code is reviewed with the designer.

Q7. Can 100 percent code coverage and 100 percent functional coverage guarantee a bug-free design? No. Coverage shows what was exercised, not that it was checked correctly, and a scenario missing from both the plan and the code is invisible to both. They give confidence, not proof.

Q8. Who writes functional coverage, and from what? The verification engineer, from the specification and the verification plan, never from the RTL.

4.9 The Self-Checking Testbench

In plain English

A testbench that only applies stimulus leaves the real work to a person: looking at waveforms or log files and deciding whether the outputs are right. That is slow, cannot be repeated nightly, and is unreliable; nobody inspects ten thousand random results accurately.

A **self-checking testbench** decides by itself. It knows, for every stimulus, what the DUT should produce; it compares that with what the DUT did produce; and at the end it prints a single verdict: PASSED or FAILED.

Technical definition

A self-checking testbench is one that automatically determines the correctness of the DUT's responses during simulation, by comparing them with expected values obtained from a reference model, a predictor or assertions, and that reports pass or fail without manual inspection.

Why it is needed

Need	Why self-checking is the only answer
Constrained-random stimulus	Expected values cannot be written by hand for stimulus that is not known in advance
Regressions	Hundreds of tests run unattended every night; a script must be able to read the result
Repeatability	The same checks are applied every time, with no human judgement
Speed of debug	A failure is reported at the time and transaction where it occurs, not discovered later in a waveform

The ingredients

Ingredient	Role
Stimulus generator	Produces the transactions
Driver	Applies them to the DUT
Monitor	Observes the DUT's inputs and outputs and rebuilds transactions
Predictor / reference model	Computes the expected response from the observed inputs
Comparator (scoreboard)	Compares actual with expected and counts matches and mismatches
Assertions	Check protocol and timing rules continuously
End-of-test report	Prints the totals and the final verdict

Ways of obtaining a self-check

Method	How	Suitable for
Reference model + scoreboard	A model computes the expected output for each transaction	Data-transforming designs: ALU, filter, encoder, processor
Data integrity scoreboard	Whatever goes in must come out, unchanged and in order	FIFOs, memories, bridges, routers
Assertions	Properties checked every cycle	Protocol and control: handshakes, flags, one-hot, timing
Inverse operation	Apply the operation and its inverse; expect the original	Encrypt/decrypt, encode/decode, write then read back
Golden file	Outputs compared with a stored file from a trusted earlier run	Legacy flows. Fragile, and only valid for fixed stimulus
Hand-written expected values	The test states the expected result	Directed tests only

Complete example: a self-checking ALU testbench

This testbench uses the final `alu_txn` class and the `alu_predict` reference model from Section 3.5. It is deliberately flat: one module, no interface, no mailbox. Its purpose is to show the *idea* of self-checking with both directed and constrained-random stimulus. Module 9 restructures the same ideas into separate generator, driver, monitor and scoreboard classes.

The design is the ALU from Module 1, repeated here so that the example folder is complete.

alu.sv

```
module alu
  import alu_pkg::*;
(
  input  logic       clk,
  input  logic       rst_n,
  input  logic       valid_in,
  input  logic [7:0] a,
  input  logic [7:0] b,
  input  alu_op_e    op,
  output logic       valid_out,
  output logic [8:0] result
);

  logic [8:0] result_nxt;

  always_comb begin
    unique case (op)
      ALU_ADD: result_nxt = a + b;
      ALU_SUB: result_nxt = a - b;
      ALU_AND: result_nxt = {1'b0, a & b};
      ALU_OR : result_nxt = {1'b0, a | b};
      ALU_XOR: result_nxt = {1'b0, a ^ b};
      ALU_NOT: result_nxt = {1'b0, ~a};
      ALU_SHL: result_nxt = {a, 1'b0};
      ALU_SHR: result_nxt = {2'b00, a[7:1]};
    endcase
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      valid_out <= 1'b0;
      result    <= '0;
    end
    else begin
      valid_out <= valid_in;
      if (valid_in) result <= result_nxt;
    end
  end

endmodule
```

tb_alu_selfcheck.sv

```
// Compile order: alu_pkg.sv, alu.sv, alu_tb_pkg.sv (includes alu_txn.sv), this file
module tb_alu_selfcheck;
  import alu_pkg::*;
  import alu_tb_pkg::*;

  // ---------------- DUT connections ----------------
  logic       clk = 0;
  logic       rst_n;
  logic       valid_in;
  logic [7:0] a, b;
  alu_op_e    op;
  logic       valid_out;
  logic [8:0] result;

  alu dut (.*);

  always #5 clk = ~clk;

  // ---------------- bookkeeping ----------------
  int unsigned n_txn;         // transactions checked
  int unsigned n_err;         // mismatches found
  bit [7:0]    op_seen;       // one bit per opcode: a hand-made coverage record
  alu_txn      t;

  // ---------------- drive one transaction and check the response ----------------
```

```
  task automatic drive_and_check(alu_txn txn);
    logic [8:0] expected;

    @(negedge clk);                       // drive away from the sampling edge
    a        = txn.a;
    b        = txn.b;
    op       = txn.op;
    valid_in = 1'b1;

    @(negedge clk);                       // the DUT sampled at the posedge in between
    valid_in   = 1'b0;
    txn.result = result;                  // ACTUAL  : captured from the DUT
    expected   = alu_predict(txn);        // EXPECTED: from the reference model

    n_txn++;
    op_seen[txn.op] = 1'b1;

    if (valid_out !== 1'b1 || txn.result !== expected) begin
      n_err++;
      $error("[%0t] MISMATCH {%s} expected=0x%h valid_out=%b",
             $time, txn.convert2string(), expected, valid_out);
    end
  endtask

  // ---------------- a directed transaction: every value chosen by hand ----------------
  task automatic directed(alu_op_e op_i, bit [7:0] a_i, bit [7:0] b_i);
    alu_txn d = new();
    d.id = n_txn;
    d.op = op_i;
    d.a  = a_i;
    d.b  = b_i;
    drive_and_check(d);
  endtask

  // ---------------- the test ----------------
  initial begin
    // reset
    rst_n    = 1'b0;
    valid_in = 1'b0;
    a        = '0;
    b        = '0;
    op       = ALU_ADD;
    repeat (2) @(posedge clk);
    if (valid_out !== 1'b0 || result !== 9'd0) begin
      n_err++;
      $error("[%0t] outputs not zero during reset", $time);
    end
    rst_n = 1'b1;

    // directed corner cases
    directed(ALU_ADD, 8'hFF, 8'hFF);
    directed(ALU_ADD, 8'h00, 8'h00);
    directed(ALU_SUB, 8'h00, 8'hFF);
    directed(ALU_SUB, 8'h7F, 8'h7F);
    directed(ALU_SHL, 8'h80, 8'h00);
    directed(ALU_SHR, 8'h01, 8'h00);

    // constrained-random
    repeat (200) begin
      t    = new();
      t.id = n_txn;
      if (!t.randomize()) $fatal(1, "randomize() failed");
      drive_and_check(t);
    end

    // end-of-test report
    $display("transactions : %0d", n_txn);
    $display("opcodes seen : %0d of 8", $countones(op_seen));
    $display("errors       : %0d", n_err);
    if (n_err == 0 && n_txn > 0 && $countones(op_seen) == 8) $display("TEST PASSED");
    else                                                     $display("TEST FAILED");
    $finish;
  end
```

```
endmodule
```

EXPECTED OUTPUT

```
transactions : 206
opcodes seen : 8 of 8
errors       : 0
TEST PASSED
```

The output is the same for every seed as long as the design is correct. (With 200 random transactions, the chance that one of the eight opcodes never appears is negligible.)

Line by line

Code	Explanation
`import alu_tb_pkg::*;`	Brings in `alu_txn` and `alu_predict`.
`alu dut (.*);`	The DUT is connected to module-level signals of the same names.
`int unsigned n_txn, n_err;`	The counters on which the final verdict is based.
`bit [7:0] op_seen;`	A hand-made stand-in for functional coverage: bit *n* is set when opcode *n* has been driven. Module 10 replaces it with a covergroup.
`@(negedge clk);` then the assignments	Stimulus changes on the falling edge, half a cycle before the DUT samples, so there is no race at the rising edge. (Module 8 replaces this with a clocking block.)
Second `@(negedge clk);`	Between the two falling edges there was one rising edge, at which the DUT sampled `valid_in` = 1 and registered the result.
`txn.result = result;`	The **actual** value is stored in the transaction, in a four-state field.
`expected = alu_predict(txn);`	The **expected** value is computed from the transaction's inputs, independently of the DUT.
`if (valid_out !== 1'b1 || txn.result !== expected)`	The comparison. `!==` is used so that an X or Z on either DUT output is a mismatch.
`$error(...)`	Reports the failure at the moment it happens, with the time and the whole transaction. The simulation continues, so that one run can reveal several problems.
`task automatic directed(...)`	Builds a transaction from hand-chosen values and sends it down the **same** drive-and-check path as the random ones. Directed and random stimulus share one checker.
Reset check	The outputs are checked while reset is active.
`directed(ALU_ADD, 8'hFF, 8'hFF);` and the others	The corners from the plan: largest sum, zero sum, largest borrow, `a == b`, the bit shifted out to the left, the bit shifted out to the right.
`t = new();` inside the loop	A new object per transaction. `t` is declared at module level and `new()` is a separate statement, for the reason given at the end of Section 3.5.
End-of-test report	Totals, then one unambiguous line.
`n_err == 0 && n_txn > 0 && ...`	A test passes only if there were no errors **and** something was actually checked **and** the intended stimulus occurred.

Seeing it catch a bug

Change one line of `alu.sv` to model a designer's mistake, for example `ALU_SUB: result_nxt = b - a;`, and run again. The testbench now prints one `$error` line for each subtraction in which `a` and `b` differ, such as

```
[70] MISMATCH {id=2 op=ALU_SUB a=0x00 b=0xff result=0x0ff} expected=0x101 valid_out=1
```

and finishes with a non-zero error count and `TEST FAILED`. (The exact wording and prefix of an `$error` message depend on the simulator.) Notice that the directed test with `a == b` (`7F - 7F`) does **not** detect this bug, because both orders give zero. A test passing does not mean a feature is correct; it means that particular stimulus did not expose a problem. This is the argument for many random transactions.

Deliberately inserting a bug to confirm that the testbench reports it is a standard practice. A testbench that has never been seen to fail has not been shown to check anything.

Requirements for a trustworthy self-checking testbench

Requirement	Reason
The expected value comes from an independent source	A model copied from the RTL agrees with the RTL's bugs
Comparisons use `===` / `!==`	X and Z on DUT outputs must fail
Every transaction is checked	A checker that silently skips transactions produces false passes
The end-of-test report states how many checks were performed	"0 errors" with 0 checks must be a failure
Nothing is left over at the end	Expected items never matched, or DUT outputs never expected, are errors
The verdict is a single, fixed, searchable line	Regression scripts search the log for it
The absence of the PASS line counts as failure	A test that crashed or timed out prints nothing
A timeout (watchdog) ends a hung simulation	Otherwise a deadlock looks like a long-running test

> **Common mistake:** A testbench whose end-of-test code prints "TEST PASSED" unconditionally, or only when `n_err == 0`, without confirming that any comparison took place. If the monitor never sees a transaction (wrong clock edge, `valid_out` never high), the error count is zero and the test passes while checking nothing.

> **Interview trap:** "What is the difference between a self-checking testbench and a testbench with a `$display` of the outputs?" Printing outputs is not checking. A self-checking testbench contains the expected values (or a way to compute them) and makes the pass/fail decision itself.

Interview questions

Q1. What is a self-checking testbench? A testbench that determines automatically whether the DUT's outputs are correct, by comparing them with expected values from a reference model or with assertions, and that reports pass or fail without anyone examining waveforms.

Q2. Why is it necessary for constrained-random verification? The stimulus is generated at run time and differs for every seed, so expected results cannot be prepared by hand. They must be computed and compared automatically.

Q3. What are the components of a self-checking testbench? A stimulus generator, a driver, a monitor, a predictor or reference model, a scoreboard that compares, assertions for protocol checks, and an end-of-test report.

Q4. How does a regression script know whether a test passed? It searches the log for a fixed pass message and for error or fatal messages (or reads the simulator's error count). A test passes only if the pass message is present and no errors are reported.

Q5. What is a false pass and how can it happen? A test that reports PASS although the design is wrong or was not checked. Causes: comparison with `!=` when the output is X; the monitor or scoreboard never receiving transactions; a reference model that shares the RTL's bug; a pass message printed unconditionally.

Q6. How do you gain confidence in the testbench itself? Insert known bugs into the design (or corrupt a transaction in the testbench) and confirm that the checkers report them; review the checkers against the specification; and confirm that the number of checks performed is what the test intended.

4.10 Expected Versus Actual Output Comparison

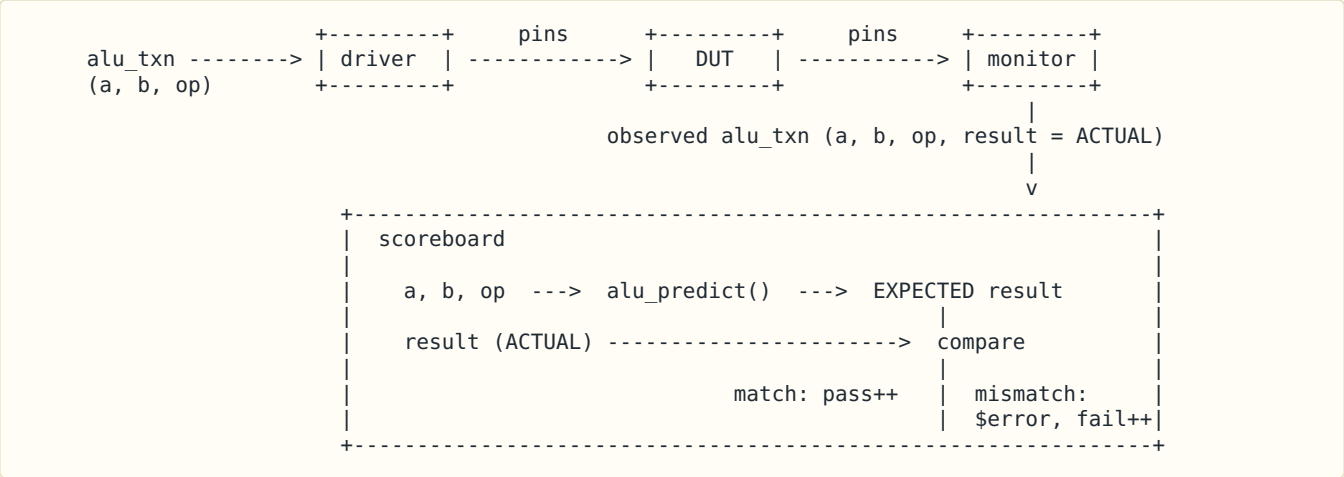
In plain English

Every check in verification has the same shape: obtain the value the DUT **should** produce, obtain the value it **did** produce, compare. This section looks at where each of the two values comes from and at the ways the comparison can go wrong.

Definitions

Term	Meaning	Source
Actual output	What the DUT really produced	Sampled from the DUT's output pins by the monitor
Expected output	What the specification says it must produce for the given input	Computed by a reference model or predictor from the inputs, or stated by the test
Scoreboard	The component that receives both and compares them	Module 9

The flow of data



A point that surprises beginners: the scoreboard takes the **inputs** from the monitor too, not from the generator. The monitor reports what was really on the DUT's pins. If the driver has a bug and drives something different from what the generator asked for, the prediction is still made from what the DUT actually received, and the comparison stays valid.

Where the expected value comes from

Source	Description	Example
Reference model (golden model)	An independent model of the function, in SystemVerilog or in C/C++ through DPI	`alu_predict` ; an instruction-set simulator for a processor
Predictor	A simplified model that transforms an input transaction into the expected output transaction	Expected FIFO output = the oldest input not yet read
The stimulus itself	For designs that only transport data, expected output equals input	FIFO, bridge, switch
The test	A hand-written value	Directed tests
Assertions	The expected behaviour is expressed as a property over time	"`valid_out` is high exactly one cycle after `valid_in`"

Kinds of comparison

Kind	Used when	Data structure in the scoreboard
Immediate, one-to-one	Each output corresponds to exactly one input and arrives in order with fixed latency	None; compare as each transaction arrives. The ALU is this case
In-order	Outputs come in the same order as inputs but after a variable delay	A queue of expected transactions; each actual is compared with the front of the queue
Out-of-order	Outputs may overtake each other	An associative array of expected transactions, indexed by an ID carried in the transaction
Transaction-level (untimed)	Only values matter	Compare fields
Cycle-accurate	The exact cycle of each output matters	Compare values and times; usually done with assertions

An in-order scoreboard, in outline:

```
alu_txn      exp_q[$];          // expected transactions waiting for their actual
int unsigned n_pass, n_fail;

function void write_expected(alu_txn t);
  exp_q.push_back(t.copy());
endfunction

function void write_actual(alu_txn act);
  alu_txn exp;
  if (exp_q.size() == 0) begin
    n_fail++;
    $error("unexpected output: {%s}", act.convert2string());
    return;
  end
  exp = exp_q.pop_front();
  if (exp.compare(act)) n_pass++;
  else begin
    n_fail++;
    $error("MISMATCH expected {%s} actual {%s}",
           exp.convert2string(), act.convert2string());
  end
endfunction

function void report();
  if (exp_q.size() != 0) begin
    n_fail++;
    $error("%0d expected transactions were never observed", exp_q.size());
  end
  $display("scoreboard: %0d passed, %0d failed", n_pass, n_fail);
endfunction
```

All three transaction methods from Part 3 appear here: `copy()` so that the stored object cannot be changed by its sender, `compare()` for the verdict, and `convert2string()` for the message.

What a good mismatch message contains

Item	Why
Simulation time	To find the place in the waveform
Transaction ID	To find the related generator, driver and monitor lines in the log
The full input	To reproduce the case
Expected and actual values, in hex	To see which bits differ
The name of the check	To know which rule was violated

The ways a comparison can be wrong

Problem	Effect	Prevention
`!=` instead of `!==`	X on the output is not reported (false pass)	Always `!==` for DUT data
Handles compared instead of contents	Always a mismatch (false fail)	`compare()`
Monitor samples on the wrong clock edge or in a race with the DUT	Old or unstable values captured; intermittent false fails	Clocking blocks (Module 8)
Actual stored in a two-state variable	X becomes 0 (false pass)	Four-state fields for DUT outputs

Problem	Effect	Prevention
Reference model has the same bug as the RTL	Bug not detected	Independent model, written from the specification
Reference model wrong, RTL right	False fail	Debug decides; the specification is the judge
Expected and actual out of step by one transaction	Every comparison after the first slip fails	End-of-test check on the queue; IDs in transactions
Nothing compared at all	Zero errors (false pass)	Require a non-zero number of checks
Width or signedness of the model differs from the DUT	Mismatches only on overflow or when the MSB is set	Model at the DUT's exact widths (Module 1)

Key point: When expected and actual differ, there are always three suspects: the design, the testbench (stimulus, monitor, reference model), and the specification. Do not report a design bug until you have checked that the testbench is right and that the specification really requires what the testbench expects.

Verification application: Debugging a mismatch follows a fixed routine. (1) Read the scoreboard message: time, ID, expected, actual. (2) Find the same ID in the generator and driver lines to confirm what was sent. (3) Open the waveform at that time and confirm that the pins match what the monitor reported. (4) Work out by hand, from the specification, what the result should be. (5) Whichever of expected or actual disagrees with the specification identifies the faulty party.

Interview questions

Q1. Where do the expected and the actual values come from? The actual value is sampled from the DUT outputs by the monitor. The expected value is computed by a reference model or predictor from the inputs that the DUT received, or is written explicitly in a directed test.

Q2. Why does the scoreboard use the inputs observed by the monitor and not the transaction created by the generator? The monitor reports what was really applied to the DUT. If the driver modifies, delays or corrupts a transaction, a prediction based on the generator's copy would be wrong. It also lets the same scoreboard be reused when the stimulus comes from another block instead of a generator.

Q3. How does a scoreboard handle a DUT with variable latency? It stores expected transactions in a queue as inputs are observed, and compares each observed output with the front of the queue. For out-of-order responses it uses an associative array keyed by a transaction ID.

Q4. What checks should a scoreboard make at the end of the test? That the queue of expected transactions is empty (nothing was lost by the DUT), that no unexpected outputs were received, and that the number of comparisons performed is greater than zero and matches the number of transactions sent.

Q5. A mismatch is reported. How do you decide whether it is a design bug or a testbench bug? Compute the correct result by hand from the specification for the reported input. If the DUT's actual value agrees with the specification, the reference model or monitor is wrong. If the expected value agrees with the specification and the waveform confirms the actual value, it is a design bug. If the specification is unclear, it is a specification issue to be resolved with its owner.

Q6. What is a golden reference model? A trusted, independent implementation of the design's intended function, often written at a higher level of abstraction (untimed SystemVerilog, C or C++), whose outputs are taken as the expected values.

Part 5: The Complete Verification Flow, Worked Through for the ALU

Part 4 described the pieces. This part applies them, in order, to one design, so that you can see what each step produces and how one leads to the next. The design is the 8-bit ALU from Module 1. It is small enough to keep every table on one page, and the steps are the same ones used for a design a thousand times larger.

5.1 The Flow at a Glance

```
1. Understand the specification  ------>  list of requirements, list of questions
         |
         v
2. Extract the features          ------>  feature list (F1, F2, ...)
         |
         v
3. Write the verification plan   ------>  per feature: stimulus, check, coverage
         |
         v
4. Design the testbench          ------>  architecture diagram, component list
         |
         v
5. Write the test plan           ------>  list of tests (T1, T2, ...)
         |
         v
6. Write the coverage plan       ------>  coverage items (C1, ...), assertions (A1, ...)
         |
         v
7. Build, bring up, run, debug   ------>  passing tests, bug reports, fixes
         |
         v
8. Regression                    ------>  pass rate over all tests and many seeds
         |
         v
9. Coverage closure              ------>  holes analysed and closed or justified
         |
         v
10. Sign-off                     ------>  exit criteria met, results reviewed
```

Steps 1 to 6 are planning and are done largely before any testbench code exists. Steps 7 to 9 form a loop that is repeated many times. Step 10 happens once.

Step	Main deliverable	Who is involved
1	Clarified specification	Verification engineer, designer, architect
2, 3	Verification plan	Verification engineer writes; designer and architect review
4	Testbench architecture	Verification engineer or lead
5, 6	Test plan, coverage plan	Verification engineer; reviewed by designer
7	Testbench code, tests, bug reports	Verification engineer; designer fixes RTL
8	Regression reports	Verification engineer; automated infrastructure
9	Coverage reports, exclusion list	Verification engineer; designer reviews exclusions
10	Sign-off review	Whole team and management

5.2 Step 1: Understand the Specification

The specification, as numbered requirements

Rewriting the specification as a numbered list is the first act of verification. Each line can later be traced to a check and a coverage item.

Req	Requirement
R1	Inputs: `a[7:0]`, `b[7:0]`, `op[2:0]`, `valid_in`. Outputs: `result[8:0]`, `valid_out`. Clock `clk`; reset `rst_n`
R2	An operation is accepted at a rising edge of `clk` at which `valid_in` is 1
R3	The result of an accepted operation appears on `result` in the following cycle, with `valid_out` = 1
R4	`valid_out` is 0 in a cycle that does not follow an accepted operation
R5	`ALU_ADD`: `result` = `a + b`; bit 8 is the carry
R6	`ALU_SUB`: `result` = `a - b` computed in nine bits; bit 8 is the borrow
R7	`ALU_AND`, `ALU_OR`, `ALU_XOR`: bitwise operation on `a` and `b`; bit 8 is 0
R8	`ALU_NOT`: `result[7:0]` = `~a`; bit 8 is 0
R9	`ALU_SHL`: `a` shifted left by one; the old `a[7]` appears in bit 8; bit 0 is 0
R10	`ALU_SHR`: `a` shifted right by one; bits 8 and 7 are 0
R11	`rst_n` is asynchronous and active low. While it is low, `valid_out` = 0 and `result` = 0

Questions raised while reading

A specification this short leaves gaps. Finding them is part of the work, and each answer is added to the specification.

#	Question	Agreed answer
Q-1	What is the value of `result` in a cycle where `valid_out` is 0?	Not specified; `result` is only meaningful when `valid_out` is 1. The testbench shall not check it otherwise
Q-2	May `valid_in` be high in consecutive cycles (back-to-back operations)?	Yes. There is no back-pressure; one operation can be accepted every cycle
Q-3	Is `b` used by the single-operand operations (NOT, SHL, SHR)?	No. `b` must have no effect on the result
Q-4	What happens if `rst_n` goes low while an operation is in progress?	The operation is discarded; outputs go to 0 immediately and stay 0 until a new operation is accepted after reset is released
Q-5	May `a`, `b` and `op` change while `valid_in` is 0?	Yes; they are ignored
Q-6	Are all eight values of `op` legal?	Yes; there are no reserved encodings

The answers Q-2, Q-3 and Q-4 each produce a new feature that was not in the original text.

> **Key point:** A question such as Q-1 must be settled by the specification owner, not by reading the RTL. The RTL happens to hold the previous `result` when `valid_in` is 0, but if the testbench checked for that, it would be verifying an implementation detail and would fail on a perfectly valid redesign.

5.3 Step 2: Extract the Features

ID	Feature	From	Priority
F1	Reset: outputs are 0 while `rst_n` is low	R11	High
F2	Handshake: `valid_out` follows `valid_in` by exactly one cycle	R2, R3, R4	High
F3	ADD, including carry	R5	High
F4	SUB, including borrow	R6	High
F5	AND	R7	High
F6	OR	R7	High
F7	XOR	R7	High
F8	NOT	R8	High
F9	SHL, including the bit shifted into bit 8	R9	High
F10	SHR, including the bit shifted out	R10	High
F11	Operand corner values with every operation: 0, `FF`, `a == b`, single bit set	R5 to R10	High
F12	Back-to-back operations with no idle cycle	Q-2	Medium
F13	Idle gaps of varying length; inputs changing while `valid_in` is 0	R2, Q-5	Medium
F14	`b` has no effect on NOT, SHL, SHR	Q-3	Medium
F15	Reset in the middle of an operation, and operation immediately after reset	Q-4	High
F16	Outputs are never X or Z after reset	General	High

5.4 Step 3: The Verification Plan

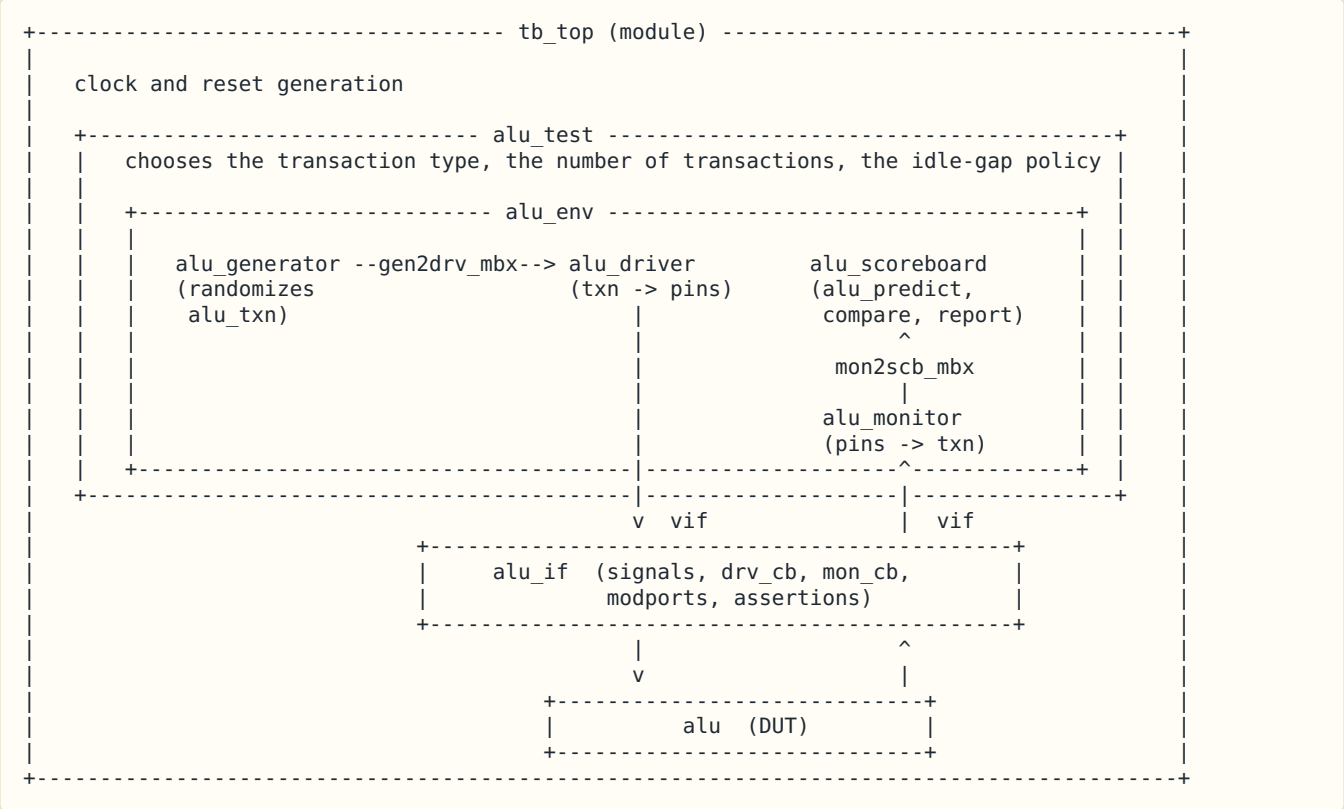
One row per feature: how it will be stimulated, how it will be checked, and what will show that it was exercised. Identifiers beginning with T, C and A refer to the test plan, coverage plan and assertion list in the following sections.

ID	Stimulus	Check	Coverage	Tests
F1	Reset at time 0	A1	C8	T1, T2
F2	Any valid operation; varied gaps	A2, A3	C6	All
F3	`alu_txn` with `op == ALU_ADD`; `alu_carry_txn`	Scoreboard with `alu_predict`	C1, C4, C5	T3, T4, T5
F4	`alu_txn` with `op == ALU_SUB`; constraint `a < b` for borrow	Scoreboard	C1, C4, C5	T3, T4, T5
F5 to F7	`alu_txn` with the logic opcodes	Scoreboard	C1, C4	T3, T4, T6
F8	`alu_txn` with `op == ALU_NOT`	Scoreboard	C1, C4	T3, T4, T6
F9, F10	`alu_shift_txn`	Scoreboard	C1, C4, C5	T3, T4, T7

ID	Stimulus	Check	Coverage	Tests
F11	`corner_c` bias in `alu_txn`; directed corners	Scoreboard	C2, C3, C4	T3, T4
F12	Driver with zero idle cycles between transactions	Scoreboard; A2	C6	T8
F13	Random idle gaps of 0 to 5 cycles; random pin values while idle	Scoreboard; A3	C6	T9
F14	Unary opcodes with unconstrained random `b`	Scoreboard (the model ignores `b`)	C7	T4, T7
F15	Reset asserted at a random time during the test	A1; scoreboard flushed on reset	C8	T10
F16	All tests	A4	n/a	All

Strategy. Block-level simulation with a layered, class-based, constrained-random testbench, plus a small number of directed tests for sanity, reset and named corners. The design is small enough for formal verification as well, which would be listed here as an additional method on a real project.

5.5 Step 4: Testbench Architecture



Component	Responsibility	Uses from `alu_txn`
`alu_generator`	Creates and randomizes transactions; sends them to the driver; signals `gen_done`	`new`, `randomize`, `copy`, `print`
`alu_driver`	Takes a transaction from `gen2drv_mbx` and drives `valid_in`, `a`, `b`, `op` through `vif`	Fields `a`, `b`, `op`; `print`

Component	Responsibility	Uses from `alu_txn`
`alu_monitor`	Samples the pins through `vif`; builds a transaction containing the observed inputs and `result`; sends it to the scoreboard	`new`, fields, `print`
`alu_scoreboard`	Predicts with `alu_predict`, compares, counts, reports	`compare` or field compare, `convert2string`
`alu_env`	Creates the components and mailboxes, connects them, starts them	
`alu_test`	Configures the environment for one scenario	Derived transaction classes
`alu_if`	Bundles the signals; clocking blocks give race-free driving and sampling; holds the assertions	
`tb_top`	Instantiates the DUT and the interface, generates clock and reset, starts the test	

The transaction class of Part 3 is the first piece of this architecture, and the flat testbench of Section 4.9 already contains every function in miniature. Modules 7 and 8 supply the two missing mechanisms (mailboxes and virtual interfaces), and Module 9 builds the complete structure.

5.6 Step 5: The Test Plan

ID	Test name	Objective	Type	Stimulus	Features
T1	`alu_sanity_test`	Design and testbench are alive	Directed	Reset, then one operation of each opcode with simple operands	F1 to F10
T2	`alu_reset_test`	Reset values; first operation after reset	Directed	Reset; check outputs; one operation in the first cycle after release	F1, F15
T3	`alu_corner_test`	Named corner cases	Directed	`FF + FF`, `00 + 00`, `FF + 01`, `00 - 01`, `00 - FF`, `7F - 7F`, `SHL 80`, `SHR 01`, `NOT 00`, `NOT FF`, logic operations with `00`, `FF`, `55`, `AA`	F3 to F11
T4	`alu_random_test`	Broad coverage of all operations and operands	Constrained-random	5,000 `alu_txn` with default constraints	F2 to F14
T5	`alu_arith_test`	Carry and borrow	Constrained-random	1,000 `alu_carry_txn`; 1,000 SUB with `a < b`	F3, F4
T6	`alu_logic_test`	Logic operations with varied bit patterns	Constrained-random	1,000 transactions with `op inside {ALU_AND, ALU_OR, ALU_XOR, ALU_NOT}`, corner bias off	F5 to F8
T7	`alu_shift_test`	Shifts, with `b` random	Constrained-random	1,000 `alu_shift_txn`	F9, F10, F14
T8	`alu_back_to_back_test`	Maximum throughput	Constrained-random	2,000 transactions, zero idle cycles	F12
T9	`alu_idle_gap_test`	Idle behaviour	Constrained-random	1,000 transactions, 0 to 5 idle cycles between them, pins randomized while idle	F13

ID	Test name	Objective	Type	Stimulus	Features
T10	`alu_reset_mid_test`	Reset during operation	Random with directed event	Random traffic; `rst_n` pulsed low at random times, several times	F15

Pass criteria for every test: the scoreboard reports zero mismatches and a non-zero number of comparisons equal to the number of transactions accepted by the DUT; no assertion fails; no `$error` or `$fatal`; the test ends by itself before the timeout.

Notice how few of these need new code. T4 to T9 use one environment; each differs only in which transaction class is given to the generator, how many transactions are requested and one driver setting. That is the pay-off of the reuse built into the transaction in Section 3.4.

5.7 Step 6: The Coverage Plan and the Assertion List

Functional coverage items

ID	Item	Bins	Goal
C1	`op`	One per opcode (8)	All hit
C2	`a`	`00`, `FF`, `01`, `80`, others	All hit
C3	`b`	`00`, `FF`, `01`, `80`, others	All hit
C4	Cross of C1 with C2 and with C3	8 × 5 each	All hit
C5	`result[8]` for ADD, SUB and SHL	0 and 1 for each of the three	All hit
C6	Idle cycles between consecutive operations	0, 1, 2 to 5	All hit
C7	For NOT, SHL, SHR: `b`	`00`, non-zero	All hit
C8	Reset while `valid_in` or `valid_out` is high	Occurred	Hit at least once
C9	`a` compared with `b`, for SUB	`a < b`, `a == b`, `a > b`	All hit

Code coverage goals

Type	Goal
Statement, branch, condition	100 percent; any exclusion needs a written, reviewed justification
Toggle	100 percent on all ports and registers
FSM	Not applicable (no state machine)

Assertions

A preview; the syntax is explained in Module 10. They are placed in `alu_if` so that they are active in every test.

ID	Rule	Property
A1	Outputs are 0 while reset is active	Checked whenever `rst_n` is low
A2	An accepted operation produces `valid_out` in the next cycle	`valid_in |=> valid_out`

ID	Rule	Property
A3	No `valid_out` without a preceding `valid_in`	`!valid_in |=> !valid_out`
A4	Outputs are known	`valid_out` is never X or Z; `result` is not X or Z when `valid_out` is 1

```
// A2 and A3, as they will appear in the interface (Module 10)
assert property (@(posedge clk) disable iff (!rst_n)  valid_in |=>  valid_out);
assert property (@(posedge clk) disable iff (!rst_n) !valid_in |=> !valid_out);
```

Division of labour between the checkers

What is checked	By
Data: is `result` the right value?	Scoreboard and reference model
Timing and protocol: does `valid_out` appear in the right cycle?	Assertions
Was it exercised?	Functional coverage

5.8 Step 7: Build, Bring Up, Run and Debug

Order of work

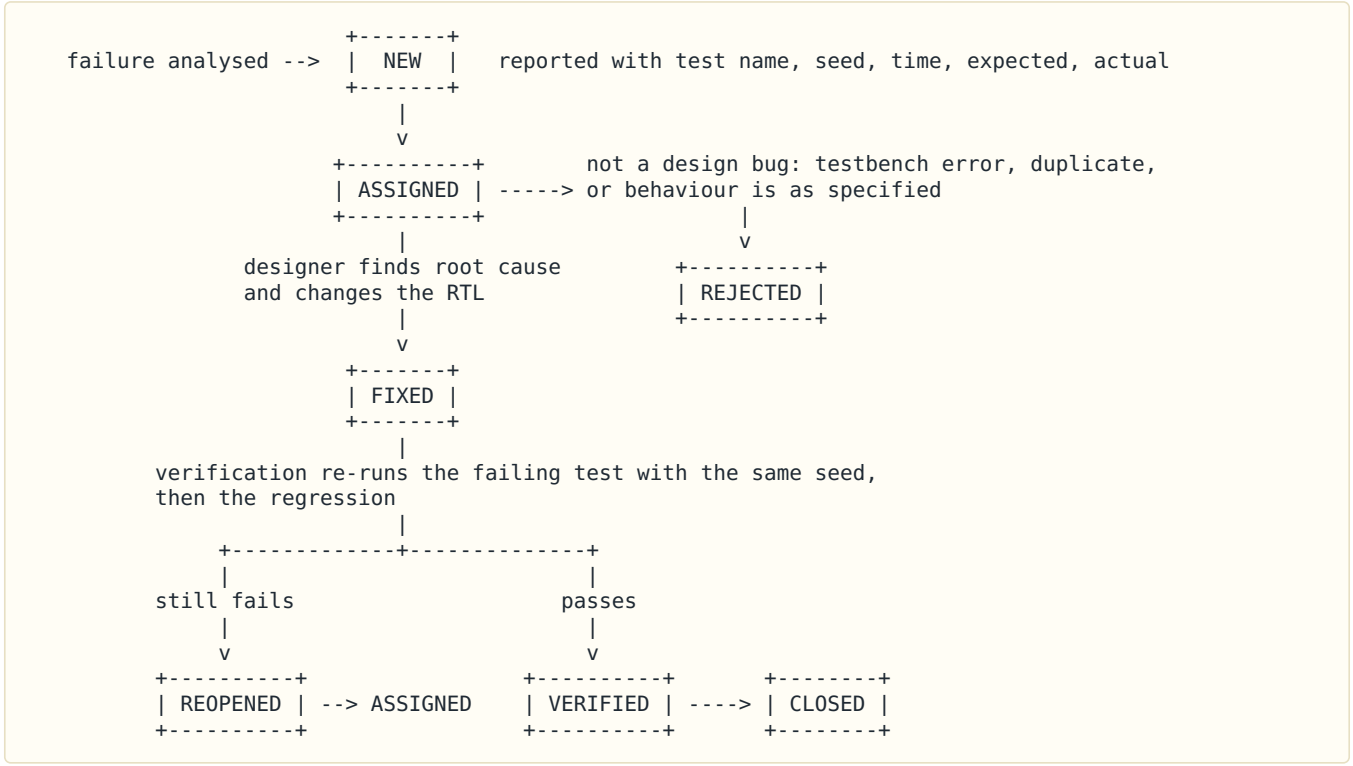
Stage	Activity	Typical first failures
1	Compile the RTL and the testbench together	Port mismatches, missing imports
2	Run T1, the sanity test	Clock or reset not toggling; driver on the wrong edge; monitor never sees `valid_out`
3	Run the remaining directed tests T2, T3	Reference model errors; real design bugs in corner cases
4	Run T4 with one seed, then with several	Constraint failures; scoreboard out of step; design bugs
5	Run the focused random tests T5 to T10	Design bugs in handshake and reset behaviour
6	Switch on coverage collection and look at the first report	Bins that can never be hit; missing bins

In the first days most failures are testbench bugs. That is normal. The sanity test exists so that these are found with a ten-line stimulus instead of a five-thousand-transaction one.

Debugging a failure

1. Read the first error in the log, not the last. Later errors are often consequences.
2. Note the time, the transaction ID and the seed.
3. Re-run with the same seed, with waveform dumping and more verbose printing.
4. Decide which of the three suspects is at fault (Section 4.10): design, testbench or specification.
5. If it is the design, reduce the case to the smallest stimulus that shows it, and file a bug.

The bug life cycle



The exact state names depend on the tracking tool; the sequence does not.

What a good bug report contains

Field	Example
Title	"ALU_SUB gives `b - a` instead of `a - b` when `a != b`"
Design and version	`alu`, RTL revision identifier
Test and seed	`alu_random_test`, seed 1234567
How to reproduce	The exact simulation command
Time of failure	70 ns
Observed	`a=00 b=FF op=ALU_SUB`, `result=0FF`
Expected	`result=101`
Specification reference	R6
Severity / priority	High: wrong functional result
Attachments	Log excerpt, waveform location

After the fix

- Re-run the failing test with the failing seed: it must pass.
- Run the full regression: the fix must not have broken anything else.
- Ask why the bug was not found earlier, and whether a coverage item or a test should be added so that this class of bug cannot return unnoticed.
- Only then close the bug.

> **Interview trap:** "Who closes a bug?" The verification engineer who reported it, after confirming the fix by re-running the failing test and the regression. The designer marks it fixed; they do not close it.

5.9 Step 8: Regression

Definition

A **regression** is the re-running of the complete set of tests, usually automatically and usually every night, to confirm that everything that passed before still passes after the latest changes to the RTL or the testbench. For constrained-random tests, each test is run many times with different **seeds**, so that every run exercises different stimulus.

How a regression is organised

Tier	Contents	Run	Purpose
Sanity / smoke	A handful of short tests, fixed seeds	Before every check-in; minutes	Stop a broken change from entering the repository
Nightly	All tests, several random seeds each	Every night; hours	Detect regressions quickly; accumulate coverage
Weekly / full	All tests, many seeds, coverage merged, sometimes gate-level	Weekly and before milestones	Coverage closure and sign-off data

Seeds

A random test is identified by its name **and** its seed. The regression script chooses new seeds for every run and records them. A failing (test, seed) pair is re-run with the same seed to reproduce the failure. The seed is passed on the simulator command line; the option differs per tool.

Simulator	Fixed seed	Random seed
Synopsys VCS	`+ntb_random_seed=1234`	`+ntb_random_seed_automatic`
Siemens Questa	`-sv_seed 1234`	`-sv_seed random`
Cadence Xcelium	`-svseed 1234`	`-svseed random`

Reproduction requires the same seed, the same simulator version and the same source code. Editing the testbench can change the stimulus a seed produces, which is why a failing seed is debugged before unrelated changes are merged.

A regression report

```
Regression 2026-11-13   RTL rev 41   TB rev 97

test                       seeds   pass   fail
-----------------------    -----   ----   ----
alu_sanity_test                1      1      0
alu_reset_test                 1      1      0
alu_corner_test                1      1      0
alu_random_test               50     49      1     seed 88213704: scoreboard mismatch
alu_arith_test                20     20      0
alu_logic_test                20     20      0
alu_shift_test                20     20      0
alu_back_to_back_test         20     20      0
alu_idle_gap_test             20     20      0
alu_reset_mid_test            20     18      2     seeds 5512, 90417: assertion A3
-----------------------    -----   ----   ----
total                        173    170      3     pass rate 98.3 %

functional coverage 94.2 %    code coverage: stmt 100, branch 100, cond 97.5, toggle 99.1
```

Triage. Each failure is examined and grouped by signature (the first error message). The two `alu_reset_mid_test` failures have the same signature and are almost certainly one bug. Three failing runs here mean two problems to debug.

Interview questions on regression

Q. What is a regression and why is it run? The periodic re-running of all tests, with many seeds, to check that recent changes have not broken previously working behaviour and to accumulate coverage.

Q. Why run the same random test with different seeds? Each seed produces a different sequence of random stimulus. More seeds mean more scenarios explored and more coverage, at no additional coding cost.

Q. A test fails in the regression but passes when you run it. What do you check? Whether you used the same seed, the same code revision, the same simulator version and the same compile and run-time options. A random test with a different seed is a different test.

5.10 Step 9: Coverage Closure

Coverage closure is the process of taking the merged coverage of the regression from "most" to the goal, by analysing every remaining hole.

The procedure

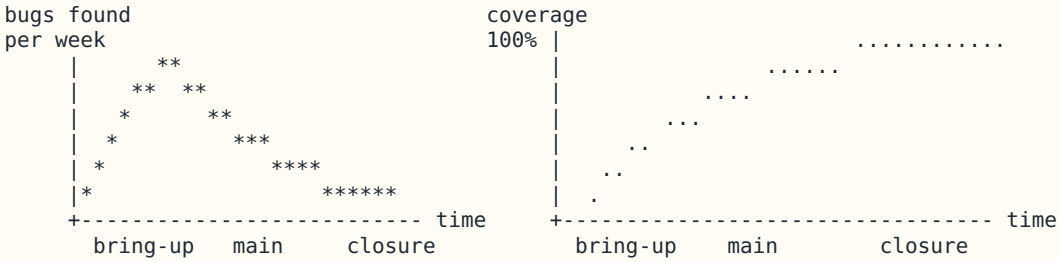
1. Merge the coverage databases of all **passing** runs.
2. List the uncovered functional bins and the uncovered code.
3. For each hole, find the cause and choose the action:

Hole (ALU example)	Cause	Action
Cross bin `ALU_SHR` × `a == 80` never hit	Rare: needs one specific value with one opcode	More seeds, or add `8'h80` and `8'h01` to the `dist` in `corner_c`
C9 bin `a == b` for SUB never hit	Probability 1 in 256 per subtraction, and only a few subtractions per test	Add a derived transaction with `op == ALU_SUB; a == b;`, or a directed test

Hole (ALU example)	Cause	Action
C8, reset while `valid_out` is high	Reset pulses in T10 landed only in idle cycles	Constrain the reset time relative to the traffic
Condition coverage: one sub-expression of the reset logic never false	Can only happen if `rst_n` is X	Exclude, with justification, reviewed by the designer
A coverage bin for an illegal `op` value	All eight encodings are legal; the bin can never exist	The coverage model is wrong: remove the bin

1. Re-run and re-merge. Repeat until the goals are met.

Typical progress



Early in the project bugs are found quickly and coverage rises fast. Toward the end the bug rate falls and coverage levels off. A bug rate that has fallen to near zero while coverage is at its goal is one of the signals that verification is nearing completion. A bug rate that is still high means it is not, whatever the coverage figure says.

> **Common mistake:** Treating coverage closure as "making the number reach 100". The number is a means. Each hole is a question: "why has this not happened?" The answer is sometimes a weak test, sometimes an error in the coverage model, and sometimes a design bug that makes the scenario impossible. Excluding holes without understanding them defeats the purpose.

5.11 Step 10: Sign-Off, or What "Done" Means

Verification can never prove the absence of bugs, so "done" cannot mean "no bugs remain". It means that a set of **exit criteria**, agreed in the verification plan at the start, has been met, and that the remaining risk has been judged acceptable.

Typical exit criteria

Criterion	ALU example
Every feature in the verification plan is implemented in tests, checks and coverage	F1 to F16 each map to at least one test, one check and one coverage item
All tests pass	100 percent pass rate over the full regression with many seeds, for an agreed number of consecutive runs
Functional coverage goal met	C1 to C9 at 100 percent

Criterion	ALU example
Code coverage goal met	Statement, branch, condition and toggle at 100 percent, with every exclusion justified and reviewed
All assertions pass and have been exercised	A1 to A4 never failed, and each was triggered (a property that was never triggered has checked nothing)
No open high-priority bugs	All critical and major bugs closed; any remaining minor ones documented and accepted
Bug discovery rate has levelled off	No new bugs for an agreed period of regressions
Reviews completed	Testbench, checkers, coverage model and exclusions reviewed by the designer and a second verification engineer
The RTL is frozen	The sign-off regression was run on the final version of the code
Results are documented	A report recording all of the above

The sign-off review

At the review the verification engineer presents: the plan and its traceability (requirement → feature → test, check, coverage); regression history; coverage reports with the exclusion list; the bug history; known limitations and anything deliberately not verified at this level. The team decides whether the criteria are met.

> **How to say it in an interview:** "Verification is done when the exit criteria in the plan are met: every planned feature has tests, checks and coverage; the regression passes cleanly over many seeds; functional and code coverage are at their goals with reviewed exclusions; there are no open critical bugs; and the bug rate has flattened. It is a judgement about risk backed by metrics, not a proof that no bugs exist."

Interview questions

Q1. Walk me through how you would verify a block from start to finish. Read the specification and resolve open questions; extract the features; write the verification plan with stimulus, checking and coverage per feature; design the testbench architecture; write the test plan and coverage plan; build the testbench and bring it up with a sanity test; run directed and constrained-random tests, debug failures and file bugs; run regressions with many seeds; analyse and close coverage holes; and sign off against the exit criteria.

Q2. How do you know when verification is complete? When the agreed exit criteria are met: all planned tests implemented and passing in regression, functional and code coverage goals reached with reviewed exclusions, assertions passing and exercised, no open high-priority bugs, and a bug discovery rate that has levelled off.

Q3. What is traceability in a verification plan? The ability to follow each requirement of the specification to the feature that represents it, and from there to the tests that stimulate it, the checkers that verify it and the coverage items that show it was exercised, and back again.

Q4. Describe the life cycle of a bug. The verification engineer reports it (new) with the test, seed and expected versus actual values. It is assigned to the designer, who either rejects it with a reason or fixes it. Verification re-runs the failing test with the same seed and then the regression; if it passes the bug is verified and closed, otherwise it is reopened.

Q5. What information do you include in a bug report? A precise title, design version, test name and seed, the command to reproduce it, the time of failure, the observed and expected behaviour with a reference to the specification, severity, and the location of logs and waveforms.

Q6. What is coverage closure? The work of analysing every uncovered item in the merged coverage results and resolving it, by more seeds, better constraints, directed tests, correcting the coverage model, or a justified exclusion, until the coverage goals are met.

Q7. The designer says "that scenario can never happen, exclude it". What do you do? Ask why it cannot happen and confirm it independently, ideally with an assertion that would fail if it ever did. Record the justification with the exclusion and have it reviewed. If the reasoning depends on how the block is used in the larger system, that assumption must be documented and checked at the level above.

Q8. During verification you find that the specification is ambiguous. What do you do? Raise it with the specification owner, get the intended behaviour decided and recorded in the specification, and update the plan, the reference model and the tests to match. The RTL's current behaviour is not the authority.

Q9. Which checks belong in the scoreboard and which in assertions? Data correctness, which needs a model of the function, belongs in the scoreboard. Timing and protocol rules about signals over clock cycles, such as handshakes and reset behaviour, are best written as assertions, which run in every test and point to the exact cycle of failure.

Q10. Why is the sanity test run first after every change? It is short and exercises the basic path through the DUT and the whole testbench. If it fails, something fundamental is broken, and there is no point in spending hours of compute on the rest of the regression.

Part 6: Module 6 Revision and Practice

6.1 Revision Summary

Transactions

- A transaction is one complete operation on an interface, described as data: **what**, not **how** or **when**.
- The driver converts transaction → pins. The monitor converts pins → transaction. Nothing else in the testbench touches pins.
- Every component uses the transaction, so it is designed first and kept in a package.
- It is a class, not a struct, because it needs `randomize()`, constraints, methods, inheritance and dynamic creation.

The fields of `alu_txn`

Field	Declaration	Kind	Reason for the type
Operands	`rand bit [7:0] a, b;`	Stimulus	Chosen by the testbench; two-state
Operation	`rand alu_op_e op;`	Stimulus	Enum: legal values only, readable, automatic coverage bins
Result	`logic [8:0] result;`	Response	Sampled from the DUT; four-state so that X is preserved
Identifier	`int unsigned id;`	Bookkeeping	Tracks one transaction through the log

Clock, reset and handshake signals are not fields. A virtual interface is never a field.

The methods of a transaction

Method	One-line description	Key point
`new()`	Put the object in a safe initial state	No return type; all arguments have defaults; no randomization inside
`randomize()`	Solve constraints and assign the `rand` fields	Always check the return value
`post_randomize()`	Compute derived fields after randomization	Called automatically; not called on failure
`convert2string()`	Return the contents as a string	Used by `print` and by error messages
`print(tag)`	Display one line in the log	One format for all components
`copy()`	Return a new, independent object with the same values	Handles are references; `t2 = t1` copies nothing
`compare(rhs)`	Return 1 if the relevant fields are equal	`===` for four-state fields; skip bookkeeping; guard against `null`

All are functions, all are `virtual`, and every one must be updated when a field is added.

Constraints in a transaction

- Two purposes: **legality** and **interest** (corner bias, distributions).
- One purpose per named block, so that tests can switch off (`constraint_mode(0)`) or replace (same name in a derived class) exactly one of them.
- Only always-true rules are hard constraints in the base class.
- Tests narrow the stimulus by inline `with { }` or, better, by a derived class.

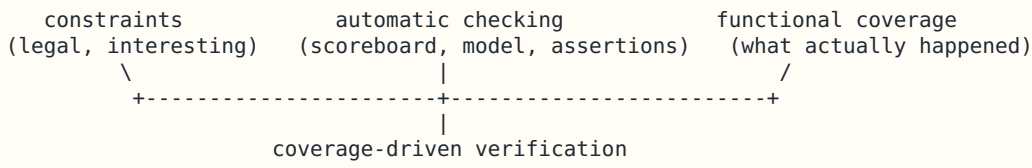
Reuse

- Derived class + new constraint name → constraint **added**.
- Derived class + same constraint name → constraint **replaced**.
- Blueprint pattern: the generator randomizes a base-class handle and sends copies; the test decides which object that handle refers to.

Verification vocabulary

Term	Meaning
Verification	Showing, before tape-out, that the RTL matches the specification
Validation	Showing, after silicon, that the chip works in the real system
Manufacturing test	Checking each fabricated chip for defects
Verification plan	What will be verified and how: features, stimulus, checks, coverage, architecture, exit criteria
Test plan	The list of test cases that implements the verification plan
Directed test	Hand-written stimulus and expected result for one scenario
Constrained-random test	Solver-generated stimulus within constraints, checked automatically, measured by coverage
Functional coverage	User-written measure of which planned scenarios occurred
Code coverage	Tool-generated measure of which RTL code was exercised
Self-checking testbench	Decides pass or fail by itself by comparing actual with expected
Reference model	Independent model that produces the expected output
Scoreboard	Component that compares actual with expected
Regression	Re-running all tests with many seeds, regularly
Coverage closure	Analysing and resolving every coverage hole
Exit criteria	The measurable conditions for sign-off

The three things constrained-random verification needs



Coverage in four lines

	Code coverage	Functional coverage
Source	RTL	Specification
Written by	Tool	Engineer
Finds	Unexercised or dead code	Unexercised scenarios
Cannot find	Missing features	Scenarios missing from the model

Neither shows that anything was **checked**. Sign-off needs both, plus passing checks.

The flow

Specification → features → verification plan → testbench architecture → test plan and coverage plan → bring-up with a sanity test → directed and random tests → debug and bug reports → regression with seeds → coverage closure → sign-off against exit criteria.

6.2 Interview Questions and Answers

The questions inside each section of Parts 1 to 5 cover the direct definitions. The questions below are additional.

Basic level

Q1. In one sentence, what is the job of a transaction class? To hold the data of one operation in a form that can be randomized, passed between testbench components, printed, copied and compared, independently of pins and timing.

Q2. Name the fields you would put in a transaction for a simple RAM. `rand bit write;`, `rand bit [ADDR_W-1:0] addr;`, `rand bit [DATA_W-1:0] wdata;` as stimulus, and `logic [DATA_W-1:0] rdata;` as response.

Q3. Which of `clk`, `rst_n`, `valid_in`, `a`, `op`, `result` belong in `alu_txn`? `a`, `op` and `result`. The others are protocol signals handled by the driver, monitor and interface.

Q4. What does the monitor do with a transaction? It creates a transaction object, fills it with the values it samples from the DUT's pins, and sends it to the scoreboard (and coverage).

Q5. What is the difference between stimulus fields and response fields? Stimulus fields are chosen by the testbench and are `rand`. Response fields are produced by the DUT, captured by the monitor, and are not `rand`.

Q6. What is a testcase? One test from the test plan: a specific configuration and stimulus scenario with defined pass criteria, run on the common testbench.

Q7. What is a DUT? The Design Under Test: the RTL block being verified.

Q8. What is a bug? A difference between what the design does and what the specification requires.

Q9. What is a seed? The starting value of the random number generator. The same seed with the same code reproduces the same random stimulus.

Q10. What does PASS mean for a test? That, for the stimulus applied in that run, no checker reported an error and the expected number of checks was performed. It does not mean the feature is free of bugs.

Q11. What is a coverage hole? A coverage item (bin, line, branch, condition) that no test has hit.

Q12. What is a watchdog or timeout in a testbench? A process that ends the simulation with an error if the test has not finished within a maximum time, so that a deadlock does not run forever.

Intermediate level

Q13. The driver needs to wait a random number of cycles between transactions. Where does that number live? In the transaction, as a `rand` field such as `rand int unsigned delay;` with a constraint. The transaction carries the *intent*; the driver implements it with clock edges. Tests can then control timing by constraining the field.

Q14. Why should `copy()` not simply be `return this;`? That returns a handle to the same object, so nothing has been copied; sender and receiver still share one object.

Q15. Your transaction has a dynamic array of bytes and a handle to a header object. Write the copy logic for both. `c.payload = payload;` copies the array elements. `c.hdr = hdr.copy();` creates a separate header object. Writing `c.hdr = hdr;` would make both transactions share one header.

Q16. Should `compare()` print an error message when the objects differ? Usually not. It should return the verdict; the scoreboard decides whether a difference is an error and how to report it. A separate variant may return a description of the differences for debug.

Q17. Why are response fields not included in constraints? They are not `rand`; the solver treats a non-`rand` variable in a constraint as a constant with its current value. A response field is typically X before the DUT responds, and an X or Z value in a constraint expression is an error.

Q18. What is the difference between a reference model and a scoreboard? The reference model computes the expected output. The scoreboard holds expected and actual values, matches them up and compares them. A scoreboard often contains or calls a reference model.

Q19. Give two reasons to keep directed tests in a constrained-random project. A fast sanity check that the environment works, and deterministic coverage of specific corners (reset values, named boundary cases, reproductions of past bugs).

Q20. Why must coverage from failing tests be discarded? A failing test did not demonstrate correct behaviour; its stimulus may even have been produced by a broken testbench. Counting its coverage would claim that scenarios were verified when they were not.

Q21. What is the difference between a feature and a test? A feature is something the design must do, from the specification. A test is a piece of stimulus that exercises one or more features. One feature may need several tests, and one random test may cover many features.

Q22. How do you decide the priority of the items in a verification plan? By risk: how likely the feature is to be wrong (new logic, complex logic, late changes) and how severe a failure would be (basic function, cannot be worked around in software). High-risk, high-impact items are verified first.

Q23. What is toggle coverage good for? It shows whether every bit of every signal has taken both values. It is particularly useful for connectivity (a bit of a bus tied off or never driven) and at chip level.

Q24. The same test passes with one seed and fails with another. Is the test unstable? No. Different seeds produce different stimulus; one of them reached a scenario that exposes a bug in the design or the testbench. That is exactly what multiple seeds are for.

Advanced and scenario-based

Q25. The scoreboard reports "0 errors" and the test passes, but the designer later finds that the block does nothing at all in that test. What went wrong in the testbench? Nothing was compared. Possible causes: the monitor never detected a valid output; the scoreboard's process was not started or the mailbox was not connected; the test ended before any response arrived. The pass decision must require a non-zero number of comparisons equal to the number of transactions sent, and an empty expected queue at the end.

Q26. Every transaction the driver receives has the same values, although the generator log shows different values. Explain. The generator created one object and sent its handle repeatedly while re-randomizing it. The print in the generator shows each new value at the time, but by the time the driver processes the handles they all refer to the single object with the latest values. The generator must send a new object or a copy each time.

Q27. After adding a `parity` field to the transaction, the scoreboard never reports parity errors even when you force one. Where do you look? At `compare()` (and `copy()`). The new field was probably not added to the comparison, or not copied, so the scoreboard compares default values. All three methods must be updated with every new field.

Q28. You have 100 percent functional coverage in the first week. What do you conclude? That the coverage model is too weak: too few points, bins that are too coarse, no crosses, or coverage sampled without regard to whether the scenario was meaningful. The model is reviewed against the verification plan before the figure is believed.

Q29. A cross-coverage bin has not been hit after 500 seeds. What are your options? Determine whether it is reachable. If it is illegal or impossible, mark it as ignored or illegal with justification. If it is reachable, raise its probability with a `dist` or a derived transaction with tighter constraints, or write a directed test for it. Also ask why it is so hard to reach; sometimes the answer is a design bug that prevents the scenario.

Q30. The reference model and the RTL were written by the same person from the same understanding. What is the risk and how do you reduce it? A common misunderstanding of the specification appears in both, so they agree and the bug is invisible. Reduce it by having a different person write the model from the specification, writing it at a different abstraction level and in a different style, reviewing it against the specification, and adding directed tests whose expected values are worked out by hand.

Q31. A bug is found at chip level that should have been found at block level. What follow-up do you do in the block-level environment? Reproduce it at block level, find out why it was missed (missing feature in the plan, missing coverage point, an over-constraint, a wrong assumption about the inputs, a weak checker), and fix that gap, not just the single case. Then check whether the same gap exists for similar features.

Q32. How would you verify the ALU's requirement that `b` has no effect on NOT, SHL and SHR? Stimulus: leave `b` fully random for those opcodes, never constrain it to 0. Checking: the reference model ignores `b`, so any dependence in the DUT appears as a scoreboard mismatch. Coverage: a cross showing that each unary opcode was seen with zero and with non-zero `b`.

Q33. The specification changes halfway through the project. What do you update, and in what order? The verification plan first (features, checks, coverage), then the transaction and constraints if the interface or legal stimulus changed, the reference model and assertions, the tests and the coverage model; then the regression is re-run and coverage is re-collected from zero for the affected features, because old coverage was measured against old behaviour.

Q34. Why might a team deliberately keep a constraint out of the base transaction even though all current tests need it? Because a hard constraint in the base class applies to every future test. If any later test needs to

violate it (an error test, a corner case), it would have to switch it off by name. Keeping test-level intent in the test, or making it `soft`, keeps the base class open.

Q35. In an interview you are given a 2-to-1 round-robin arbiter and five minutes. What do you say? Interfaces: two requests, two grants, clock, reset. Features: no grant without request; at most one grant at a time; a single requester is granted; with both requesting, grants alternate; no requester waits more than one turn (fairness); reset state. Stimulus: random request patterns, including both held continuously, both dropped together, and requests changing every cycle. Checks: assertions for one-hot-or-zero grant and grant-implies-request; a small model for the alternation. Coverage: all four request combinations, each with each previous grant; long continuous contention.

Q36. What would make you refuse to sign off even though every metric is at 100 percent? Bugs still being found at a steady rate; checkers or the coverage model not reviewed; exclusions without justification; a regression that has only been clean once; RTL changed after the last full regression; or features in the specification with no trace to any test, check or coverage item.

6.3 Output-Prediction Questions

Work out the output on paper first, then read the answer, then run the code. These examples are self-contained and do not need `alu_pkg`.

P1.

```
module p1;
  class txn;
    int a;
    function txn copy();
      txn c = new();
      c.a = a;
      return c;
    endfunction
  endclass

  txn t1, t2, t3;

  initial begin
    t1   = new();
    t1.a = 5;
    t2   = t1;
    t3   = t1.copy();
    t1.a = 9;
    $display("%0d %0d %0d %0d %0d", t1.a, t2.a, t3.a, t1 == t2, t1 == t3);
  end
endmodule
```

Answer: `9 9 5 1 0`. `t2` is a second handle to the same object, so it sees the change to 9. `t3` is a separate object created before the change, so it still holds 5. Handle equality is 1 for `t1`/`t2` and 0 for `t1`/`t3`.

P2.

```
module p2;
  class txn;
    int a;
  endclass

  txn t;
  txn q[$];

  initial begin
    for (int i = 0; i < 3; i++) begin
      if (i != 1) t = new();
      t.a = i;
      q.push_back(t);
    end
    $display("%0d %0d %0d", q[0].a, q[1].a, q[2].a);
  end
endmodule
```

Answer: `1 1 2`. In iteration 0 a new object is created and stored. In iteration 1 no object is created, so the **same** object is modified (`a` becomes 1) and its handle is stored again. In iteration 2 a new object is created. Entries 0 and 1 are one object, which now holds 1.

P3.

```
module p3;
  class txn;
    logic [3:0] r;
    function bit cmp_loose (txn rhs);  return (r ==  rhs.r);  endfunction
    function bit cmp_strict(txn rhs);  return (r === rhs.r);  endfunction
  endclass

  txn t1, t2;

  initial begin
    t1 = new();  t1.r = 4'b1010;
    t2 = new();  t2.r = 4'b10x0;
    $display("%0d %0d", t1.cmp_loose(t2), t1.cmp_strict(t2));
    if (t1.r != t2.r) $display("MISMATCH");
    else              $display("no mismatch reported");
  end
endmodule
```

Answer: `0 0`, then `no mismatch reported`. `1010 == 10x0` is X; returned through a `bit` it becomes 0, so the loose function happens to say "not equal". `===` gives a definite 0. But the inline `if (t1.r != t2.r)` evaluates to X, which is treated as false, so the `else` branch runs and the mismatch is missed. This is why `!==` is used.

P4.

```
module p4;
  class txn;
    rand bit [3:0] a;
    bit      [3:0] b;
    constraint c { a == b + 1; }
  endclass

  txn t;
  int ok;

  initial begin
    t   = new();
    t.b = 4;
    ok  = t.randomize();
    $display("%0d %0d", ok, t.a);
    t.b = 15;
    ok  = t.randomize();
    $display("%0d %0d", ok, t.a);
  end
endmodule
```

Answer: `1 5`, then `0 5`. `b` is not `rand`, so it is a constant for the solver. With `b` = 4, `a` must be 5. With `b` = 15, `b + 1` is evaluated in 32 bits (the literal `1` is 32 bits wide) and equals 16, which a 4-bit `a` can never be. `randomize()` returns 0 and leaves `a` at its previous value. The simulator also prints a constraint-failure message.

P5.

```
module p5;
  class base;
    rand bit [3:0] v;
    constraint range_c { v inside {[0:3]}; }
  endclass

  class d1 extends base;
    constraint range_c { v == 9; }
  endclass

  class d2 extends base;
    constraint other_c { v == 9; }
  endclass

  base b;
  d1   x;
  d2   y;
  int  ok;

  initial begin
    x  = new();
    y  = new();
    b  = x;
    ok = b.randomize();
    $display("%0d %0d", ok, b.v);
    b  = y;
    ok = b.randomize();
    $display("%0d %0d", ok, b.v);
  end
endmodule
```

Answer: `1 9`, then `0 0`. In `d1` the block has the same name as the base block and **replaces** it, so only `v == 9` applies. In `d2` the block has a different name and is **added**: `v` must be in 0 to 3 and also equal 9, which is impossible, so `randomize()` fails and `v` keeps its default of 0. In both cases the call is made through a base-class handle and the constraints of the actual object are used.

P6.

```
module p6;
  class base;
    virtual function string str();   return "base";  endfunction
    function string         label(); return "base";  endfunction
    function void print();
      $display("%s %s", str(), label());
    endfunction
  endclass

  class der extends base;
    virtual function string str();   return {super.str(), "+der"};  endfunction
    function string         label(); return "der";                  endfunction
  endclass

  base b;
  der  d;

  initial begin
    d = new();
    b = d;
    b.print();
    $display("%s %s", b.label(), d.label());
  end
endmodule
```

Answer: `base+der base`, then `base der`. `str()` is virtual, so the derived version runs even though it is called from a base-class method. `label()` is not virtual: inside `base::print` and through the base handle `b`, the base version runs; only through the derived handle `d` does the derived version run. This is why the methods of a transaction are declared `virtual`.

P7.

```
module p7;
  bit   [7:0] a = 8'd5, b = 8'd10;
  logic [8:0] r1, r2;
  initial begin
    r1 = a - b;
    r2 = {1'b0, a - b};
    $display("%h %h", r1, r2);
  end
endmodule
```

Answer: `1fb 0fb`. In `r1 = a - b` the operands are extended to the nine bits of the left-hand side before subtracting, so the borrow appears in bit 8. Inside a concatenation each operand is self-determined: `a - b` is computed in eight bits (`FB`) and the zero is then placed in front. A reference model written the second way would disagree with the DUT on every subtraction that borrows.

6.4 Find-the-Error and Debugging Questions

Compile-time errors

E1.

```
class alu_txn;
  rand bit [7:0] a;
  function void new();
    a = 0;
  endfunction
endclass
```

A constructor has no return type. Write `function new();`.

E2.

```
if (!t.randomize() with (a < b)) $fatal(1, "failed");
```

Inline constraints are written in braces, each ending with a semicolon: `t.randomize() with { a < b; }`.

E3.

```
alu_txn t;
initial t = new alu_txn();
```

The class name is not repeated after `new`. Write `t = new();`. (`new t2` with a handle is the shallow-copy syntax, which is something else.)

E4.

```
class alu_txn;
  rand real gain;
endclass
```

In IEEE 1800-2017, `rand` may be applied only to integral types (and to arrays, structs and handles built from them). A real cannot be randomized this way. Randomize an integer and scale it in `post_randomize()`.

E5.

```
class alu_txn;
  rand bit [7:0] a;
  constraint a_c { a inside [1:10]; }
endclass
```

The set after `inside` must be in braces: `a inside {[1:10]};`.

Run-time errors and logic bugs (the code compiles)

D1.

```
class alu_txn;
  rand bit [7:0] a;
  function new(bit [7:0] a = 0);
    a = a;
  endfunction
endclass
```

The argument hides the property, so the statement assigns the argument to itself. The property is never set. Write `this.a = a;`.

D2.

```
alu_txn t;
initial begin
  t.a = 8'd5;
end
```

`t` is `null`; no object was created. The simulation stops with a null-handle error. Add `t = new();`.

D3.

```
function alu_txn copy();
  copy.a  = a;
  copy.b  = b;
  copy.op = op;
endfunction
```

The implicit return variable `copy` is a handle and starts as `null`; accessing `copy.a` is a null-handle error. An object must be constructed first: `alu_txn c = new(); ... return c;`.

D4.

```
function bit compare(alu_txn rhs);
  return (this == rhs);
endfunction
```

This compares handles, so it returns 1 only when both refer to the very same object. Compare the fields.

D5.

```
alu_txn t = new();
repeat (10) begin
  void'(t.randomize());
  gen2drv_mbx.put(t);
end
```

Two bugs. The return value of `randomize()` is discarded, so a failure is invisible. And the same handle is put into the mailbox ten times, so the driver sees ten references to one object. Check the return value, and create a new object (or put `t.copy()`) for each transaction.

D6.

```
class alu_txn;
  rand bit [7:0] a, b;
  logic [8:0]    result;
  constraint sum_c { result == a + b; }
endclass
```

`result` is not `rand`, so the solver treats it as a constant with its current value, which is X. An X in a constraint is an error and `randomize()` fails. The expected result is computed after randomization, by the reference model, not by a constraint.

D7.

```
$display("op = %s", t.op);
```

`%s` with an enum variable prints the numeric value interpreted as characters, not the name. Use `t.op.name()`.

D8.

```
class alu_txn;
  rand alu_op_e op;
  constraint op_c { op dist {ALU_ADD := 1, ALU_SUB := 1}; }
endclass
...
if (!t.randomize() with { op == ALU_XOR; }) $fatal(1, "failed");
```

The `dist` allows only ADD and SUB; XOR is not in the list, so the combined constraints have no solution and the test stops. Give every legal opcode a non-zero weight in the base class, or switch `op_c` off for this test.

D9.

```
if (n_err == 0) $display("TEST PASSED");
```

A test in which no comparison was ever made also has zero errors. Require that checks were performed: `if (n_err == 0 && n_checked > 0 && n_checked == n_sent)`.

D10.

```
class alu_txn;
  virtual alu_if vif;
  rand bit [7:0] a, b;
  task drive();
    @(vif.drv_cb);
    vif.drv_cb.a <= a;
  endtask
endclass
```

Not a language error but a design error. The transaction now depends on the interface and contains timing, so it cannot be created by a monitor, stored by a scoreboard or reused elsewhere without dragging the interface along. Driving belongs in the driver.

D11.

```
class alu_delay_txn extends alu_txn;
  rand int unsigned delay;
  function string convert2string();
    return $sformatf("delay=%0d", delay);
  endfunction
endclass
```

The override discards the base fields; the log now shows only the delay. Call the base version: `return {super.convert2string(), $sformatf(" delay=%0d", delay)};`. (If the base method is `virtual`, the override is virtual too, even without the keyword; writing `virtual` again is good practice.)

D12. A regression log shows this for a failing test, and the engineer files a design bug immediately:

```
MISMATCH {id=41 op=ALU_NOT a=0x0f b=0x33 result=0x0f0} expected=0x1f0
```

Work it out by hand first. `~0F` is `F0` and bit 8 must be 0, so the correct result is `0F0`. The DUT is right and the **reference model** is wrong: it inverted a nine-bit value (`~{1'b0, a}`) instead of placing a zero in front of the eight-bit inversion. Always check both sides against the specification before filing.

6.5 Coding Exercises with Solutions

Try each exercise before reading the solution. Exercises 2 to 5 use the final `alu_txn` class; compile `alu_pkg.sv` and `alu_tb_pkg.sv` (with `alu_txn.sv` in the include path) first.

Exercise 1: a transaction class for a FIFO

Write a complete transaction class for the synchronous FIFO of Section 4.4. One transaction is one clock cycle of activity: idle, write, read, or simultaneous write and read. Include constraints, helper methods, and `convert2string`, `print`, `copy` and `compare`.

ex01_fifo_txn.sv

```
module ex01_fifo_txn;

  typedef enum bit [1:0] {FIFO_IDLE, FIFO_WRITE, FIFO_READ, FIFO_WRITE_READ} fifo_kind_e;

  class fifo_txn;
    // stimulus
    rand fifo_kind_e kind;
    rand bit [7:0]   din;
    // response
    logic [7:0]      dout;
    logic            full;
    logic            empty;
    // bookkeeping
    int unsigned     id;

    constraint kind_dist_c {
      kind dist { FIFO_IDLE := 1, FIFO_WRITE := 4, FIFO_READ := 4, FIFO_WRITE_READ := 1 };
    }

    function bit is_write();
      return kind inside {FIFO_WRITE, FIFO_WRITE_READ};
    endfunction

    function bit is_read();
      return kind inside {FIFO_READ, FIFO_WRITE_READ};
    endfunction
```

```
    virtual function string convert2string();
      return $sformatf("id=%0d kind=%s din=0x%h dout=0x%h full=%b empty=%b",
                       id, kind.name(), din, dout, full, empty);
    endfunction

    virtual function void print(string tag = "");
      $display("%s: %s", tag, convert2string());
    endfunction

    virtual function fifo_txn copy();
      fifo_txn c = new();
      c.kind  = kind;
      c.din   = din;
      c.dout  = dout;
      c.full  = full;
      c.empty = empty;
      c.id    = id;
      return c;
    endfunction

    virtual function bit compare(fifo_txn rhs);
      if (rhs == null) return 0;
      return (kind  ==  rhs.kind)  &&
             (din   ==  rhs.din)   &&
             (dout  === rhs.dout)  &&
             (full  === rhs.full)  &&
             (empty === rhs.empty);
    endfunction
  endclass

  fifo_txn t, c;

  initial begin
    t       = new();
    t.id    = 3;
    t.kind  = FIFO_WRITE;
    t.din   = 8'hA5;
    t.dout  = 8'h00;
    t.full  = 1'b0;
    t.empty = 1'b1;
    t.print("WR");

    c = t.copy();
    $display("copy equal: %0d   is_write=%0d is_read=%0d",
             t.compare(c), t.is_write(), t.is_read());

    c.full = 1'bx;
    $display("after corrupting full: %0d", t.compare(c));
  end

endmodule
```

EXPECTED OUTPUT

```
WR: id=3 kind=FIFO_WRITE din=0xa5 dout=0x00 full=0 empty=1
copy equal: 1   is_write=1 is_read=0
after corrupting full: 0
```

The structure is the same as `alu_txn`: stimulus fields `rand` and two-state, response fields four-state, `id` not compared, `===` on everything sampled from the DUT. `kind` has a two-state base type here, so `==` is sufficient for it.

Exercise 2: extend the transaction with a delay field

Create `alu_delay_txn`, which adds a random idle-cycle count of 0 to 5. Make `print` and `copy` handle the new field correctly when the object is used through an `alu_txn` handle.

ex02_delay_txn.sv

```
module ex02_delay_txn;
  import alu_pkg::*;
  import alu_tb_pkg::*;

  class alu_delay_txn extends alu_txn;
    rand int unsigned delay;

    constraint delay_c { delay inside {[0:5]}; }

    virtual function string convert2string();
      return {super.convert2string(), $sformatf(" delay=%0d", delay)};
    endfunction

    virtual function alu_txn copy();
      alu_delay_txn c = new();
      c.a      = a;
      c.b      = b;
      c.op     = op;
      c.result = result;
      c.id     = id;
      c.delay  = delay;
      return c;
    endfunction
  endclass

  alu_delay_txn d, d2;
  alu_txn       base_h, base_copy;

  initial begin
    d       = new();
    d.id    = 7;
    d.op    = ALU_OR;
    d.a     = 8'h0F;
    d.b     = 8'hF0;
    d.delay = 3;

    base_h    = d;                    // base-class handle to the derived object
    base_copy = base_h.copy();        // virtual: the derived copy() runs
    base_copy.print("COPY");          // virtual: the derived convert2string() runs

    if ($cast(d2, base_copy))
      $display("the copy is an alu_delay_txn, delay=%0d", d2.delay);

    base_copy = new();                // a plain alu_txn
    if (!$cast(d2, base_copy))
      $display("a plain alu_txn cannot be cast to alu_delay_txn");
  end

endmodule
```

EXPECTED OUTPUT

```
[0] COPY: id=7 op=ALU_OR a=0x0f b=0xf0 result=0xxxx delay=3
the copy is an alu_delay_txn, delay=3
a plain alu_txn cannot be cast to alu_delay_txn
```

`print()` is inherited unchanged; it calls `convert2string()`, which is virtual, so the derived text is used. `copy()` creates a derived object and returns it through the base return type. `$cast` recovers a derived handle when the object really is of the derived type, and fails cleanly when it is not.

Exercise 3: pack and unpack

A serial version of the ALU receives its stimulus as one 19-bit word `{op, a, b}`. Write functions that pack an `alu_txn` into that word and rebuild a transaction from it.

ex03_pack_unpack.sv

```
module ex03_pack_unpack;
  import alu_pkg::*;
  import alu_tb_pkg::*;

  typedef bit [18:0] alu_bits_t;

  function automatic alu_bits_t pack_txn(alu_txn t);
    return {t.op, t.a, t.b};
  endfunction

  function automatic alu_txn unpack_txn(alu_bits_t bits);
    alu_txn t = new();
    if (!$cast(t.op, bits[18:16])) $error("illegal opcode %b", bits[18:16]);
    t.a = bits[15:8];
    t.b = bits[7:0];
    return t;
  endfunction

  alu_txn    t, u;
  alu_bits_t w;

  initial begin
    t    = new();
    t.op = ALU_XOR;
    t.a  = 8'hAA;
    t.b  = 8'h0F;

    w = pack_txn(t);
    $display("packed           : %h", w);

    u = unpack_txn(w);
    $display("unpacked         : %s", u.convert2string());
    $display("round trip equal : %0d", t.compare(u));
  end

endmodule
```

EXPECTED OUTPUT

```
packed           : 4aa0f
unpacked         : id=0 op=ALU_XOR a=0xaa b=0x0f result=0xxxx
round trip equal : 1
```

`ALU_XOR` is `100`, so the word is `100_1010_1010_0000_1111`, which is `4AA0F` when written as five hex digits. `$cast` is used for the opcode so that an illegal encoding would be reported instead of silently stored. The two objects compare equal because `compare()` uses `===` and both results are X.

Exercise 4: an in-order scoreboard

Write the checking part of a scoreboard as functions: one receives each stimulus transaction and stores the expected result, one receives each observed transaction and compares, one reports at the end. Exercise it with a correct result, a wrong result, an X result, an unexpected output and a missing output.

ex04_scoreboard.sv

```
module ex04_scoreboard;
  import alu_pkg::*;
  import alu_tb_pkg::*;

  alu_txn      exp_q[$];
  int unsigned n_pass, n_fail;

  function automatic void write_expected(alu_txn t);
    alu_txn e = t.copy();
    e.result = alu_predict(e);
    exp_q.push_back(e);
```

```
  endfunction

  function automatic void write_actual(alu_txn act);
    alu_txn e;
    if (exp_q.size() == 0) begin
      n_fail++;
      $display("FAIL unexpected {%s}", act.convert2string());
      return;
    end
    e = exp_q.pop_front();
    if (e.compare(act)) begin
      n_pass++;
      $display("PASS {%s}", act.convert2string());
    end
    else begin
      n_fail++;
      $display("FAIL expected {%s}", e.convert2string());
      $display("     actual   {%s}", act.convert2string());
    end
  endfunction

  function automatic void report();
    if (exp_q.size() != 0) begin
      n_fail++;
      $display("FAIL %0d expected transaction(s) never observed", exp_q.size());
    end
    $display("scoreboard: %0d passed, %0d failed", n_pass, n_fail);
    if (n_fail == 0 && n_pass > 0) $display("TEST PASSED");
    else                           $display("TEST FAILED");
  endfunction

  function automatic alu_txn make(int unsigned id, alu_op_e op, bit [7:0] a, bit [7:0] b);
    alu_txn t = new();
    t.id = id;
    t.op = op;
    t.a  = a;
    t.b  = b;
    return t;
  endfunction

  alu_txn s, d;

  initial begin
    s = make(0, ALU_ADD, 8'd200, 8'd100);  write_expected(s);
    d = s.copy();  d.result = 9'h12C;      write_actual(d);     // correct

    s = make(1, ALU_SUB, 8'd5, 8'd10);     write_expected(s);
    d = s.copy();  d.result = 9'h0FB;      write_actual(d);     // borrow bit lost

    s = make(2, ALU_SHL, 8'h81, 8'h00);    write_expected(s);
    d = s.copy();  d.result = 'x;          write_actual(d);     // output is X

    d = make(3, ALU_OR, 8'h01, 8'h02);
    d.result = 9'h003;                     write_actual(d);     // nothing was expected

    s = make(4, ALU_AND, 8'hFF, 8'h0F);    write_expected(s);   // never observed

    report();
  end

endmodule
```

EXPECTED OUTPUT

```
PASS {id=0 op=ALU_ADD a=0xc8 b=0x64 result=0x12c}
FAIL expected {id=1 op=ALU_SUB a=0x05 b=0x0a result=0x1fb}
     actual   {id=1 op=ALU_SUB a=0x05 b=0x0a result=0x0fb}
FAIL expected {id=2 op=ALU_SHL a=0x81 b=0x00 result=0x102}
     actual   {id=2 op=ALU_SHL a=0x81 b=0x00 result=0xxxx}
FAIL unexpected {id=3 op=ALU_OR a=0x01 b=0x02 result=0x003}
FAIL 1 expected transaction(s) never observed
scoreboard: 1 passed, 4 failed
TEST FAILED
```

Five different situations, five correct verdicts. The last two are the ones most often forgotten: an output that nobody expected, and an expected output that never came. `$display` is used instead of `$error` only so that the output is identical on every simulator.

Exercise 5: a test-specific transaction that checks itself

The coverage plan needs subtractions with `a == b`. Write a derived transaction for it, generate 100, and confirm that every one is a subtraction of equal operands whose predicted result is zero.

ex05_equal_txn.sv

```
module ex05_equal_txn;
  import alu_pkg::*;
  import alu_tb_pkg::*;

  class alu_equal_txn extends alu_txn;
    constraint equal_c { op == ALU_SUB; a == b; }
  endclass

  alu_equal_txn t;
  int unsigned  n_ok;

  initial begin
    t = new();
    repeat (100) begin
      if (!t.randomize()) $fatal(1, "randomize() failed");
      if (t.op == ALU_SUB && t.a == t.b && alu_predict(t) === 9'h000) n_ok++;
    end
    $display("%0d of 100: SUB with a == b and predicted result 0", n_ok);
  end

endmodule
```

EXPECTED OUTPUT

```
100 of 100: SUB with a == b and predicted result 0
```

`equal_c` is added to the inherited `op_dist_c` and `corner_c`; all three can be satisfied together. One object is randomized repeatedly here, which is safe because its handle is not passed to anything else.

Exercise 6 (paper exercise): plan the verification of a counter

Specification. A 4-bit up/down counter. Ports: `clk`, `rst_n` (asynchronous, active low), `en`, `up`, `load`, `din[3:0]`, `count[3:0]`. On a rising clock edge: if `load` is 1, `count` takes `din`; otherwise, if `en` is 1, `count` increments when `up` is 1 and decrements when `up` is 0; otherwise it holds. The counter wraps in both directions. Reset sets `count` to 0. `load` has priority over `en`.

Write the feature list, the transaction, the checks, the coverage items and the test list. A model answer:

Features

ID	Feature
F1	Reset sets `count` to 0
F2	Count up when `en` = 1, `up` = 1
F3	Count down when `en` = 1, `up` = 0
F4	Hold when `en` = 0 and `load` = 0
F5	Load `din` when `load` = 1
F6	`load` has priority over `en`
F7	Wrap from 15 to 0 when counting up
F8	Wrap from 0 to 15 when counting down
F9	Change of direction at any value, including 0 and 15
F10	Reset during counting

Transaction. One transaction is one clock cycle: `rand bit en, up, load; rand bit [3:0] din; logic [3:0] count;`. Constraints: `load` asserted about 10 percent of the time; `din` biased toward 0 and 15; a derived transaction with `en == 1` and `up` held for long runs, so that the wraps are reached.

Checks. A reference model in the scoreboard: a 4-bit variable updated every cycle by the same rules (`if (load) m = din; else if (en) m = up ? m + 1 : m - 1;`), compared with `count` using `!==` on every clock edge. Assertions: `count` is 0 while `rst_n` is low; `count` is never X after reset; `count` is stable when `en` and `load` are both 0.

Coverage

Item	Bins
`count`	All 16 values
Operation	up, down, hold, load
Cross operation × `count` at 0 and 15	Includes both wraps and loads at the limits
`load` and `en` both 1	Occurred
`din` on load	0, 15, others
Direction change	up→down and down→up, at 0, at 15, elsewhere
Reset while counting	Occurred

Tests

Test	Type	Purpose
`cnt_sanity_test`	Directed	Reset, count up five, count down five
`cnt_wrap_test`	Directed	Up through 15 → 0; down through 0 → 15
`cnt_load_test`	Directed	Load 0, 15 and a middle value; load with `en` = 1
`cnt_random_test`	Constrained-random	All controls random, many cycles, many seeds
`cnt_long_run_test`	Constrained-random	Long runs in one direction to reach the wraps repeatedly

Test	Type	Purpose
`cnt_reset_test`	Random with directed reset	Reset at random times

Exit criteria. All tests passing over many seeds; all coverage items at 100 percent; statement, branch, condition and toggle coverage at 100 percent; no open bugs.

Further practice (no solutions given)

1. Add a `rand bit inject_x;` field to `alu_txn` meaning "the driver shall drive X on `b` for this transaction". Which methods must change? Which component acts on it? What must the scoreboard do for unary and for binary operations?
2. Write `compare_report()` from Section 3.3 into `alu_txn` and use it in the Exercise 4 scoreboard so that only the differing fields are printed.
3. Write a transaction class for an APB-like bus: `write`, `addr[7:0]`, `wdata[31:0]`, `rdata[31:0]`, `slverr`. Constrain the address to be word-aligned and add a derived class that generates only writes to addresses `0x00` to `0x1C`.
4. Change `tb_alu_selfcheck.sv` so that a second directed call follows the first with no idle cycle between them (back-to-back). What must change in `drive_and_check`?
5. Insert each of these bugs into `alu.sv`, one at a time, and record which tests of `tb_alu_selfcheck.sv` (directed, random, or both) detect it: `ALU_ADD` result truncated to eight bits; `ALU_SHR` shifting left; `valid_out` permanently 1; `ALU_NOT` inverting `b` instead of `a`.
6. Write the verification plan tables (features, checks, coverage, tests) for a 2-to-1 round-robin arbiter, using the outline in Q35.
7. For the FIFO scenarios of Section 4.4, decide for each one whether a scoreboard, an assertion or both should check it, and write one sentence of justification.

6.6 Concepts You Must Understand Before Moving Forward

Do not start Module 7 until you can answer each of these without looking at the notes.

Transactions

- [ ] I can explain what a transaction is and why the testbench works at transaction level.
- [ ] I can sort the ports of a DUT into stimulus fields, response fields and protocol signals.
- [ ] I can explain why `a` and `b` are `rand bit` and `result` is a non-`rand` `logic`.
- [ ] I can write `new`, `convert2string`, `print`, `copy` and `compare` for a transaction from memory.
- [ ] I can explain the shared-handle bug and two ways to avoid it.
- [ ] I can explain why `compare` uses `===`, skips `id`, and checks for `null`.
- [ ] I can explain why the transaction methods are `virtual`.

Randomization and reuse

- [ ] I always check the return value of `randomize()` and can say what happens when it fails.
- [ ] I can explain the two purposes of constraints and why they are split into named blocks.
- [ ] I can create a test-specific transaction by extension, and I know when a derived constraint adds and when it replaces.

- [ ] I can describe the blueprint pattern.

Verification planning

- [ ] I can define verification, validation and manufacturing test.
- [ ] I can list the contents of a verification plan and explain how it differs from a test plan.
- [ ] I can take a small specification and derive scenarios category by category.
- [ ] I can compare directed and constrained-random testing and say when each is used.
- [ ] I can compare code coverage and functional coverage and interpret the four combinations of high and low.
- [ ] I can list what makes a testbench self-checking and four ways a test can falsely pass.
- [ ] I can explain where the expected value comes from and why the scoreboard uses the monitor's view of the inputs.
- [ ] I can describe the complete flow from specification to sign-off, the bug life cycle, and the exit criteria.

6.7 What Comes Next

The transaction now exists, and you know what the testbench around it has to achieve. Two mechanisms are still missing before the layered testbench can be built.

- **Module 7** covers inter-process communication: events, `fork`...`join`, mailboxes and semaphores. It answers "how does the generator hand an `alu_txn` to the driver while both are running in parallel?" with `gen2drv_mbx`, and "how does the test know the generator has finished?" with `gen_done`.
- **Module 8** covers interfaces, clocking blocks and virtual interfaces. It answers "how does a class reach the DUT's pins?" with `vif`.
- **Module 9** puts everything together: `alu_generator`, `alu_driver`, `alu_monitor`, `alu_scoreboard`, `alu_env` and `alu_test`, built around the `alu_txn` class and the verification plan from this module.